# PARKLINE: MULTI-LEVEL PARKING OPERATIONS AND CAPACITY PLANNING SYSTEM

**A PROJECT REPORT SUBMITTED TO**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**

**IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF**

## BACHELOR OF COMPUTER APPLICATIONS

**BY**

**[STUDENT NAME]**

**REG. NO. [REGISTER NUMBER]**

**UNDER THE GUIDANCE OF**

**[PROJECT GUIDE NAME] [QUALIFICATIONS]**

**[GUIDE DESIGNATION], SRMIST-DDE**

**DEPARTMENT OF COMPUTER APPLICATIONS**
**DIRECTORATE OF DISTANCE EDUCATION**
**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**
**(Deemed to be university u/s 3 of UGC Act, 1956)**
**Kattankulathur – 603 203**
**Chennai, Tamil Nadu**
**NOVEMBER 2026**

# BONAFIDE CERTIFICATE

This is to certify that the project report titled **"PARKLINE: MULTI-LEVEL PARKING OPERATIONS AND CAPACITY PLANNING SYSTEM"** is a bonafide work carried out by **[STUDENT NAME] ([REGISTER NUMBER])** under my supervision for the award of the Degree of Bachelor of Computer Applications. To my knowledge, the work reported herein is the original work of the candidate.

| **PROJECT GUIDE** | **HEAD OF DEPARTMENT** |
|---|---|
| [PROJECT GUIDE NAME]; [GUIDE DESIGNATION] | [HEAD / PROGRAMME COORDINATOR NAME]; Department of Computer Applications |
| Signature: _____ | Signature: _____ |

| **INTERNAL EXAMINER** | **EXTERNAL EXAMINER** |
|---|---|
| Signature: _____ | Signature: _____ |

*Submission template: complete candidate and signatory details and obtain actual institutional approval. No signature or approval is asserted by this generated document.*

# DECLARATION

I, **[STUDENT NAME] ([REGISTER NUMBER])**, hereby declare that the project report entitled **"PARKLINE: MULTI-LEVEL PARKING OPERATIONS AND CAPACITY PLANNING SYSTEM"**, submitted to SRM Institute of Science and Technology in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications, is a record of original software engineering and academic project work carried out by me under the supervision and guidance of **[PROJECT GUIDE NAME]**, [GUIDE DESIGNATION], Department of Computer Applications, Directorate of Distance Education.

I further declare that this report or any part thereof has not been submitted earlier to another university or institution for the award of a degree, diploma or fellowship. Source materials, software dependencies and the supplied formatting reference are acknowledged in the report.

[STUDENT NAME]

[REGISTER NUMBER]

Signature of the Candidate: _____

Place: Chennai

Date: November 2026

*Candidate review is required before signing. Personal details are intentionally left as editable placeholders.*

# ACKNOWLEDGEMENT

I express my sincere gratitude to SRM Institute of Science and Technology and the Directorate of Distance Education for providing the academic opportunity to undertake this application-development project.

I thank the Department of Computer Applications, the programme coordinator and the faculty members for the learning environment and project-review framework that support the development of analytical, programming and documentation skills.

I extend my gratitude to my project guide, **[PROJECT GUIDE NAME]**, for guidance, technical review and encouragement during the analysis, design, implementation and documentation of this project.

I am grateful to my family, friends and peers for their encouragement throughout the preparation of the project and its documentation. I also acknowledge the maintainers of the open-source frameworks, libraries and verification tools used by the application.

The supplied college project report was used as a reference for academic structure and visual formatting. Its unrelated implementation claims and personal identities were not transferred into this report. All technical descriptions of Parkline are grounded in the reviewed code and the stated evidence boundary.

[STUDENT NAME]

# ABSTRACT

Managing parking across multiple floors requires a consistent view of individual bays, active vehicles and planned capacity. Disconnected records can confuse reserved inventory with availability, permit duplicate assignments or lose occupied resources during capacity changes. This project presents Parkline, a browser-based multi-level parking operations and capacity planning application that makes these relationships visible and testable.

The application is implemented with Next.js 16.4.3, React 19.0.0, TypeScript and a Tailwind/CSS styling pipeline. Typed fixtures provide four demonstration garage locations. A React dashboard supports location and floor selection, bay search and filtering, vehicle arrival, active-session departure, protected floor resizing and configurable new locations. Pure domain helpers enforce available-bay allocation, normalized Tamil Nadu registration input, location-local duplicate-session rejection and preservation of occupied or reserved bays during capacity reduction.

Planning and operational changes are serialized into browser local storage. This is device-local demo persistence, not a shared database. The current application has no authentication, role-based authorization, payment processing, reservation lifecycle, gate control or live sensor connection. Static interface labels and sample trends are distinguished from derived occupancy data throughout the report.

The reviewed revision passed nine existing unit tests, linting, type checking and a production build. A complete run of three existing browser scenarios produced two passes and one reload-persistence failure. Sixteen supplemental documentation assertions produced fifteen passes and one narrow-screen overflow failure. Genuine screenshots, source-grounded diagrams, requirements, a data dictionary and an open-findings register document both the working MVP and its limitations.

The project demonstrates a practical local state model and an evidence-based engineering workflow. It provides a foundation for further hardening and later server-backed operations, while avoiding unsupported claims of production readiness or physical deployment.

*Keywords: Multi-Level Parking; Capacity Planning; React; Next.js; TypeScript; Browser Local Storage; State Transitions; Vehicle Sessions; Vitest; Playwright.*

# TABLE OF CONTENTS


Title Page i
Bonafide Certificate ii
Declaration iii
Acknowledgement iv
Abstract v
Table of Contents vi
List of Figures x
List of Tables xi
Abbreviations / Acronyms xiii

**CHAPTER 1 — INTRODUCTION** 1

1.1 Overview of the Project 1
1.2 Background of the Study 2
1.3 Problem Statement 3
1.4 Objectives of the Project 4
1.5 Scope of the Project 5
1.6 Project Motivation 6
1.7 Assumptions and Constraints 7
1.8 Organization of the Report 8
1.9 Documentation Method and Evidence Boundary 8

**CHAPTER 2 — LITERATURE REVIEW** 9

2.1 Introduction and Review Method 9
2.2 Review of Existing Approaches and Related Guidance 10
2.2.1 Manual Registers and Spreadsheet-Based Tracking 10
2.2.2 Component-Based Interactive Web Applications 11
2.2.3 Typed Domain Models and Immutable Transformations 12
2.2.4 Browser Storage and Persistence Semantics 13
2.2.5 Accessible Status-Oriented Interfaces 14
2.2.6 Visual Design and Operational Honesty 14
2.2.7 Automated Verification and Test Layering 15
2.3 Comparative Study of Related Approaches 16
2.4 Gaps Identified 16
2.5 Summary of Literature Review 16

**CHAPTER 3 — SYSTEM ANALYSIS** 17

3.1 Existing Workflow Analysis 17
3.2 Limitations of a Disconnected Tracking Workflow 18
3.3 Proposed System and Operator Workflow 19
3.4 Advantages within the Defined Scope 19
3.5 Software Requirements Specification 20

# TABLE OF CONTENTS (CONTINUED)


3.5.1 Functional Requirements — Operational Dashboard 20
3.5.2 Functional Requirements — Planning and Local Persistence 21
3.5.3 Non-Functional Requirements and Quality Expectations 22
3.6 Feasibility Study 23
3.6.1 Technical Feasibility 23
3.6.2 Economic Feasibility 24
3.6.3 Operational Feasibility 24
3.7 Input Validation and Error Analysis 25
3.8 Risk Assessment and Acceptance Traceability 26
**CHAPTER 4 — SYSTEM DESIGN** 27
4.1 Design Overview and Architectural Principles 27
4.2 System Architecture 28
4.3 Data Flow Diagrams 29
4.3.1 Level 0 — System Context 29
4.3.2 Level 1 — Functional Decomposition 30
4.3.3 Level 2 — Vehicle Arrival 31
4.4 Use Case Diagram and Actor Specification 32
4.4.1 Detailed Use Case — Inspect and Start a Session 33
4.4.2 Detailed Use Case — Complete a Departure 34
4.4.3 Detailed Use Cases — Capacity and Location Planning 35
4.5 Object-Oriented and Behavioral Models 36
4.5.1 Domain Interface / UML Class View 36
4.5.2 Sequence Model — Arrival and Local Save 37
4.5.3 Bay State Machine and Invariants 38
4.6 Conceptual Entity Relationships 39
4.7 Data Dictionary of Implemented Records 40
4.7.1 ParkingLocation and ParkingLevel 40
4.7.2 ParkingBay, BayStatus and BayKind 41
4.7.3 ParkingSession and ParkingActivity 42
4.7.4 Browser Storage Structure and Restoration 43
4.8 Input and Output Design 44
4.9 User Interface Design System 45
4.10 Accessibility, Responsive Design and Design Limitations 46
**CHAPTER 5 — TECHNOLOGIES AND TOOLS USED** 47
5.1 Software Requirements and Dependency Baseline 47
5.2 Hardware and Execution Environment 48
5.3 Programming Languages Used 49
5.3.1 TypeScript for Domain and Application Logic 49

# TABLE OF CONTENTS (CONTINUED)


5.3.2 TSX and Declarative Markup 49
5.3.3 CSS and Configuration Data 49
5.4 React State, Derived Values and Effects 50
5.5 Next.js Routing, Layout and Build 51
5.6 Styling, PostCSS and Iconography 52
5.7 Unit, Browser and Static Verification Tools 53
5.8 Development Workflow and Reproducibility 54
**CHAPTER 6 — SYSTEM IMPLEMENTATION** 55
6.1 Implementation Overview and Source Inventory 55
6.2 Application Initialization and State Composition 56
6.2.1 View State versus Operational State 56
6.3 Seed Inventory and Managed Location Construction 57
6.4 Occupancy Calculation and Registration Normalization 58
6.5 Vehicle Check-In Implementation 59
6.6 Vehicle Check-Out Implementation 60
6.6.1 Meaning of Departure in the Current Model 60
6.7 Capacity Resizing and Plan Commit 61
6.8 Browser Persistence Implementation 62
6.9 Location Switching, Search and Dialog Coordination 63
6.9.1 Planning Dialog Coordination 63
6.10 Output Screens — Garage Overview 64
6.10.1 Floor Selection and Status Filtering 65
6.10.2 Vehicle Arrival Form 66
6.10.3 Active Session and Departure Form 67
6.10.4 Floor Capacity Planning 68
6.10.5 New Location Creation 69
6.10.6 Registration Validation and Error Feedback 70
6.10.7 Mobile Output and an Observed Layout Defect 71
6.11 End-to-End Demonstration Reconciliation 72
**CHAPTER 7 — TESTING AND EVALUATION** 73
7.1 Testing Strategy and Evidence Baseline 73
7.2 Existing Unit and Fixture Test Cases 74
7.3 Existing Browser Workflow Results 75
7.4 Supplemental Documentation Browser Checks 76
7.5 Boundary, Negative and Recovery Test Design 77
7.6 Static Checks, Build Verification and Evidence Handling 78
7.6.1 Evidence Preservation 78
7.7 Security and Privacy Review 79

# TABLE OF CONTENTS (CONTINUED)


7.8 Accessibility and Responsive Evaluation 80
7.9 Performance and Scalability Reasoning 81
7.10 Defect Register, Acceptance Review and Evaluation Summary 82
**CHAPTER 8 — CONCLUSION AND FUTURE ENHANCEMENTS** 83
8.1 Conclusion 83
8.2 Achievements against Project Objectives 84
8.3 Limitations of the Current Work 85
8.4 Future Enhancements 86
8.5 Academic Contribution and Closing Summary 87
8.5.1 Recommended Demonstration Narrative 87
**REFERENCES** 88
Repository and Directly Inspected Sources 88
Standards, Official Tool Guidance and Books 89
**APPENDIX A — SAMPLE SOURCE CODE** 90
A.1 Complete Domain Type Declarations 90
A.2 Occupancy and Registration Helpers 91
A.3 Complete Local Check-In Transition 92
A.4 Complete Protected Capacity-Resizing Helper 93
A.5 Checkout and an Existing Protection Test 94
**APPENDIX B — OPERATIONAL GUIDE AND PROPOSED EXTENSION** 95
B.1 Installation and Demonstration Setup 95
B.2 Operator Guide — Inspect, Arrive and Depart 96
B.2.1 Inspect the Correct Scope 96
B.2.2 Start a Demonstration Session 96
B.2.3 Complete a Departure 96
B.3 Operator Guide — Capacity, Locations and Local Data 97
B.3.1 Saved Data and Safe Reset 97
B.4 Proposed Shared-Service Design — Not Implemented 98
B.4.1 Proposed Transaction Boundary 98
B.4.2 Proposed Integration Gates 98

# LIST OF FIGURES


Figure 4.1: Implemented browser-centered application architecture 28
Figure 4.2: Level 0 data flow and the actual system boundary 29
Figure 4.3: Level 1 functional decomposition and local data stores 30
Figure 4.4: Arrival validation, collection transformation and UI commit 31
Figure 4.5: Operator use cases within the local MVP boundary 32
Figure 4.6: Protected capacity-resizing decision workflow 35
Figure 4.7: Actual TypeScript domain records and location-scoped references 36
Figure 4.8: Arrival sequence from form submission to local snapshot 37
Figure 4.9: Implemented bay-state transitions and reserved-state boundary 38
Figure 4.10: Conceptual relationships of the in-memory parking model 39
Figure 4.11: Current local-storage load/write lifecycle and review concern 43
Figure 6.1: Authentic Riverside overview with derived inventory and sample presentation labels 64
Figure 6.2: Upper-deck map filtered to its two reserved sample bays 65
Figure 6.3: Arrival dialog with a formatted TN registration and an available bay 66
Figure 6.4: Departure selector for the newly checked-in demonstration vehicle 67
Figure 6.5: Capacity dialog with occupied/reserved minimums and a draft basement expansion 68
Figure 6.6: Add-location form configured for Harbour Point with two floors and fifteen bays each 69
Figure 6.7: Actual format-validation error for a registration outside the project's TN policy 70
Figure 6.8: Upper region of the authentic mobile capture, retaining the horizontal-overflow defect 71

# LIST OF TABLES


Table 1.1: Project identity and verified architectural profile 2
Table 1.2: Objectives and implementation evidence 4
Table 1.3: Functional scope and exclusion matrix 5
Table 1.4: Assumptions, constraints and their consequences 7
Table 2.1: General workflow alternatives and applicable trade-offs 10
Table 2.2: Compile-time assistance versus runtime guarantees 12
Table 2.3: Verification layers and the questions they answer 15
Table 2.4: Architectural comparison without vendor-specific claims 16
Table 3.1: Workflow weaknesses, effects and MVP responses 18
Table 3.2: Operational functional requirements FR-01 to FR-10 20
Table 3.3: Planning and persistence requirements FR-11 to FR-18 21
Table 3.4: Quality requirements, evidence and current boundaries 22
Table 3.5: Technical feasibility assessment by capability 23
Table 3.6: Economic feasibility categories and estimation inputs 24
Table 3.7: Validation rules and rejection behavior 25
Table 3.8: Principal risks and proposed mitigations 26
Table 3.9: Acceptance mapping from requirements to evidence 26
Table 4.1: Use case UC-03 — Check in a vehicle 33
Table 4.2: Use case UC-04 — Check out an active session 34
Table 4.3: Data dictionary — location and level records 40
Table 4.4: Data dictionary — bay fields and enumerated values 41
Table 4.5: Data dictionary — active session record 42
Table 4.6: Data dictionary — recent activity record 42
Table 4.7: Input controls, derived outputs and interpretation 44
Table 4.8: Implemented visual tokens and layout conventions 45
Table 4.9: Accessibility and responsive review by interaction 46
Table 5.1: Technology baseline declared by the repository 47
Table 5.2: Execution needs and unmeasured planning guidance 48
Table 5.3: Application entry files and relevant configuration 51
Table 5.4: Styling mechanisms and their practical role 52
Table 5.5: Verification commands and evidence categories 53
Table 6.1: Application and test source inventory 55
Table 6.2: Initial inventory counts derived from the seed code 57
Table 6.3: Supplemental scenario state progression 72
Table 7.1: Recorded verification summary 73
Table 7.2: Existing Vitest cases and observed results 74
Table 7.3: Existing Playwright scenarios and final recorded outcomes 75
Table 7.4: Supplemental browser assertions B01 to B16 76
Table 7.5: Proposed additional boundary and recovery cases — not executed as a new suite 77

# LIST OF TABLES (CONTINUED)

Table 7.6: Source-level security/privacy assessment and future controls 79
Table 7.7: Accessibility/responsive evidence and open review items 80
Table 7.8: Dominant operation costs inferred from source 81
Table 7.9: Open findings and suggested acceptance evidence 82
Table 8.1: Objective-by-objective outcome assessment 84
Table 8.2: Staged enhancement roadmap and acceptance gates 86
Table B.1: Current route and interaction inventory — no parking API exists 95
Table B.2: Common operator messages and appropriate response 96
Table B.3: Demonstration and submission checklist 97
Table B.4: Proposed authoritative records and invariants — future design only 98

# ABBREVIATIONS / ACRONYMS

| Acronym | Meaning / contextual qualification |
|---|---|
| A / O / R / T | Available / Occupied / Reserved / Total bay counts |
| API | Application Programming Interface — no parking API in this MVP |
| ARIA | Accessible Rich Internet Applications |
| B1 | Basement level identifier in the sample floor plan |
| BCA | Bachelor of Computer Applications |
| CSS | Cascading Style Sheets |
| DFD | Data Flow Diagram |
| DOM | Document Object Model |
| E2E | End-to-End browser testing |
| ER | Entity Relationship — conceptual in this report |
| EV | Electric Vehicle; bay-kind indicator only |
| FR / NFR | Functional Requirement / Non-Functional Requirement |
| HTML | HyperText Markup Language |
| HTTP / HTTPS | HyperText Transfer Protocol / HTTP Secure |
| ID | Identifier; several bay and floor IDs are location-scoped |
| ISO | International Organization for Standardization; date-string context |
| JSON | JavaScript Object Notation |
| L1 / L2 | Ground-floor / upper-deck identifiers in the sample plan |
| MVP | Minimum Viable Product |
| npm | JavaScript package manager used by this repository |
| OWASP | Open Worldwide Application Security Project |
| RBAC | Role-Based Access Control — future capability, not implemented |
| RTO | Regional Transport Office; syntax digits only in plate validation |
| SRS | Software Requirements Specification |
| TN | Tamil Nadu prefix in the project's registration policy |
| TS / TSX | TypeScript / TypeScript with JSX markup |
| UAT | User Acceptance Testing — no formal stakeholder UAT conducted |
| UI / UX | User Interface / User Experience |
| UML | Unified Modeling Language |
| UTC | Coordinated Universal Time |
| WCAG | Web Content Accessibility Guidelines |

# CHAPTER 1
# INTRODUCTION

## 1.1 Overview of the Project

Parkline is a web-based multi-level parking operations and capacity planning application. It presents a garage manager with a consolidated view of parking capacity, occupied bays, available bays, reserved sample records and active vehicle sessions. The application supports several garage locations, each with independent operational data. Within a selected location, the operator can inspect individual floors, search for a bay or registration, register a vehicle arrival, complete a departure and adjust the planned capacity of each floor. The software package is named `multi-level-parking` and its current declared version is `0.1.0` [1, 7].

The implemented application is a functional user-interface and domain-logic minimum viable product, rather than an installed physical parking-control system. Its main page is provided by the Next.js App Router. A React client component owns interactive state, while independent TypeScript functions implement occupancy calculation, Tamil Nadu registration validation, check-in, check-out and protected capacity resizing. Typed sample fixtures supply four initial garage locations. Browser local storage retains planning and operational changes on the same device and origin; no shared database is connected [2–5].

The central engineering contribution is the consistent treatment of parking as a set of controlled state transitions. A vehicle can enter only an available bay. A normalized registration cannot have two active sessions within the selected location. Checking out a session releases its assigned bay. A capacity reduction cannot discard occupied or reserved bays. These rules make the demonstration useful for studying operational integrity without implying the transactional guarantees of a production backend.

This report documents what exists in the reviewed source revision, explains its design, reproduces genuine application screens and records observed verification results. Proposed authentication, server persistence, payment processing, reservation workflows and sensor connectivity are discussed only as future enhancements. Decorative interface labels such as “LIVE” are not interpreted as evidence of a real-time occupancy service.

## 1.2 Background of the Study

A multi-level garage separates a single parking facility into several spatially distinct floors. An operator therefore needs more than a total count: the operator must know which floor contains a usable bay, whether an apparent vacancy is reserved, and which vehicle is associated with an occupied bay. When this information is recorded in different notebooks or spreadsheets, a change at the gate may not be reflected in the floor-level view. The resulting coordination problem motivates a shared visual model, even before physical equipment or network services are introduced.

Parkline addresses this coordination problem at the browser-interaction level. Each location contains levels, bays, sessions and activity. A bay has both an operational status and a physical kind. For example, an EV bay can be available or occupied; EV is not an occupancy state. Similarly, a reserved bay is unavailable for ordinary arrival even though it is not counted as a parked vehicle. This separation prevents a common modeling error in which unrelated properties are combined into one status field.

**Table 1.1: Project identity and verified architectural profile**

| Attribute | Description of the reviewed application |
|---|---|
| Application identity | Parkline; npm package `multi-level-parking`; version 0.1.0 |
| Primary purpose | Operator dashboard, vehicle sessions and capacity planning |
| Framework | Next.js 16.4.3 with React 19.0.0 and TypeScript |
| Initial inventory | Four locations; three floors and 72 bays per seeded location |
| Operational statuses | available, occupied and reserved |
| Physical bay kinds | standard, ev and accessible |
| Persistence | Device-local JSON in browser local storage |
| Domain verification | Nine existing Vitest tests in one test file |
| Browser workflows | Three existing Playwright scenarios |
| Explicit exclusions | No API, authentication, payment service or live sensors |

The initial names and vehicle registrations are demonstration fixtures. They do not establish a relationship with the named commercial sites, confirm a physical site's capacity or identify actual customer visits. The report treats them solely as software test data.

## 1.3 Problem Statement

The project investigates how a browser application can represent parking availability and operator actions coherently across locations and floors. The problem is not to prove that automated parking has been deployed; it is to build and evaluate a correct, understandable software model for the operational tasks within the current scope.

1. **Fragmented visibility:** A garage-wide total is insufficient when an operator must direct a vehicle to a specific floor. The application must expose location totals and floor-specific details without confusing their scopes.
2. **Ambiguous bay availability:** An unoccupied bay may still be reserved. Occupied, reserved and available counts must remain distinct and their sum must equal the actual inventory size.
3. **Invalid vehicle identifiers:** Registration text may contain lowercase characters, spaces or hyphens. Accepted input needs a single normalized form before duplicate checking and display.
4. **Double allocation:** Reusing an occupied or reserved bay would create conflicting operational records. The transition owner must reject an arrival into anything other than an available bay.
5. **Duplicate active vehicles:** A registration already present in the location's active sessions must not receive a second session simply because its formatting differs.
6. **Unsafe capacity changes:** Reducing a floor's capacity must not silently remove a bay that contains a vehicle or a reserved sample record. Expansion must allocate unused bay numbers.
7. **Location contamination:** Switching garages must restore the selected location's own inventory and sessions rather than carry another garage's state into the view.
8. **Unclear persistence expectations:** Browser-local saving can support a demonstration, but it cannot coordinate multiple operators or provide durable enterprise records. This limitation must be visible in both the software explanation and the project evaluation.

The desired outcome is an operator-centered MVP with testable invariants and transparent boundaries. Production concerns such as concurrent gate operators, authorization and payment settlement remain separate problems. Documenting those boundaries is necessary because a visually polished dashboard can otherwise suggest capabilities that the source does not implement.

## 1.4 Objectives of the Project

The primary objective is to implement and explain an interactive parking dashboard whose visible totals, bay map and active sessions agree after supported operations. Secondary objectives concern input quality, safe planning, maintainable organization and evidence-based evaluation. Objectives are intentionally phrased as demonstrable software behavior rather than claims about traffic reduction, revenue growth or customer satisfaction, none of which has been measured.

**Table 1.2: Objectives and implementation evidence**

| ID | Objective | Evidence in the application |
|---|---|---|
| O1 | Present location and floor occupancy | `occupancyForLevel` and dashboard summary cards |
| O2 | Support visual bay inspection | Floor tabs, bay buttons, legends and status filters |
| O3 | Normalize and validate registrations | `normalizeTamilNaduPlate` in `parking.ts` |
| O4 | Prevent invalid arrival transitions | Available-bay guard and active-session duplicate guard |
| O5 | Release a bay on departure | `checkOut` returns updated bays and sessions |
| O6 | Preserve protected bays during resizing | `resizeLevelBays` protects non-available inventory |
| O7 | Create configurable garage locations | `createManagedLocation` and planning dialog |
| O8 | Keep location operations separate | `operationsByLocation` indexed by location ID |
| O9 | Retain demonstration data locally | Storage key `parkline-garage-planning` |
| O10 | Verify behavior and expose limitations | Unit tests, browser tests and documented observations |

A successful check-in is not judged only by a confirmation message. The target bay must become occupied, a normalized session must appear, the occupied total must increase and the available total must decrease while capacity and reserved counts remain unchanged. Checkout must reverse the occupancy effect without changing the physical kind of the bay.

Capacity planning has a different success criterion: the inventory count changes while existing occupied and reserved bay identities remain present. This distinction is important because a superficially correct capacity number could conceal loss of operational records. The report uses these objective-specific criteria throughout its requirements, design and testing chapters.

## 1.5 Scope of the Project

The project scope is the current browser-local implementation, including its source code, test suite and operator-facing screens. “Implemented” below means that an identifiable code path exists; it does not imply that every edge case has passed testing or that the feature is ready for production use. Verification outcomes and open defects are reported separately in Chapter 7.

**Table 1.3: Functional scope and exclusion matrix**

| Area | Included in the MVP | Outside the implemented scope |
|---|---|---|
| Locations | Switch four seeds; add a named location | Editing or deleting a location; physical-site verification |
| Floor inventory | Configure 1–10 floors at creation | Adding or removing floors on an existing location |
| Bay capacity | Set 1–99 bays per floor; protect non-available bays | Geometric floor-plan editing; structural capacity approval |
| Arrivals | Select an available bay; validate a TN registration | Ticket printing; gate opening; number-plate recognition |
| Departures | Select an active session on the current level | Duration billing; payments; historical session ledger |
| Reservation display | Display seeded reserved bays and notices | Creating, cancelling, expiring or fulfilling reservations |
| EV and accessibility | Display special-kind indicators | Charging telemetry; eligibility verification; charging fees |
| Persistence | Save JSON in the current browser and origin | Shared database, backup, account synchronization |
| Security identity | Static operator presentation only | Login, logout, authorization and audit attribution |
| Analytics | Derive present inventory counts | Historical trends, forecasting and measured utilization |

Search and filtering operate on the currently selected floor. The arrival bay selector can list available bays across the selected location, while the departure selector is restricted to active sessions on the selected level. These details affect operator instructions and are retained in the report rather than replaced with a more ambitious specification.

The documentation includes conceptual data relationships because they help explain the TypeScript model. It does not describe an existing SQL schema or API. Any relational or service-oriented design in the enhancement discussion is explicitly labeled as proposed.

## 1.6 Project Motivation

Parking is a suitable college-project domain because its rules are easy to demonstrate yet rich enough to expose meaningful software engineering decisions. A bay can hold only one active vehicle, a floor contains a bounded inventory and a departure must release the correct resource. These simple statements produce useful questions about identity, validation, state ownership and error handling. Parkline turns those questions into visible interactions instead of leaving them as isolated algorithms.

Consider an operator working at the Riverside demonstration location. The garage begins with 72 bays across three floors. Its inventory contains 34 occupied, six reserved and 32 available bays. A new vehicle assigned to B1-01 changes only the relevant operational state: capacity stays 72, occupied becomes 35, available becomes 31 and reserved remains six. The same operator can subsequently inspect a different location without altering Riverside's records. This scenario joins a user task with a precise set of data expectations.

Capacity planning introduces another educational dimension. If a floor contains ten occupied and two reserved bays, its capacity cannot safely be reduced below twelve. Available bays may be removed in a deterministic order, but protected records must survive. A new location, by contrast, starts with entirely available standard bays and no sessions. Explaining the difference between these operations demonstrates why business rules should reside in reusable domain functions rather than only in form controls.

The use of a modern frontend stack also supports study of component state, derived values, effects and responsive layout. However, the motivation for using these technologies is maintainable interaction, not the appearance of production infrastructure. A simulated “LIVE” badge or a static manager profile is not a substitute for a network feed or authenticated identity.

The project's academic value therefore lies in a reviewable end-to-end prototype: typed records, focused transition functions, genuine screens, automated checks and a candid account of missing capabilities. This report preserves that emphasis and avoids estimating real-world financial or traffic benefits without deployment evidence.

## 1.7 Assumptions and Constraints

The reviewed MVP assumes one operator's effective working state in a browser tab. It can retain data after a reload, subject to browser storage availability and the implementation's restoration behavior, but it does not provide a synchronization protocol. Two devices do not share a garage state, and two tabs can overwrite the same local-storage record without transaction coordination. For this reason, the no-duplicate-session rule is a local application invariant rather than a database-backed global guarantee.

**Table 1.4: Assumptions, constraints and their consequences**

| Assumption or constraint | Consequence for interpretation |
|---|---|
| Registration policy accepts `TN` plus two digits, two letters and four digits | Syntactic validation only; not verification against a transport authority |
| Bay numbers use a two-digit suffix from 01 to 99 | A floor cannot exceed 99 bays under the current helper |
| Location creation permits 1–10 floors | Maximum newly created inventory is 990 bays |
| Operations are passed for one selected location | Duplicate checking is not global across locations |
| Dates originate from the browser clock | Timestamps are not trusted server audit times |
| Seed arrival ages are generated at module evaluation | Demonstration times vary between runs |
| Activity stores human-readable time strings | “Just now” and seed relative times do not continuously age |
| Saved JSON has only shallow restoration checks | Corrupt or incompatible stored shapes may not recover safely |
| No reservation transition is implemented | Reserved samples remain protected but cannot be managed end to end |

The interface also contains static decorative values: the manager name, the displayed date, the yesterday comparison, the next-arrival message and the EV charging summary. They are not derived historical or telemetry outputs. Treating them as measured results would be inaccurate.

These constraints shape the evaluation method. The report measures no physical garage performance, claims no regulatory certification and does not infer security from visual polish. It records executed checks and distinguishes source-level reasoning, observed browser behavior and future design recommendations.

## 1.8 Organization of the Report

This report follows the supplied college-project reference: formal preliminary pages, an abstract, contents, lists of figures and tables, abbreviations, eight numbered chapters, references and appendices. The cover treatment, serif typography, chapter presentation, running headings and Roman/Arabic page-numbering distinction are adapted from that document. The subject matter has been rewritten for Parkline rather than transferred from the reference's healthcare application.

Chapter 1 establishes the problem, objectives, scope and constraints. Chapter 2 reviews relevant software-engineering approaches and official technical guidance. Chapter 3 converts the observed behavior into functional requirements, quality considerations and feasibility analysis. Chapter 4 presents architecture, data flow, behavioral diagrams, conceptual relationships and the actual TypeScript data dictionary. Chapter 5 explains the technologies and tools used. Chapter 6 walks through implementation and authentic screen outputs. Chapter 7 reports tests, observed defects and evaluation limits. Chapter 8 concludes the study and proposes a staged enhancement roadmap.

## 1.9 Documentation Method and Evidence Boundary

The principal evidence is the checked-out application at commit `aa4b73016f271a7e0778c6c8bbd6c17346f52346`, reviewed on 31 October 2026. The implementation was inspected directly, dependencies were installed from the lockfile and the application was built for screen capture. Unit, lint, type and production-build commands were executed. Existing Playwright scenarios and additional documentation-focused browser checks were also exercised. Machine-readable results accompany the report source.

The evidence distinguishes three categories. **Implemented behavior** is supported by a source path and explained algorithm. **Observed behavior** is supported by a recorded check or screen capture. **Proposed behavior** describes a future change and is never presented as already deployed. No stakeholder interviews, field measurements, payment transactions or physical sensor experiments were performed.

Candidate names, registration numbers and signatory details remain editable placeholders because the reference's personal identities cannot safely be assumed to apply to this project. The certificate and declaration are submission templates requiring the candidate's review and the institution's actual approval; no signature or endorsement has been manufactured.

# CHAPTER 2
# LITERATURE REVIEW

## 2.1 Introduction and Review Method

The literature review places Parkline within established approaches to small operational web applications. It is a technical and design-oriented review, not a field survey of commercial parking installations. The available evidence does not include vendor demonstrations, customer interviews or a systematic collection of experimental papers. Accordingly, the chapter compares architectural alternatives and consults recognized software-engineering and web-platform references without inventing performance statistics or product claims.

The review is organized around six questions. How should a resource-allocation domain be represented? Where should interactive state live? Which constraints can static types enforce? What guarantees does browser storage provide? How should the interface communicate status accessibly? How can tests distinguish a correct transition from a merely plausible screen? These questions map directly to the code and to the limitations that must be addressed before production adoption.

Software-engineering texts support decomposition, requirements traceability and verification [18, 19]. Next.js and React guidance explain the server/client boundary and component state [8, 9]. The TypeScript Handbook informs the interpretation of union types, interfaces and compile-time checks [10]. Web Storage guidance is relevant because the application persists its only saved copy in local storage [11]. WCAG and OWASP guidance supply evaluation lenses for accessibility and browser-resident data risks [12, 13]. Testing-tool documentation supports the separation of unit and browser evidence [14, 15].

The repository and installed Next.js documentation were inspected directly. External standards and official documentation in the bibliography are supplied as authoritative reader resources; this report does not claim a live retrieval or independent certification against every external document. The review links design principles to the observed implementation rather than presenting citations as proof that Parkline complies with all recommendations.

The resulting research gap is practical: a compact frontend prototype can demonstrate correct local parking transitions and safe capacity planning, but production coordination, durable history and identity remain unresolved. This boundary guides the comparative analysis in the following sections.

## 2.2 Review of Existing Approaches and Related Guidance

### 2.2.1 Manual Registers and Spreadsheet-Based Tracking

A manual register can record a vehicle registration, entry time and assigned floor with almost no technical setup. It is understandable to an operator and can remain usable during a network outage. Its weakness is not that paper is inherently unusable, but that cross-record consistency depends on the operator: the count of occupied bays, the assignment list and the visible floor inventory are not automatically reconciled.

Spreadsheets improve search and counting, but a general-purpose grid does not by itself enforce the same transition rules as a domain model. A cell marked “available” can disagree with a separate sheet listing an active session. A row can be deleted while it still represents an occupied bay. Formulas and protected sheets may reduce these risks, yet the protection depends on the spreadsheet design and user discipline rather than on a single reusable transition function.

Parkline's distinction is its explicit operation boundary. `checkIn` simultaneously returns a changed bay collection and an extended session collection; `checkOut` releases the relevant bay and removes the selected active session. `resizeLevelBays` treats occupied and reserved bays as protected resources. This organization follows the broader principle of locating domain rules where state changes are authorized [18, 19]. It does not, however, make browser state transactional across devices.

**Table 2.1: General workflow alternatives and applicable trade-offs**

| Approach | Useful property | Limitation relevant to this project |
|---|---|---|
| Paper register | Low setup effort; human-readable record | Counts and bay assignments require manual reconciliation |
| Spreadsheet | Flexible tabulation and ad hoc search | Cross-sheet invariants require additional design discipline |
| Browser-local dashboard | Immediate visual feedback and reusable rules | State remains local and can be overwritten or cleared |
| Server-backed operations system | Can coordinate shared records and authorization | Requires backend design, deployment and operational controls |

These are category-level observations, not a measured comparison of named products. The current project deliberately occupies the third category so that interaction and domain correctness can be evaluated before introducing a shared service.

### 2.2.2 Component-Based Interactive Web Applications

React models an interface as a function of state. In Parkline, changing the selected location, floor, filter, search text or dialog mode causes the visible dashboard to be recalculated. This is preferable to manually editing unrelated DOM elements after each operator action: the metric cards, grid and session selector can be derived from the same state snapshot. The model is especially useful when an arrival must update several visible outputs together [9].

Next.js adds route composition, document metadata, build tooling and the server/client component boundary. The page route itself contains very little logic; it renders `ParkingDashboard`. That dashboard begins with the `use client` directive because it uses hooks, event handlers and browser local storage. The presence of a Next.js runtime does not mean the application has a parking API. There are no application route handlers or database calls in the reviewed source tree [5, 8].

The selected architecture keeps a simple deployment surface for a demonstration. It also concentrates many responsibilities in one component: form state, dialog coordination, local persistence, location switching and visual rendering. This is acceptable for a small MVP but creates a maintainability consideration. As the application grows, hooks or smaller components could separate persistence, planning and session interaction without relocating the business invariants into presentation-only code.

The review therefore supports a two-level separation. Pure domain helpers should describe valid parking transitions independently of React. UI components should translate operator input into helper calls, commit the returned state and present success or failure. This separation allows Vitest tests to inspect transition results without launching a browser, while Playwright verifies that labels, selectors and forms are connected correctly.

A client-side design also has a clear trust limit. All data and checks can be inspected or altered by the browser user. A future server must repeat validation and authorize operations; it cannot treat successful client validation as a security boundary. This remains an architectural requirement for future work rather than a feature of the current dashboard.

### 2.2.3 Typed Domain Models and Immutable Transformations

The TypeScript model uses union types to restrict `BayStatus` to `available`, `occupied` or `reserved`, and `BayKind` to `standard`, `ev` or `accessible`. Interfaces describe locations, levels, bays, sessions and activity. These declarations improve editor feedback and make the intended structure explicit. They also reduce accidental spelling differences between the domain module and dashboard rendering [2, 10].

Static types cannot prove every business rule. A `ParkingBay` interface permits optional `plate` and `since` fields, so the compiler does not establish that every occupied bay has a valid active session. Likewise, parsing local-storage JSON and asserting that it has a TypeScript shape does not validate its runtime content. This distinction is essential when discussing reliability: source-level types provide useful constraints, but untrusted or corrupted data still requires runtime schema checking.

The transition helpers favor immutable collection updates. `checkIn` maps the bay collection and appends a new session; `checkOut` maps the assigned bay and filters the session collection. Resizing returns the original bay array when no size change is needed, otherwise it returns filtered or extended arrays. This approach makes before-and-after states easy to compare in tests and helps React detect changed state references.

**Table 2.2: Compile-time assistance versus runtime guarantees**

| Concern | Assistance already provided | Additional runtime responsibility |
|---|---|---|
| Status vocabulary | String-literal union | Reject invalid restored JSON values |
| Record fields | Typed interfaces | Verify required values in imported or saved records |
| Registration shape | A string parameter | Normalize and match the required regular expression |
| Bay availability | Typed bay status | Check actual status at the transition boundary |
| Duplicate vehicle | Typed session array | Compare normalized plate against active sessions |
| Capacity bounds | Numeric parameter | Require an integer within 1–99 |

The lesson drawn from the review is that types, transition guards and tests complement one another. None replaces the others, and none supplies the multi-operator atomicity that would require a server-side authority.

### 2.2.4 Browser Storage and Persistence Semantics

Web Storage provides an origin-scoped key/value interface accessible to browser JavaScript [11]. Parkline serializes its planning and operational data into a single JSON value under `parkline-garage-planning`. This is a practical choice for a classroom demonstration because it needs no database installation and allows an operator to revisit a plan in the same browser. It is not equivalent to durable shared storage.

A saved value belongs to the current browser profile and origin. Clearing site data can remove it. Private-browsing behavior, storage policies, quota limits or unavailable storage can affect it. A second device starts with its own data rather than inheriting the first device's plan. Multiple tabs can hold different React state snapshots and overwrite the same key without a transaction or conflict protocol. These properties directly limit the operational claims that can be made about the MVP.

The implementation restores data in an effect after initial render and writes later state changes in another effect. The load path performs only shallow checks for a non-empty locations collection and an operations object. The design needs stronger runtime validation and a versioned saved-data schema if it is to survive structural changes safely. It also needs carefully coordinated hydration and saving; Chapter 7 records a browser-test failure in which the selected added location was not retained after reload.

OWASP guidance is relevant because local storage is readable by scripts executing in the application's origin [13]. Vehicle registrations should therefore be treated as potentially identifying operational data, not as harmless configuration. The browser copy has no implemented access-control, retention or encryption policy. This report recommends fictional data for demonstrations and does not treat local storage as an appropriate location for credentials or payment details.

The review supports retaining local storage for prototype convenience while naming its limitations explicitly. Migration to a production service would require data ownership, authorization, transactional updates, schema migration, backup and conflict handling—not merely replacing `setItem` with a network call.

### 2.2.5 Accessible Status-Oriented Interfaces

Parking interfaces must communicate more than color. A user should be able to identify the selected location, determine a bay's state and understand the effect of an action through labels, text and keyboard-operable controls. WCAG 2.2 provides a relevant framework for reviewing perceivability, operability and understandable feedback [12]. This report uses that framework as guidance, not as a claim of audited conformance.

Parkline uses native buttons, inputs and selects, associated form labels, accessible bay names and visible focus styling. A bay button's accessible name includes its ID, status and registration when present. Metric values and a textual legend supplement the bay colors. Success notices use a status role, while form errors use an alert role. These choices support non-visual interpretation of changes and make browser tests less dependent on fragile CSS selectors.

The implementation also exposes gaps that should not be concealed by the presence of ARIA attributes. Dialogs declare `role=dialog` and `aria-modal=true`, but the reviewed code does not implement a focus trap, Escape-key closing or restoration of focus to the invoking control. Floor tabs use tab roles without a complete roving-tabindex and arrow-key interaction pattern. At narrower widths, the location switcher is hidden. A mobile overflow check at 390 CSS pixels also failed in the documentation run.

### 2.2.6 Visual Design and Operational Honesty

The interface uses a coherent green, amber and neutral palette with spacing tokens, subtle panels and Lucide icons. However, operational language must remain aligned with data provenance. A “Live operations” label, an “8.4%” yesterday comparison or an EV station count can be mistaken for telemetry if the report describes them without qualification. In this MVP, those elements include static presentation content.

The review therefore recommends two complementary criteria: make actions understandable and make the source of displayed information clear. Accessible controls improve interaction quality, while honest labeling prevents a polished demonstration from being interpreted as a connected physical system.

### 2.2.7 Automated Verification and Test Layering

Vitest runs the domain tests in a Node environment, using an alias that resolves `@` to the source directory. This makes it suitable for deterministic checks of counts, validation errors, immutable updates and fixture consistency. The existing nine tests cover key transition rules and managed-location creation. They are useful evidence, but they are not a complete boundary or property-based test suite [6, 14].

Playwright exercises the browser-facing application. The three existing scenarios switch garage locations, create and resize a location with a reload, and perform an arrival followed by departure. They verify that accessible labels and controls are connected to the intended behavior. The documentation also includes a separate capture script that executes sixteen supplemental checks and saves authentic screen images. These checks are identified separately so that they are not misrepresented as original repository tests [15].

A healthy verification strategy combines static checks, unit tests, browser workflows and human review. Linting can detect problematic patterns but cannot prove a session invariant. Type checking can validate source relationships but cannot establish that saved JSON is trustworthy. A production build confirms compilation and route generation but does not prove persistence behavior. Browser tests can expose integration failures that all three earlier checks miss.

**Table 2.3: Verification layers and the questions they answer**

| Layer | Principal question | Important boundary |
|---|---|---|
| ESLint | Does source conform to configured checks? | Not a functional test |
| TypeScript | Are declared source types consistent? | Does not validate storage JSON |
| Vitest | Do domain functions satisfy tested cases? | Does not inspect actual DOM interaction |
| Production build | Can the application compile and generate routes? | Does not prove runtime user journeys |
| Playwright | Do browser workflows produce expected outcomes? | Limited to exercised environment and cases |
| Review | Are semantics, privacy and scope honestly represented? | Requires judgment and additional evidence |

The review supports evidence-based reporting: a failed integration check is recorded as a finding, not overwritten by a statement that all tests passed because the unit suite was green.

## 2.3 Comparative Study of Related Approaches

**Table 2.4: Architectural comparison without vendor-specific claims**

| Criterion | Manual / spreadsheet workflow | Current Parkline MVP | Proposed shared system |
|---|---|---|---|
| Floor visibility | Operator-maintained representation | Derived floor totals and bay map | Shared authoritative inventory |
| Transition rules | Human discipline or custom formulas | Pure TypeScript guards | Server guards plus transactional constraints |
| Persistence | Physical files or document storage | One browser-local JSON value | Managed durable database and backups |
| Concurrent operators | Manual reconciliation | No safe coordination mechanism | Versioning, locking or transactional conflict handling |
| Identity | External organizational process | Static profile only | Authentication and role-based authorization |
| Reservation lifecycle | Manual booking process | Reserved samples only | Explicit create, expire, cancel and consume transitions |
| Verification | Manual review | Unit and browser checks | Broader integration, recovery and security testing |

## 2.4 Gaps Identified

The first gap is shared correctness: another operator can work from a different snapshot despite local duplicate guards. The second is durable history: departure deletes the active session instead of retaining a completed visit. The third is trust: browser edits are not authorized business actions, and the static profile does not identify an operator.

The fourth gap is evidence breadth. A small suite can demonstrate expected cases while leaving malformed storage, maximum capacity, date validity, accessibility interaction and repeated reload behavior insufficiently tested. Observed mobile and persistence failures show why the quality discussion must be separated from the implementation inventory.

## 2.5 Summary of Literature Review

The reviewed approaches justify the MVP's typed local model, component-driven interface and layered tests as an appropriate educational foundation. They also establish that a production parking platform needs additional architecture and operational controls. The next chapter converts that foundation into explicit requirements and acceptance criteria, preserving a distinction between current functionality, defects and future proposals.

# CHAPTER 3
# SYSTEM ANALYSIS

## 3.1 Existing Workflow Analysis

System analysis identifies the information and rules needed to support parking operations. The baseline considered here is a generic manual or loosely connected tracking workflow, not a surveyed installation. In that baseline, an operator receives a vehicle, identifies a free bay, records a registration and later marks the vehicle as departed. A separate capacity plan may state how many spaces exist on each level. Unless those records are reconciled, a capacity summary can disagree with actual assignments.

Parkline makes these relationships explicit. A location groups its floors and operational collections. A level identifies a floor and its planned capacity. A bay identifies one physical resource within a level. An active session associates a normalized registration and vehicle description with one bay. Activity provides a lightweight recent-events display. This model supports analysis of each operation's preconditions and postconditions before discussing implementation details.

Three information scopes must remain separate. **Location scope** determines the relevant garage and duplicate-session collection. **Level scope** determines the current map, occupancy strip and departure selector. **Bay scope** determines whether a specific arrival or departure action is valid. Many interface errors can be understood as accidental mixing of those scopes, for example showing another floor's sessions or preserving an old floor after changing locations.

The project also has two distinct forms of change. Operational changes alter status and sessions without changing physical inventory. Planning changes alter inventory size while preserving existing non-available resources. This distinction drives both the design and the tests: an arrival conserves total capacity, whereas expansion increases total capacity while conserving occupied and reserved counts.

The analysis is derived from the checked-out code and executable workflows. It includes no claim that the named demonstration locations currently operate this application. The following sections translate the observed functions into a requirements specification and identify the conditions under which those functions remain safe.

## 3.2 Limitations of a Disconnected Tracking Workflow

A disconnected workflow can be adequate for a small, supervised operation, but it creates recurring opportunities for inconsistent records. The analysis below identifies general failure modes relevant to the prototype. It does not assign incident rates, average waiting times or financial losses because those quantities were not measured.

**Table 3.1: Workflow weaknesses, effects and MVP responses**

| Weakness | Potential effect | Response within Parkline's scope |
|---|---|---|
| Separate occupancy and session records | A bay appears free while a session still exists | Transition helpers return both updated collections |
| Unnormalized registration entry | Equivalent text is treated as different vehicles | Spaces and hyphens removed; letters uppercased |
| No explicit reserved state | Reserved inventory is offered to an arrival | Reserved bays are excluded from available choices |
| Manual capacity editing | Active assignments are accidentally removed | Non-available bays are protected during resizing |
| A single undifferentiated garage count | Operator cannot identify the relevant floor | Floor tabs, summaries and map filtering |
| Shared labels without scoped identity | Similar bay IDs from different sites are confused | Operations are indexed by location ID |
| Unclear save location | Operators expect unavailable cross-device data | Local-browser persistence is documented explicitly |
| Static dashboard decoration mistaken for facts | Unmeasured trends are interpreted as analytics | Report identifies decorative values and provenance |

The response column describes the intended implemented mechanism, not an assertion of complete reliability. For example, the storage mechanism exists, but a browser reload scenario failed during verification. Similarly, native controls support basic keyboard use, but dialog focus management remains incomplete. These distinctions prevent the requirements chapter from concealing defects beneath feature descriptions.

A production operation would also need procedures for lost tickets, disputed registrations, emergency closures, inaccessible spaces and overlapping operator actions. Those procedures cannot be inferred from this frontend code. They require stakeholder decisions before a server-side specification can be finalized.

The immediate analysis goal is therefore narrower: identify what the current system must maintain after a supported local action, then test that relationship with observable evidence.

## 3.3 Proposed System and Operator Workflow

For the academic project, the proposed system is the implemented Parkline MVP. The operator first selects a garage and inspects the overall capacity cards. The operator can then choose a floor, inspect bay colors and labels, search within that floor or filter by status. A bay click routes the operator toward the action appropriate to its state: available opens arrival, occupied opens departure and reserved displays an explanatory notice.

During arrival, the operator enters a Tamil Nadu registration and optional vehicle description, then chooses an available bay. The domain helper normalizes the registration, validates the format, verifies bay availability and rejects an already-active registration in the selected location. Only a successful result is committed to operational state. A new arrival activity item is prepended and a status message identifies the registration and bay.

During departure, the operator selects an active session from the current level. The domain helper finds that session, makes its bay available, clears its vehicle-specific fields and removes it from the active collection. A departure event is displayed. The prototype does not calculate a tariff, collect money, print a receipt or retain a completed-session ledger.

Capacity planning opens a separate management dialog. Existing floor capacities can be changed within allowed limits. The application computes prospective changes before committing updated levels and bays. An additional tab creates a new location with a trimmed name, one to ten floors and one to ninety-nine standard available bays per floor. The UI also rejects a duplicate name case-insensitively.

## 3.4 Advantages within the Defined Scope

The main advantages are coherent visual feedback, reusable local rules, deterministic bay allocation during resizing and a small deployment footprint. These properties make the application suitable for demonstrations and iterative design review. They are not evidence of reduced congestion, increased revenue or secure multi-user operation.

The system's key trade-off is simplicity versus authority. Keeping state in the browser lowers setup effort, but the browser cannot arbitrate competing operators. Future work must introduce a trusted service if the same physical inventory is to be managed by multiple clients.

## 3.5 Software Requirements Specification

### 3.5.1 Functional Requirements — Operational Dashboard

The following requirements describe observable behavior in the current design. “Implemented” denotes a source-backed feature. An associated test can still fail, and a missing test must not be inferred from the status. Each requirement has an identifier used again in the traceability and evaluation sections.

**Table 3.2: Operational functional requirements FR-01 to FR-10**

| ID | Requirement and acceptance condition | Principal implementation |
|---|---|---|
| FR-01 | Select a garage and display only its operational data | `changeLocation`; `operationsByLocation` |
| FR-02 | Show total, occupied, reserved and available counts whose sum reconciles | `occupancyForLevel`; total reduction |
| FR-03 | Select a floor and show its matching bay inventory | Level state and floor tabs |
| FR-04 | Search bay IDs or plates on the selected floor, case-insensitively | `filteredBays` calculation |
| FR-05 | Filter selected-floor bays by all/available/occupied/reserved | Status select and predicate |
| FR-06 | Normalize and validate registrations before starting a session | `normalizeTamilNaduPlate` |
| FR-07 | Reject an arrival unless the chosen bay exists and is available | `checkIn` bay guard |
| FR-08 | Reject a normalized registration already active in this location | `checkIn` session guard |
| FR-09 | Complete an active-session departure and release its bay | `checkOut`; `submitDeparture` |
| FR-10 | Display recent arrival/departure activity and outcome feedback | Activity list and status notice |

The dashboard derives most counts from bay records rather than trusting separately entered totals. This is an important requirement interpretation: the planned `capacity` field helps describe the floor plan, but the occupancy helper counts the actual records passed to it. Keeping those two representations synchronized is part of capacity planning.

FR-04 does not normalize spaces inside a search registration; it trims and lowercases the query and performs substring matching. FR-09's UI selector is limited to the current level even though the pure helper itself accepts a session ID from the collection supplied to it. These details are reflected in the user guide so that expected behavior matches the source.

### 3.5.2 Functional Requirements — Planning and Local Persistence

Planning requirements are separated from session operations because they change inventory structure. A capacity value is accepted only when it is an integer from one to ninety-nine. When reducing a floor, the resulting capacity must remain at least the count of occupied plus reserved bays. Existing protected IDs are retained even if the remaining numbering is no longer contiguous.

**Table 3.3: Planning and persistence requirements FR-11 to FR-18**

| ID | Requirement and acceptance condition | Principal implementation |
|---|---|---|
| FR-11 | Expand a floor with unused numbered standard available bays | `resizeLevelBays` addition branch |
| FR-12 | Shrink a floor by removing only available bays | `resizeLevelBays` removal branch |
| FR-13 | Preserve all occupied and reserved bay identities during resizing | Protected-count guard and removal filter |
| FR-14 | Create a location with 1–10 floors and 1–99 bays per floor | `createManagedLocation` |
| FR-15 | Reject a blank or duplicate location name in the UI | Helper trim check; UI name comparison |
| FR-16 | Save locations, operations and selected location in this browser | Persistence effect and storage key |
| FR-17 | Attempt to restore a saved plan on reload | Initial storage-reading effect |
| FR-18 | Reset floor, search, filter and session dialog when switching sites | `changeLocation` state resets |

FR-16 and FR-17 are deliberately written as local persistence behavior, not cloud synchronization. The recorded reload failure means that persistence needs further engineering and regression testing before it can be accepted as reliable. A successful supplemental reload check establishes only that one exercised sequence worked in that run.

Location creation does not support an arbitrary custom floor-naming scheme. The first floor is B1/Basement, the second is L1/Ground floor and later floors use incrementing level identifiers. New bays are standard; the planning form does not designate EV or accessible bays. Duplicate-name prevention is enforced in the dashboard rather than in the reusable creation helper, so direct helper callers must supply their own uniqueness policy.

These qualifications are part of the requirements baseline. They prevent a generic “manage locations” statement from implying edit, delete, special-bay assignment or multi-user uniqueness features that are absent from the implementation.

### 3.5.3 Non-Functional Requirements and Quality Expectations

Non-functional requirements describe desirable qualities and the evidence needed to evaluate them. Some are partially supported by the current design; others remain targets. The table avoids presenting an untested target as an achieved guarantee.

**Table 3.4: Quality requirements, evidence and current boundaries**

| ID | Quality expectation | Evidence and qualification |
|---|---|---|
| NFR-01 | Maintainable separation of rules and presentation | Pure helpers and shared types exist; dashboard remains large |
| NFR-02 | Predictable local state transitions | Nine unit tests pass for selected cases; broader boundaries untested |
| NFR-03 | Responsive presentation | CSS breakpoints exist; 390-pixel overflow check failed |
| NFR-04 | Understandable keyboard and assistive feedback | Native labels, focus styles and roles exist; no full accessibility audit |
| NFR-05 | Reliable reload restoration | Save/restore effects exist; one existing browser scenario failed |
| NFR-06 | Honest information provenance | Browser-local limitations documented; some UI labels remain decorative |
| NFR-07 | Protection of registration data | No authentication or retention policy; fictional demo data recommended |
| NFR-08 | Reproducible verification | Lockfile, scripts and report evidence retained |
| NFR-09 | Bounded planning input | Integer checks constrain floors and bays |
| NFR-10 | Safe concurrent physical allocation | Not satisfied by a browser-only architecture; future server required |

No response-time threshold is claimed as achieved because endpoint benchmarking was not performed and no parking API exists. Similarly, a successful build does not establish availability, backup recovery or cross-browser compatibility. Those qualities would need a deployment specification and an evaluation environment.

For this project, the strongest current evidence concerns understandable local behavior and domain rules. The weakest areas concern persistence reliability, narrow-screen interaction, runtime validation of restored data and shared operational authority. Making that distinction explicit helps prioritize improvements rather than treating all quality attributes as equally complete.

The requirements also carry a privacy implication. A registration may identify or be linked to a vehicle user. Even in a prototype, demonstrations should avoid unnecessary personal data, and a future production system should specify access, masking, retention and deletion policies before collecting real customer records.

## 3.6 Feasibility Study

### 3.6.1 Technical Feasibility

The current prototype is technically feasible within a standard web-development environment. It requires Node.js 20.9 or newer according to the repository README, npm and a modern browser. The documentation run used Node.js 22.22.3 and npm 10.9.8. The dependency lockfile installed successfully, and the application passed linting, type checking and production compilation. These results establish that the reviewed revision can be built in the tested environment; they do not certify every operating system or browser.

The architecture is intentionally compact. A single application route renders a client dashboard. Data is represented as arrays of typed records, and domain helpers use filtering, mapping and bounded number allocation. A newly created location can contain at most 990 bays under the current limits. This upper bound makes the chosen collection-based design plausible for a demonstration, although no maximum-size performance benchmark was performed.

A production version would be a materially different engineering project. It would need an authoritative data store, authenticated requests, conflict handling and server-side validation. Existing pure functions provide a useful specification, but browser-only duplicate checks cannot simply be promoted to concurrency guarantees. Session and bay updates would need to be committed atomically, with a database constraint or equivalent mechanism preventing multiple active assignments to the same bay.

**Table 3.5: Technical feasibility assessment by capability**

| Capability | Feasibility within the reviewed MVP |
|---|---|
| Local demonstration and classroom review | Supported by successful installation and build |
| Domain-rule regression tests | Supported by the passing Vitest suite |
| Browser workflow verification | Available; one existing scenario failed in the recorded run |
| Single-device saved planning | Implemented but needs restoration-race and corruption hardening |
| Multiple synchronized operators | Not supported without a new authoritative backend |
| Sensor or gate integration | Requires hardware protocol, identity and failure-mode design |

The conclusion is therefore conditional: the local educational application is feasible and demonstrable, while production feasibility depends on additional requirements, infrastructure and safety controls not present in this repository.

### 3.6.2 Economic Feasibility

The repository uses a standard open-source web stack and requires no paid parking API or database subscription for the current local demonstration. This lowers setup effort, but development, equipment, review and future support still have costs.

No currency-denominated return on investment is calculated because the project has no measured deployment baseline, operator-time study, revenue records or hosting quotation. The analysis therefore identifies cost categories and evidence needed for a future budget instead of inventing annual savings.

**Table 3.6: Economic feasibility categories and estimation inputs**

| Cost or benefit category | Current demonstration position | Evidence needed for a production estimate |
|---|---|---|
| Development tools | npm and editor-based workflow | Team licenses and organizational tooling policy |
| Hosting | Local production server used for evidence | Deployment size, bandwidth and availability target |
| Persistence | Browser storage only | Database capacity, backup and retention requirements |
| Identity and security | Not implemented | User count, authentication method and review effort |
| Operator training | User-guide walkthrough can be demonstrated | Number of staff, shift patterns and training duration |
| Equipment integration | No hardware expense within scope | Sensor/gate inventory, protocol and installation quote |
| Operational benefit | Better local visibility is demonstrable | Measured baseline and post-deployment task timings |

### 3.6.3 Operational Feasibility

The operator workflow uses recognizable actions: choose a garage, choose a floor, inspect a bay, check in, check out and plan capacity. Labels and confirmation notices support demonstration without a complex setup process. However, the interface has no authenticated role separation, so “garage manager” is a presentation label rather than an enforced permission.

Operational adoption would require procedures for incorrect entry, lost browser data, disputed departures, inaccessible bays, overrides and emergency shutdowns. The current dashboard does not define those procedures. It is therefore suitable for supervised demonstration and requirements discovery, but not as the sole record of a real garage's operations.

## 3.7 Input Validation and Error Analysis

Validation occurs at two levels. Native HTML controls communicate required fields and numeric ranges. Domain helpers then enforce the important transition rules even when called independently of the form. This layered approach is necessary because presentation constraints alone can be bypassed or become inconsistent with other callers.

**Table 3.7: Validation rules and rejection behavior**

| Input or action | Rule | Rejection or handling |
|---|---|---|
| Registration | Remove spaces/hyphens, uppercase, match `^TN\d{2}[A-Z]{2}\d{4}$` | Explain required TN format |
| Arrival bay | Must exist and be available | Ask the operator to choose an available bay |
| Active registration | Must not already occur in the location's sessions | Report an existing active session |
| Departure session | Must exist in the supplied active list | Report that an active session was not found |
| Capacity | Integer, 1–99 | Explain the allowed bay range |
| Reduced capacity | At least occupied plus reserved count | Reject removal of protected inventory |
| Location name | Trimmed non-empty string | Ask for a location name |
| Duplicate location | Case-insensitive match blocked in UI | Report an existing name |
| Floors at creation | Integer, 1–10 | Explain the allowed floor range |

Several validation gaps remain. The domain helper does not validate the `startedAt` string before parsing it into a session identifier. Restored JSON is not checked against a complete runtime schema. The model does not enforce EV or accessible eligibility, because no eligibility data is collected. The helper that creates a location does not itself enforce a unique name across a location list.

The UI shows domain errors without committing successful-state updates. Nevertheless, user feedback and security are different concerns: an error message does not create authorization. A future server must validate the same business conditions against trusted current data and return a conflict response if another operator has already changed the bay.

The analysis recommends preserving precise error messages and adding tests for each rejection path, including boundary values, malformed timestamps and incompatible saved records. These recommendations are recorded as future quality work rather than silently added behavior.

## 3.8 Risk Assessment and Acceptance Traceability

Risks combine source review and recorded observations. Priorities are qualitative, not statistically estimated failure probabilities.

**Table 3.8: Principal risks and proposed mitigations**

| Risk | Current significance | Proposed mitigation |
|---|---|---|
| Reload loses the selected added-location plan | Observed in one existing browser scenario | Coordinate restoration and saving; add repeated reload regression |
| Mobile horizontal overflow | Observed at 390 CSS pixels | Identify wide descendants; test narrow viewports |
| Multiple tabs overwrite each other's state | Architectural limitation | Add synchronization or an authoritative server |
| Stored JSON has incompatible structure | Shallow validation only | Versioned schema and runtime validation |
| Vehicle registrations remain on a shared device | No access or retention controls | Use fictional demo data; define privacy policy |
| Static labels suggest live telemetry | Interpretation risk | Mark sample values and remove unsupported live claims |
| Dialog keyboard focus escapes | Missing focus-management implementation | Adopt a tested accessible dialog pattern |

**Table 3.9: Acceptance mapping from requirements to evidence**

| Requirement group | Evidence source | Acceptance interpretation |
|---|---|---|
| FR-02, FR-06 to FR-09 | Existing domain tests | Selected scenarios passed, not exhaustive proof |
| FR-11 to FR-14 | Resize and creation tests | Valid expansion, protection and creation cases passed |
| FR-01, FR-18 | Existing location-switch browser test | Passed in the recorded browser run |
| FR-14, FR-16, FR-17 | Existing add/resize/reload browser test | Failed after reload; remains an open issue |
| FR-04, FR-05, FR-10, FR-15 | Supplemental report browser checks | Exercised selected interactions, distinct from original suite |
| NFR-03 | Supplemental geometry assertion | Failed at 390 pixels; not accepted as overflow-free |

The analysis concludes with an actionable baseline: the model and core local transitions are demonstrable, while reliable persistence, narrow-screen behavior and production controls require further work. Chapter 4 shows how those observations arise from the actual architecture and data structures.

# CHAPTER 4
# SYSTEM DESIGN

## 4.1 Design Overview and Architectural Principles

System design translates the analysis into component boundaries, data structures and valid state changes. Parkline uses a browser-centered architecture with a small server-rendered application shell. The key design principle is to distinguish the physical inventory from active use of that inventory, then derive visible outputs from the current operational collections rather than maintain unrelated counters.

**Separation of concerns.** The route and root layout compose the application. `ParkingDashboard` handles interaction and presentation. `parking.ts` owns reusable transition rules. `parking-data.ts` defines fixtures and managed-location construction. `parking.ts` in the types directory declares the shared data contracts. This separation allows the domain module to be tested independently of the UI.

**Explicit state ownership.** The dashboard owns a locations collection and an `operationsByLocation` object. The selected location ID determines which bays, sessions and activity are in scope. The selected level determines which map and departure choices are displayed. Inputs and dialog selections are separate view state; they do not become operational records until a valid operation completes.

**Conservation of inventory.** Arrival and departure change status, not capacity. Occupancy summaries use the actual bay array. The invariant is total equals occupied plus reserved plus available. Planning is the controlled exception that changes total inventory while preserving non-available bays.

**Immutable transformations.** Helpers return new arrays or preserve an unchanged array rather than mutate the input collections in place. This makes operation results straightforward to inspect and reduces the risk that a failed operation partially changes React state. It is not a substitute for a database transaction across independent clients.

**Transparent boundaries.** The design contains no parking API, authentication service, live sensor feed or payment processor. Local storage is a browser copy of application state, not an authoritative multi-operator store. Architectural diagrams in this chapter reflect that boundary. Proposed production mechanisms are discussed only after the implemented model has been described.

These principles favor a teachable MVP. Improve runtime validation and persistence coordination first, then introduce server authority for shared operations.

## 4.2 System Architecture

The application has three practical execution areas: Next.js composition, interactive browser state and pure domain/data modules. The route at `/` imports the dashboard. The root layout sets page metadata, imports global styles and declares English as the document language. The dashboard is the only major client component and contains the hooks, browser effects and event handlers [5, 8].

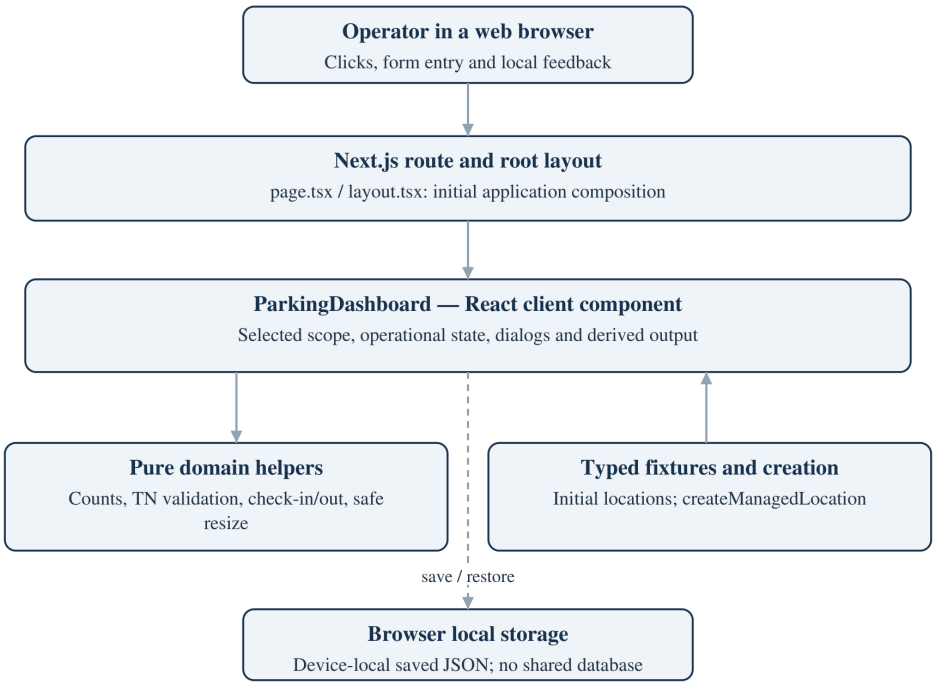



**Figure 4.1: Implemented browser-centered application architecture**

The domain helpers are ordinary TypeScript functions. They receive the collections and identifiers required for an operation and return new collections or throw an error. They do not call React setters, local storage, HTTP endpoints or a database. This keeps the business rules reusable and enables focused tests.

Fixture creation belongs to a separate module. Four seed locations provide initial inventory, while `createManagedLocation` constructs a new empty location from validated input. After initialization, the dashboard's operational map is the working source for sessions, bays and activity. The location objects still contain nested collections, which means stored JSON contains some duplicated representations. Chapter 7 discusses the consistency risk.

The dashed persistence connection in the figure represents client effects rather than a remote service. A browser can save its current state and later attempt to restore it, but no process arbitrates changes from another browser. That distinction is central to the architecture's trust and concurrency limits.

## 4.3 Data Flow Diagrams

### 4.3.1 Level 0 — System Context

The context diagram treats Parkline as a single local application. The external actor is an operator who supplies manually observed vehicle and bay information. The application responds with occupancy summaries, map states, validation messages and activity. The only persistent store in the implemented design is browser local storage.

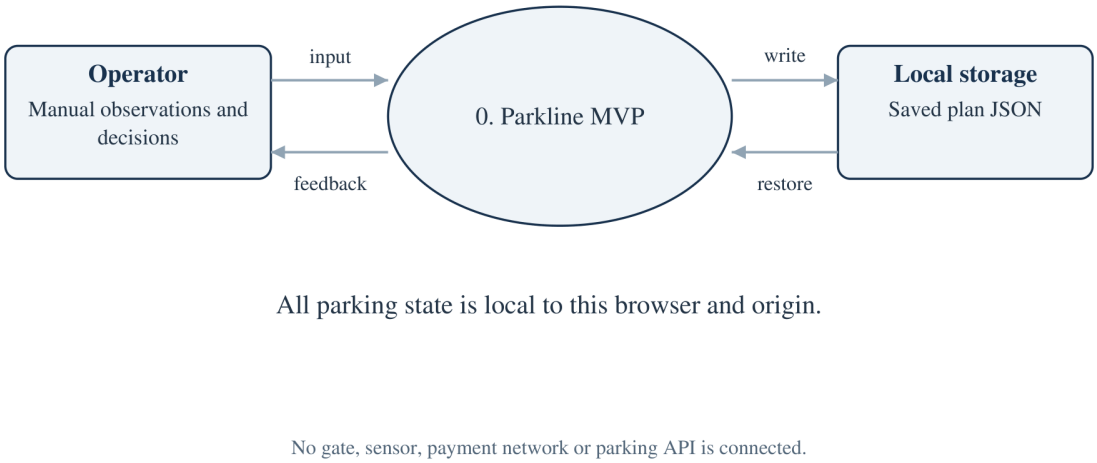



**Figure 4.2: Level 0 data flow and the actual system boundary**

Operator input includes location selection, floor selection, search text, status filters, registration, vehicle description, selected bay, selected active session and planning values. The output includes both stateful facts and presentation feedback. For example, the occupied count is derived from bay records, while a confirmation message explains the result of the latest action.

Local storage is shown separately because it survives a normal page lifecycle, but it remains inside the browser's operational environment. It is not an independent organization or a network backend. A developer-tool edit to the saved value is outside the intended operator workflow yet is possible because the browser owns the data.

The absence of other actors is deliberate. There is no authenticated administrator, customer account, payment gateway, sensor controller or transport-authority validation service in the reviewed code. Adding such an actor would create new trust boundaries and requirements. The current context diagram therefore supports accurate scoping rather than the visual appearance of a larger connected platform.

### 4.3.2 Level 1 — Functional Decomposition

The main process decomposes into inspection, session start, session end, inventory planning and rendering. Inspection changes view state but does not alter a bay. Session operations change bays and active sessions. Planning changes location structure and inventory. Rendering derives cards, floor summaries and map output from the selected scope.

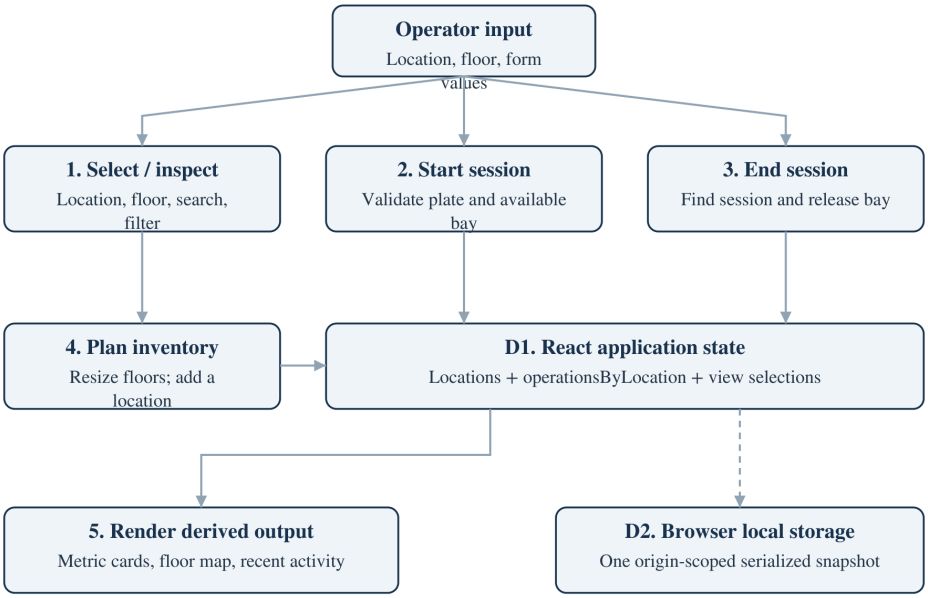



**Figure 4.3: Level 1 functional decomposition and local data stores**

The working state store contains `garageLocations`, `operationsByLocation` and the relevant selections. This is a conceptual data-flow store, not a separate service. The operational map is keyed by location ID so that switching locations retrieves a different collection rather than resetting all garages to a common inventory.

The rendering process does not independently persist a count. It computes selected-floor statistics and reduces statistics across the selected location's levels. This reduces the risk of an occupied counter drifting away from the bay records. Search and filter predicates affect only which current-floor bays are shown; they do not change the statistics or the underlying inventory.

The local-storage connection is an effect after state changes. That placement keeps storage operations out of pure transition helpers, but it introduces ordering and recovery responsibilities at the UI boundary. The existing implementation needs stronger coordination of loading and writing, as shown by the reload test finding. The diagram describes intended data movement without asserting that every restoration sequence is reliable.

### 4.3.3 Level 2 — Vehicle Arrival

Arrival is expanded because it combines user input, validation, resource allocation and feedback. The domain function first looks up the target bay and normalizes the registration. It then verifies the bay's available state and checks for an equivalent active registration. Only after those guards does it construct a new active session and changed bay array.

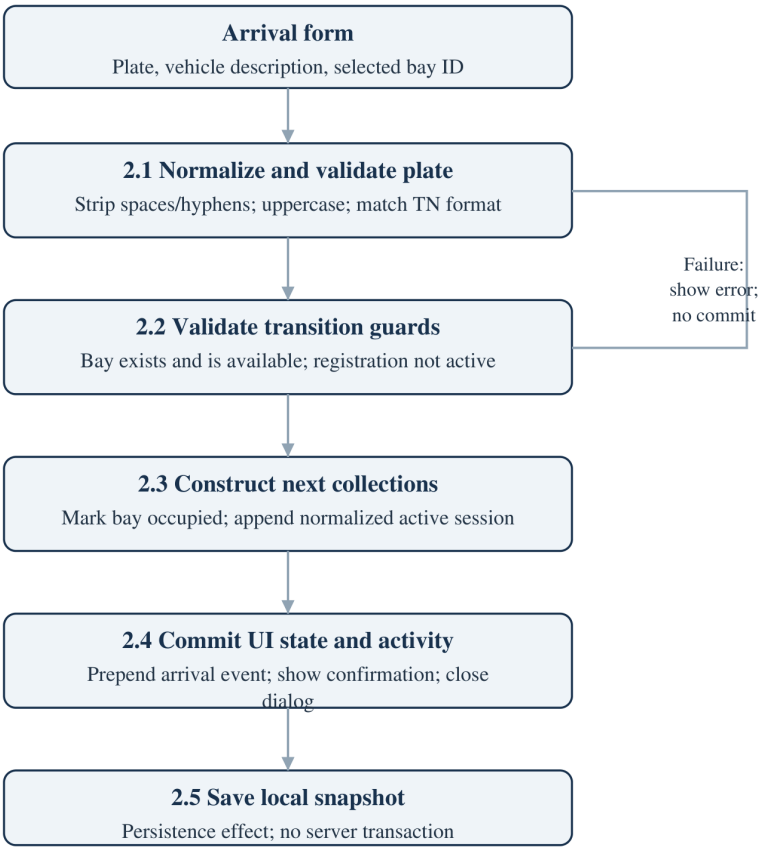



**Figure 4.4: Arrival validation, collection transformation and UI commit**

The session stores the bay and level identifiers, normalized plate, trimmed vehicle description and supplied start timestamp. A blank description becomes “Vehicle”. The bay retains its physical kind while gaining occupied status, plate and `since` timestamp. The dashboard commits the returned collections together in its location operation state and prepends an arrival activity item.

Any thrown error is displayed as feedback while the successful-state update is skipped. The function does not reserve a bay while the form is open, and the browser copy is not protected against another tab changing the same resource. Consequently, the diagram represents a local transition, not a concurrent reservation or transactional server workflow.

## 4.4 Use Case Diagram and Actor Specification

The implemented actor is an operator using the dashboard. The visible profile says “Garage manager”, but there is no sign-in or permission check. All supported actions are available to a user who can open the application. The diagram therefore uses one actor rather than inventing authenticated administrator and customer roles.

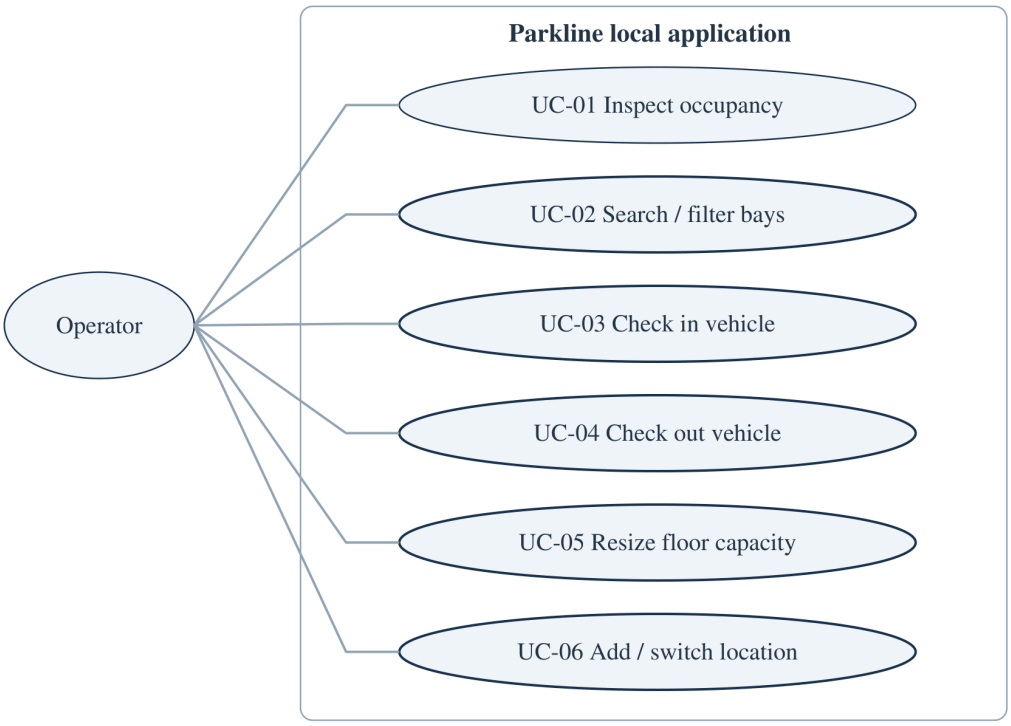



**Figure 4.5: Operator use cases within the local MVP boundary**

UC-01 covers observing location and floor occupancy. UC-02 covers current-floor search and status filtering. UC-03 starts a session only after the registration and bay guards pass. UC-04 ends a selected active session. UC-05 changes planned bay counts while preserving protected resources. UC-06 creates a new location or changes the selected garage.

Viewing reserved inventory is included within inspection. There is no use case for making a reservation because no such transition is implemented. Similarly, EV and accessible icons describe bay kind; the application does not verify driver eligibility or manage charging sessions.

The use cases define the current educational demonstration. A future production actor model should be determined through stakeholder requirements, such as gate operator, capacity planner, supervisor and auditor. Those responsibilities cannot safely be inferred from the present static profile.

### 4.4.1 Detailed Use Case — Inspect and Start a Session

Inspection provides the context for arrival. The operator should confirm the selected location and floor before assigning a vehicle. Clicking an available bay preselects that bay in the arrival dialog. The global “New arrival” button chooses the first available bay on the current level when possible, while the dialog's selector lists available bays from the selected location.

**Table 4.1: Use case UC-03 — Check in a vehicle**

| Element | Specification |
|---|---|
| Primary actor | Operator; no authenticated role is enforced |
| Trigger | “New arrival” button or click on an available bay |
| Preconditions | Selected location exists; target bay is available; plate meets TN policy |
| Input | Registration, optional vehicle description, bay ID |
| Main step 1 | Open arrival dialog and preselect a suitable bay |
| Main step 2 | Normalize registration by removing spaces/hyphens and uppercasing |
| Main step 3 | Reject a missing/non-available bay or an active duplicate registration |
| Main step 4 | Create a session and map the chosen bay to occupied |
| Main step 5 | Commit collections; prepend arrival activity; show confirmation |
| Success postcondition | One additional active session; occupied +1; available −1 |
| Conserved values | Total capacity, reserved count, bay ID and bay kind |
| Failure postcondition | Domain exception is shown; successful-state commit is skipped |
| Persistence boundary | Later effect saves locally; no server transaction or gate action |

An important alternate flow is a validly formatted registration that is already active. The UI does not close the dialog; it shows the duplicate message. Another alternate flow is a bay that is no longer available in the collection supplied to the helper. The helper rejects it even if the form was opened earlier with that bay selected.

These guards protect consistency only within the input snapshot. A future backend must check current authoritative state and commit the bay/session association atomically. The local use case remains valuable as an executable specification for that future transaction.

### 4.4.2 Detailed Use Case — Complete a Departure

Departure is initiated by the top-level “Check out” action or an occupied bay. The dashboard tries to select a session associated with the clicked bay, then falls back to a session on the current level. The visible selector lists only sessions whose `levelId` matches the selected level. An operator must switch floors to check out a session from another level through this UI.

**Table 4.2: Use case UC-04 — Check out an active session**

| Element | Specification |
|---|---|
| Primary actor | Operator |
| Trigger | “Check out” action or occupied-bay click |
| Preconditions | An active session is available on the current level |
| Input | Selected active session ID |
| Main step 1 | Resolve the selected session from current location operations |
| Main step 2 | Call `checkOut` with the current bay and session collections |
| Main step 3 | Mark the session's bay available and clear plate/since fields |
| Main step 4 | Remove the active session from the returned session list |
| Main step 5 | Prepend departure activity, show confirmation and close dialog |
| Success postcondition | Occupied −1; available +1; session no longer active |
| Conserved values | Total inventory, reserved count and physical bay kind |
| No-session path | Display a hint and disable completion on a level with no sessions |
| Missing-ID path | Reject an unknown active session |
| Explicit exclusions | No fee, payment, receipt or completed-session archive |

The helper assumes a reasonably consistent input model. It finds the session but does not independently throw if its referenced bay is missing; mapping an unrelated bay list would simply leave that list unchanged while removing the session. This is a useful runtime-validation and test gap, not a behavior to hide behind the ideal use-case description.

A future model should record departure time and preserve a completed session rather than delete its historical record. It should also validate the bay/session relationship within a transaction. Those changes are proposals and are not included in the current use case.

### 4.4.3 Detailed Use Cases — Capacity and Location Planning

UC-05 accepts draft capacities for all current levels. The save handler calculates prospective bays and levels before committing them. Any error prevents the final update. Within each floor, the helper rejects a target below the protected count, removes highest-numbered available bays when shrinking and fills the lowest unused numeric suffixes when expanding.

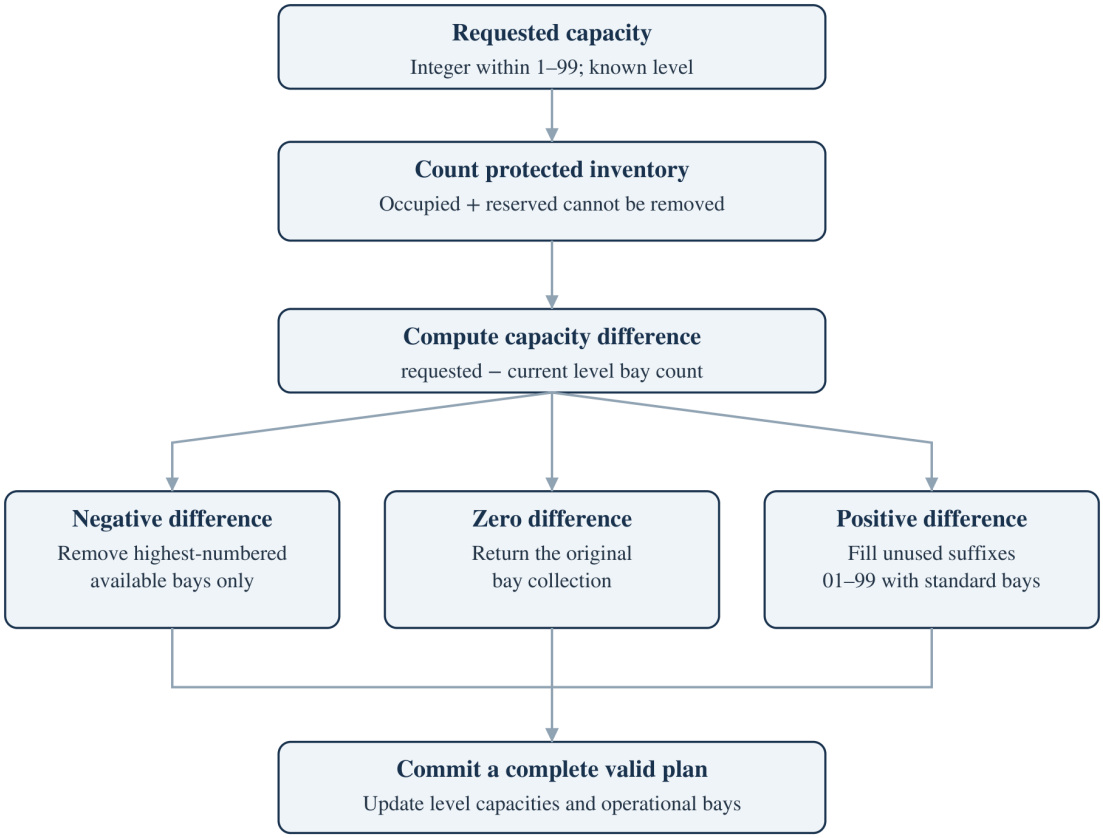



**Figure 4.6: Protected capacity-resizing decision workflow**

UC-06 creates a trimmed, non-blank location name with validated floor and per-floor counts. The dashboard separately rejects case-insensitive duplicate names. A successful creation adds both a location object and an operational-map entry, selects the new location, resets the current floor/search/filter and displays its total capacity. Every new bay begins as standard and available; sessions and activity begin empty.

Capacity changes do not add or remove floors from an existing location. New-location creation is the only implemented way to choose a different floor count. A reduction can leave gaps in bay numbering because protected bays retain their original identities. That is an intentional safety consequence, not an instruction to renumber active resources.

## 4.5 Object-Oriented and Behavioral Models

### 4.5.1 Domain Interface / UML Class View

The source uses TypeScript interfaces rather than class instances with methods. The UML-style view below is therefore a structural model of record shapes, not a claim that the application instantiates an object-oriented service hierarchy. Locations aggregate arrays of levels, bays, sessions and activity [2].

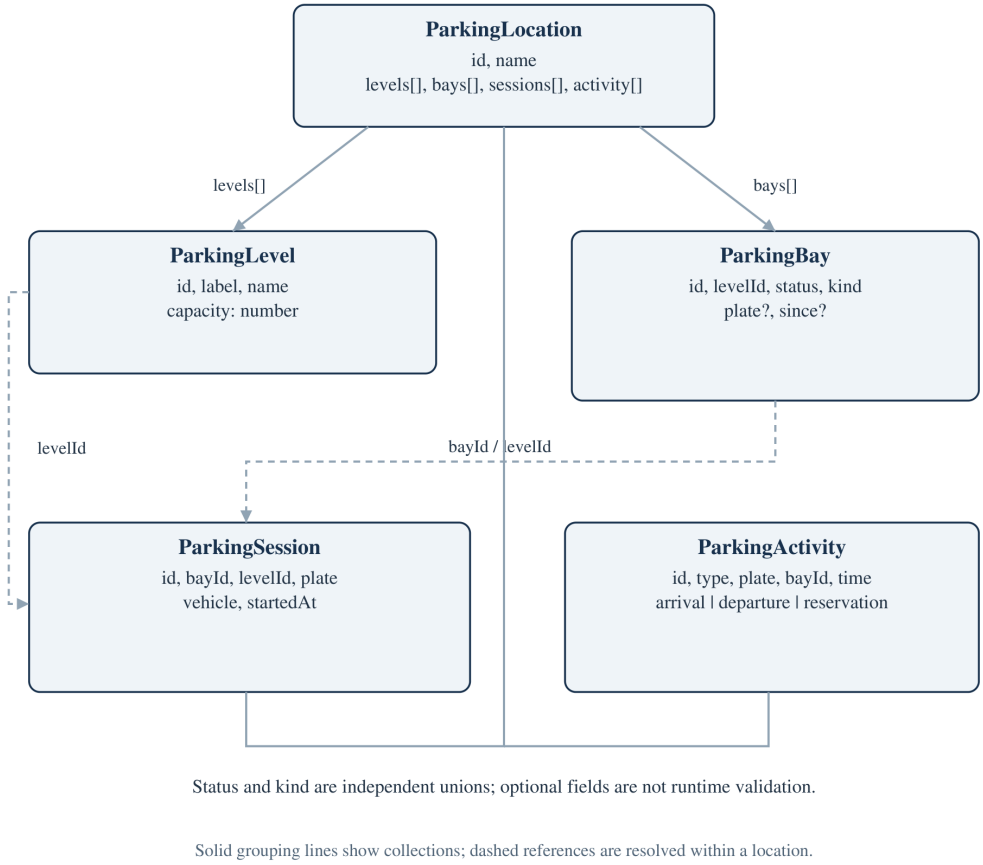



**Figure 4.7: Actual TypeScript domain records and location-scoped references**

A level has a stable identifier and separate presentation labels. A bay has its own identifier and `levelId`, while a session stores both `bayId` and `levelId` for display and filtering. Activity references a bay by text and stores a human-readable time label. These relationships are resolved by array lookup rather than database foreign keys.

The model's optional fields provide flexibility for available and reserved bays, but the interface alone does not enforce a state-dependent record shape. For example, TypeScript does not require an occupied bay to carry a plate. A stronger future design could use discriminated unions or a runtime schema to express that relationship. It would still require consistency checks between a bay and its session collection.

### 4.5.2 Sequence Model — Arrival and Local Save

The sequence diagram separates operator interaction, dashboard orchestration, pure domain processing and browser persistence. It highlights that saving is an effect following a state change, not a prerequisite that the domain helper awaits.

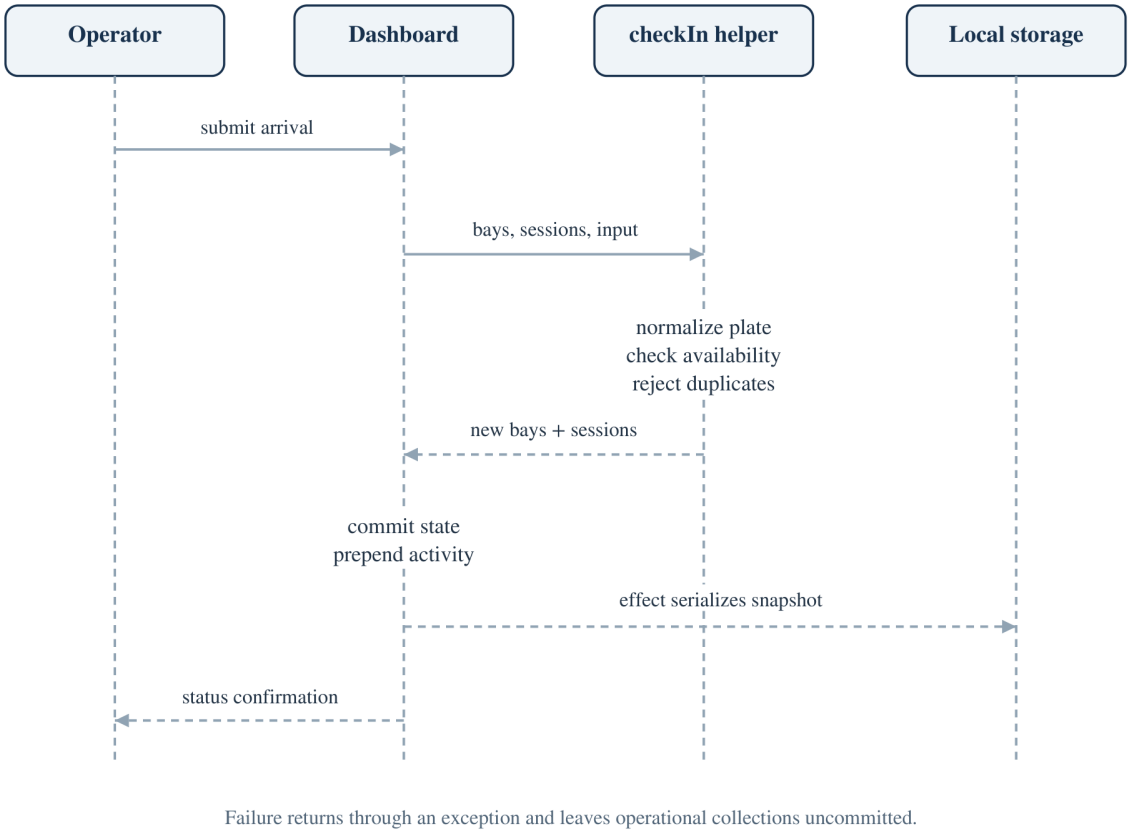



**Figure 4.8: Arrival sequence from form submission to local snapshot**

The dashboard generates a current ISO timestamp for the UI-driven arrival and supplies it to `checkIn`. The helper constructs an identifier using the bay ID and parsed time. If validation succeeds, the dashboard updates the selected location's operations, adds activity and closes the dialog. The user sees a confirmation identifying both plate and bay.

If normalization or a transition guard fails, an exception returns control to the dashboard's catch block. No successful collection update is performed. The same structural pattern applies to departure, although departure resolves a session rather than validates a new plate.

The storage effect can fail independently of the visible transition. Its catch path logs a warning, and the UI does not roll back the action or present an explicit save-failure banner. As a result, a successful check-in notice should not be interpreted as proof that durable persistence succeeded. A future implementation should define persistence status, recovery and authoritative commit semantics explicitly.

### 4.5.3 Bay State Machine and Invariants

The operational state machine is deliberately small. `checkIn` changes available to occupied, and `checkOut` changes the session's bay to available. Reserved is represented in fixtures and display logic, but there is no implemented reservation lifecycle. Drawing a reserve/cancel arrow would therefore misstate the code.

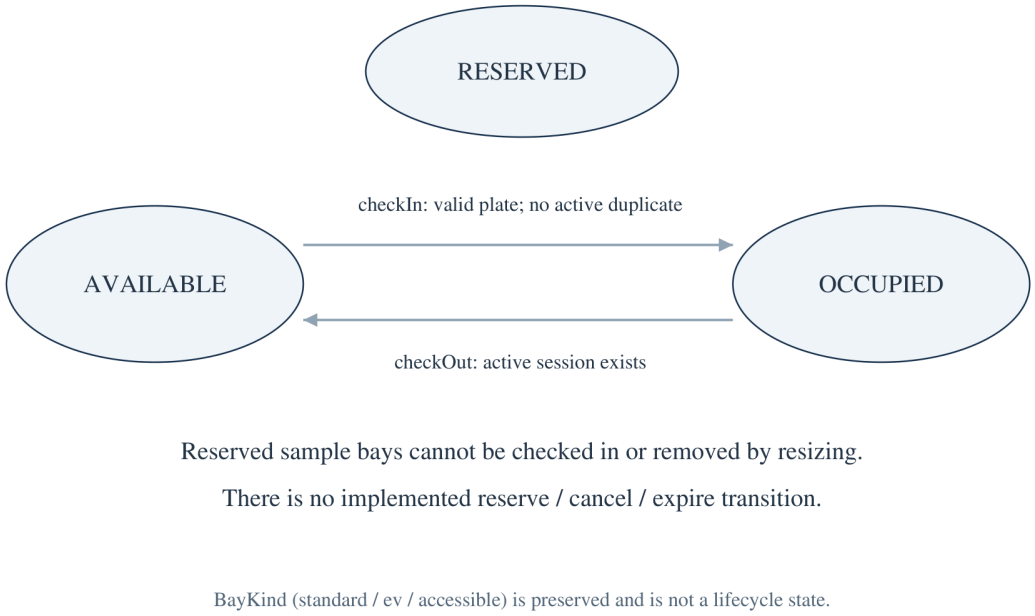



**Figure 4.9: Implemented bay-state transitions and reserved-state boundary**

For a level with total T, occupied O and reserved R, the helper derives available A as $T - O - R$. Occupancy percentage is round($100 \times O / T$), with zero used when T is zero. Reserved bays do not contribute to parked-vehicle occupancy. This distinction makes a floor with many reservations appear less occupied while still correctly limiting available bays.

The intended consistency invariant is that each active session refers to one occupied bay within its location, and a normalized plate occurs at most once in that location's active sessions. The helper guards support these invariants for valid input snapshots. They do not repair arbitrary inconsistent data restored from storage, nor do they coordinate simultaneous clients.

Bay kind is independent of the state machine. An accessible or EV bay remains that kind after arrival or departure. Capacity expansion creates standard bays only, while shrinking can remove any available bay kind because no separate protection rule for accessible or EV inventory is implemented. A production planning policy would need to define such protection explicitly.

## 4.6 Conceptual Entity Relationships

The project has no relational database. The following ER-style view explains logical relationships already represented in arrays and identifiers. It should not be read as an implemented SQL schema, a migration plan or evidence of enforced database constraints.

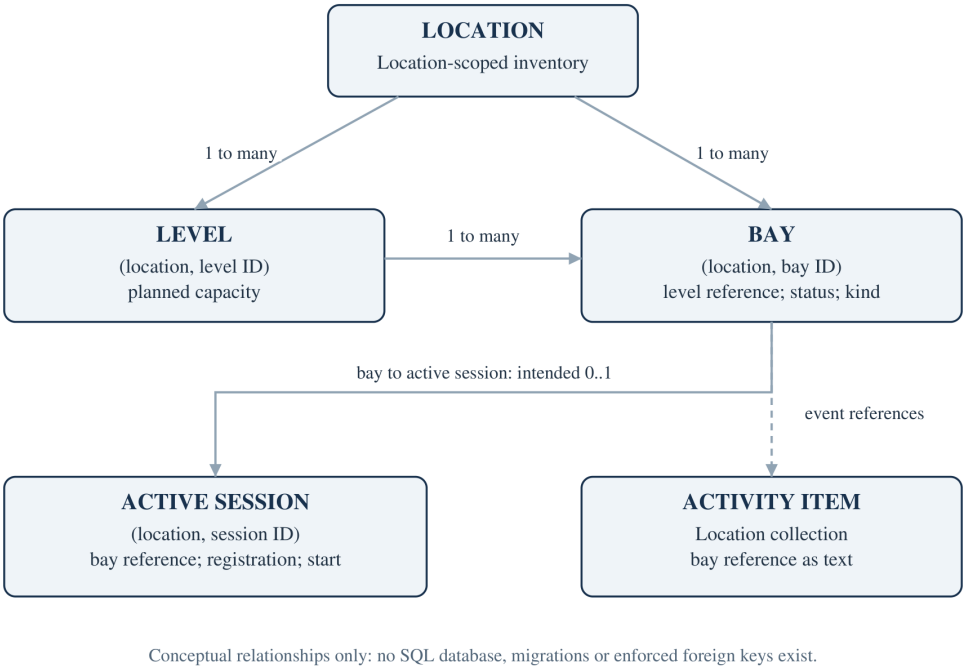



**Figure 4.10: Conceptual relationships of the in-memory parking model**

A location contains many levels and bays. A level can be associated with many bays. A bay is intended to have zero or one active session, while a session identifies one bay and one normalized registration. Activity is a lightweight local collection referencing a bay; it is not a complete immutable event ledger.

Identity is scoped carefully. Seeded locations reuse identifiers such as B1 and B1-01. The location ID selects the collection in which those identifiers are meaningful. A production relational design would need either globally unique keys or composite uniqueness that includes location. Creating a single global bay table keyed only by B1-01 would incorrectly collapse separate garage inventories.

Session IDs are also demonstration identifiers rather than distributed unique IDs. Initial fixtures and newly generated sessions use different construction patterns. A future authoritative service should choose an identity strategy and validate foreign-key relationships rather than assume browser-generated strings are globally collision-free.

The conceptual diagram is useful because it exposes missing guarantees without pretending they already exist. It also gives a foundation for future database design after operational policies, retention and concurrency requirements are agreed.

## 4.7 Data Dictionary of Implemented Records

### 4.7.1 ParkingLocation and ParkingLevel

The data dictionary records exact interface field names and their current interpretation. It is a TypeScript/in-memory dictionary, not a table specification for an existing database. All identifiers are strings, capacities are numbers and collections are JavaScript arrays.

**Table 4.3: Data dictionary — location and level records**

| Record / field | Type | Meaning and current constraint |
|---|---|---|
| ParkingLocation.id | string | Key used to select operational collections; new values use current time |
| ParkingLocation.name | string | Display name; new names trimmed and checked for non-blank input |
| ParkingLocation.levels | ParkingLevel[] | Floor definitions and planned capacities |
| ParkingLocation.bays | ParkingBay[] | Nested initial/planning inventory; operational map may be newer |
| ParkingLocation.sessions | ParkingSession[] | Nested sessions; working sessions are read from operations map |
| ParkingLocation.activity | ParkingActivity[] | Nested activity; working activity is read from operations map |
| ParkingLevel.id | string | B1 or L-number in managed creation; scoped to a location |
| ParkingLevel.label | string | Compact tab label such as B1, 01 or 02 |
| ParkingLevel.name | string | Basement, Ground floor or generated Level name |
| ParkingLevel.capacity | number | Planned bay count; validated as 1–99 during supported changes |

The distinction between nested location collections and `operationsByLocation` matters. Session handlers update the operational map, not every nested copy inside `garageLocations`. Capacity saving updates nested location levels and bays and also changes the operational map's bays. Saving both representations can therefore retain a mixture of newer operational data and older nested values.

The dashboard normally reads the operational map for the selected location, falling back to nested collections when no entry exists. The fallback is convenient but raises consistency questions. A future format should define one canonical source and derive redundant views.

At creation, floor count is validated once and used to generate all levels and bays. Existing locations do not expose a floor-count editing operation, so the dictionary should not be interpreted as implying arbitrary structural editing.

### 4.7.2 ParkingBay, BayStatus and BayKind

The bay is the core resource record. It combines identity, floor association, current availability state and a physical classification. Optional vehicle fields are present when fixtures or successful arrivals populate them and are cleared during departure.

**Table 4.4: Data dictionary — bay fields and enumerated values**

| Field or type | Type / values | Interpretation |
|---|---|---|
| ParkingBay.id | string | Location-scoped floor ID plus two-digit bay suffix |
| ParkingBay.levelId | string | Identifier of the associated floor |
| ParkingBay.status | BayStatus | Operational availability category |
| ParkingBay.kind | BayKind | Physical classification, separate from status |
| ParkingBay.plate | optional string | Vehicle registration shown for occupied seed/arrival records |
| ParkingBay.since | optional string | ISO timestamp used for the current occupancy start |
| BayStatus | available | Can be selected for an ordinary arrival |
| BayStatus | occupied | Represents a currently parked vehicle |
| BayStatus | reserved | Unavailable sample reservation; no lifecycle controls |
| BayKind | standard | Default kind, including all newly created bays |
| BayKind | ev | Visual EV indicator; no charging integration |
| BayKind | accessible | Visual accessible indicator; no eligibility enforcement |

The ID suffix allocation rule is bounded by 99. Expansion scans candidate suffixes from one upward and skips those already in use. Shrinking sorts available bays by descending numeric suffix and removes the required number. The algorithm preserves protected bay IDs and does not compact numbering after a reduction.

Optional fields do not amount to a state-dependent schema. An externally modified saved record could claim occupied status without a plate or point to an unknown level. Such states are not created by the intended successful helper paths, but the restoration code does not comprehensively reject them. Runtime validation is therefore a material future requirement.

The current resize policy does not preserve a minimum number of EV or accessible bays. It protects status, not kind. That is acceptable as a documented prototype rule but requires stakeholder review before any real capacity-planning decision is based on the application.

### 4.7.3 ParkingSession and ParkingActivity

An active session represents present occupancy, not a permanent visit history. Checkout removes the session from the active array. Activity provides a recent-event display, but it lacks the structured timestamps and actor attribution needed for an audit ledger.

**Table 4.5: Data dictionary — active session record**

| Field | Type | Meaning |
|---|---|---|
| id | string | Session selector key; new IDs contain bay ID and parsed start time |
| bayId | string | Bay occupied by the session within the selected location |
| levelId | string | Floor used for display and departure filtering |
| plate | string | Normalized registration; locally unique among active sessions |
| vehicle | string | Trimmed description or fallback “Vehicle” |
| startedAt | string | Supplied ISO-style start timestamp; helper does not validate it |

**Table 4.6: Data dictionary — recent activity record**

| Field | Type / values | Meaning |
|---|---|---|
| id | string | UI list key; new event IDs use current time |
| type | arrival / departure / reservation | Determines event label and icon |
| plate | string | Registration or sample display text such as “Reserved” |
| bayId | string | Bay reference displayed beside the event |
| time | string | Human-readable text such as “Just now” or “2 min ago” |

The activity panel shows only the first five items using an array slice, although the underlying collection can continue growing. New events are prepended, so recent user operations displace older items from the visible list without deleting them from storage. There is no pagination or historical export.

A future history model should separate a structured event time from its localized display label, record actor identity and preserve completed sessions. It should also define retention and deletion policies for registration data. These are proposed changes, not additional fields already present in the current interfaces.

### 4.7.4 Browser Storage Structure and Restoration

The storage key contains one serialized object with locations, operational collections and the selected location ID. The selected floor, search text and status filter are not persisted; restoration chooses the saved location's first level. The schema has no explicit version field.

```
{
  "locations": ["ParkingLocation records"],
  "operationsByLocation": {
    "riverside": { "bays": [], "sessions": [], "activity": [] }
  },
  "locationId": "riverside"
}
```

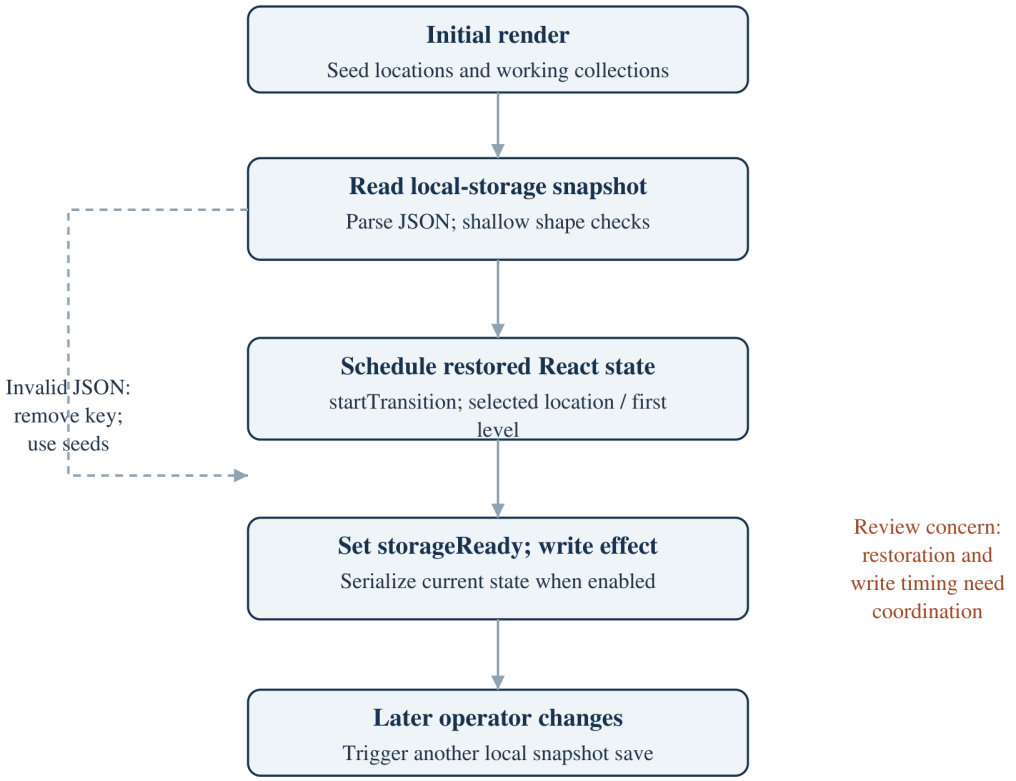



**Figure 4.11: Current local-storage load/write lifecycle and review concern**

The illustration is schematic: real saved arrays contain full records, not the descriptive string in the example. The read effect catches parse failures and attempts to remove the key. The write effect catches save failures and logs a warning. Neither path is a complete recovery protocol, and shallow shape checks cannot establish referential consistency. Restoration/write ordering is a review concern supported by the failed browser reload scenario.

## 4.8 Input and Output Design

Input design constrains operator entry while preserving clear error recovery. Forms use native required fields and numeric bounds, but business rules remain in domain helpers. An arrival's registration input permits common spacing within a maximum length, then normalizes before validation. Vehicle description is optional. Planning values are converted from string drafts into numbers only when the form is submitted.

**Table 4.7: Input controls, derived outputs and interpretation**

| Area | Input / control | Output and scope |
|---|---|---|
| Garage selection | Native location select | Selected site's totals, first floor and operations |
| Floor selection | Tabs and next-level button | Current-floor map and active departure choices |
| Search | Trimmed lowercase substring | Current-floor ID/plate matches only |
| Status filter | all / available / occupied / reserved | Current-floor matching bays; no data mutation |
| Arrival | Plate, description, available-bay select | Occupied bay, active session and arrival notice |
| Departure | Active session from current level | Released bay and departure notice |
| Capacity | Integer draft for each existing level | Updated planned capacities and actual bay inventory |
| New location | Name, floor count, bays per floor | New empty location and total-capacity preview |

Output design distinguishes derived values from decoration. Total, occupied, available and reserved cards derive from current bay records. Floor occupancy percentage excludes reserved bays. The EV charging note, yesterday comparison, next-arrival message, manager name and displayed date include static content. They are not historical analysis or connected telemetry.

The empty-search state explicitly says that no bays match the search. A reserved-bay click reports the reserved state rather than opening an arrival. Success notices identify the affected registration or capacity, and validation errors remain close to the corresponding form. These patterns support operational clarity, while the static labels remain a transparency improvement opportunity.

## 4.9 User Interface Design System

The interface uses a neutral canvas, white panels, green primary actions and amber reservation accents. Its source of visual truth is `src/app/globals.css`, with Tailwind directives available through the build pipeline. Many application-specific components are styled through ordinary CSS class selectors rather than large inline utility strings.

**Table 4.8: Implemented visual tokens and layout conventions**

| Token or convention | Source value / behavior | Purpose |
|---|---|---|
| Main ink | `#1b2825` | Primary text |
| Soft ink | `#485752` | Supporting text |
| Muted | `#88938f` | Secondary labels |
| Canvas | `#f5f7f5` | Dashboard background |
| Paper | `#ffffff` | Cards and panels |
| Primary green | `#157b5b` | Main actions and active navigation |
| Bright green | `#46c692` | Accent line and positive details |
| Amber | `#c58229` | Reservation-related emphasis |
| Radius | `10px` base token | Consistent panel corners |
| Desktop sidebar | 238-pixel base width | Branding, garage selector and navigation |
| Bay grid | Six columns normally; four at narrow width | Compact visual inventory |
| Responsive breakpoints | 1150, 900 and 620 pixels | Reduce sidebar and stack panels |

The primary actions are grouped at the page heading: planning, checkout and arrival. The parking map is the main working panel, while floor summaries and recent activity provide context. Native selects are used where a set of choices must remain explicit, such as location, status and session.

The bay grid is a visual inventory, not an accurate architectural drawing of a physical floor. It has no coordinate geometry, obstruction data or navigation routing. The one-way lane graphic is a visual cue rather than a model of validated circulation. This distinction matters if the application is later extended into genuine facility planning.

Animations and hover effects support feedback, and a reduced-motion media query shortens animation and transition durations. These styling provisions improve presentation but do not by themselves demonstrate accessibility conformance or mobile usability; the recorded overflow finding remains open.

## 4.10 Accessibility, Responsive Design and Design Limitations

The design includes useful foundations: semantic buttons, native selects, associated labels, visible focus outlines, text alternatives in bay names, status feedback and alert-marked form errors. Floor and planning controls declare tab semantics, and dialogs declare modal roles with labelled headings. These are source-backed implementation facts, not a complete accessibility audit.

**Table 4.9: Accessibility and responsive review by interaction**

| Interaction | Implemented support | Remaining limitation |
|---|---|---|
| Bay inspection | Accessible name includes ID/status/plate | Color contrast and screen-reader experience not comprehensively audited |
| Forms | Visible labels, required fields and native controls | Error association could be more explicit |
| Dialogs | Dialog role, modal flag, autofocus on selected fields | No focus trap, Escape close or focus return implementation |
| Floor tabs | Tablist and selected-state semantics | No full arrow-key / roving-tabindex pattern |
| Feedback | Status notices and alert errors | Save failure is only a console warning |
| Motion | Reduced-motion stylesheet | No user study of animation effects |
| Narrow layout | Panels stack; sidebar collapses; four-column bay grid | Location selector hidden; 390-pixel overflow observed |

The mobile observation is specific and measurable: in the documentation run, a 390-pixel viewport had a document width of 457 pixels. The heading's primary action button extended to the right edge of that wider document. The report preserves the captured mobile screen and does not label the interface “fully responsive” solely because media queries exist.

The design also contains intentionally limited controls. Notification, account and activity-options buttons do not implement full menus or back-office functions. Sidebar links navigate to sections of the same page rather than separate management routes. These choices are acceptable within the MVP if they are described accurately.

The next design iteration should prioritize dialog interaction, narrow-width action wrapping, visible location selection and honest sample-data labeling. A later shared-service iteration should then add authorization, server authority and durable history. Keeping those stages separate avoids treating visual cleanup as a substitute for operational safety.

# CHAPTER 5
# TECHNOLOGIES AND TOOLS USED

## 5.1 Software Requirements and Dependency Baseline

Parkline is built with Next.js, React, TypeScript and a CSS/PostCSS toolchain. The package manifest declares the application scripts, while `package-lock.json` records resolved dependencies. The report distinguishes exact manifest pins from version ranges: an entry such as `^5.7.2` is an allowed range, not a claim that every installation uses precisely 5.7.2 [7].

**Table 5.1: Technology baseline declared by the repository**

| Category | Package / tool | Manifest declaration | Role |
|---|---|---|---|
| Application framework | next | 16.4.3 | App Router, development server and production build |
| UI runtime | react / react-dom | 19.0.0 / 19.0.0 | Components, hooks and DOM rendering |
| Language tooling | typescript | ^5.7.2 | Static type checking and build integration |
| Styling | tailwindcss | ^3.4.17 | CSS utility framework and directives |
| CSS processing | postcss / autoprefixer | ^8.5.32 / ^10.4.20 | Stylesheet processing and prefix handling |
| Icons | lucide-react | ^0.468.0 | Consistent interface icons |
| Linting | eslint / eslint-config-next | ^9.39.5 / 16.4.3 | Configured source checks |
| Unit testing | vitest / vite | ^5.0.5 / ^7.3.7 | Node-based tests and transform toolchain |
| Browser testing | @playwright/test | ^1.64.1 | End-to-end operator scenarios |

The documentation environment used Node.js 22.22.3 and npm 10.9.8. A machine-readable resolved-version inventory is included with the report source. These environment facts support reproducibility but do not imply that another lockfile or future dependency update will produce identical results.

No database driver, authentication library, payment SDK or sensor integration package is declared for the current application. A framework dependency alone does not establish such capabilities. The technology chapter therefore explains the actual frontend and verification stack instead of substituting the reference report's unrelated Java and database technologies.

## 5.2 Hardware and Execution Environment

The application is a browser-based MVP rather than hardware-dependent parking equipment. It does not require a gate controller, occupancy sensor, camera, RFID reader or charging-station controller. The documentation was prepared by running a local production build and opening it with headless Chromium for repeatable screen capture.

The repository specifies Node.js 20.9 or newer. It does not publish measured minimum RAM, CPU, disk or browser-version requirements. Consequently, the hardware guidance below is a practical planning recommendation, not an experimentally established minimum.

**Table 5.2: Execution needs and unmeasured planning guidance**

| Environment | Required capability | Qualification |
|---|---|---|
| Development workstation | Supported Node.js, npm, editor and modern browser | Repository states the Node baseline; no hardware benchmark provided |
| Dependency installation | Network access to the npm registry and adequate disk | Lockfile dependencies must be downloadable |
| Unit/static verification | Ability to run Node processes | Browser installation not required for Vitest, lint or type checks |
| Browser verification | Compatible Chromium or another configured Playwright browser | Browser binaries and platform libraries are separate prerequisites |
| Demonstration server | Ability to bind and serve the Next.js application | A local server is not a shared parking backend |
| End-user browser | JavaScript and usable local storage | Data belongs to that browser profile and origin |

For a remote preview, the server should bind to an externally reachable interface and accept the deployment host. Browser-facing requests must use the served application origin, not a developer's localhost address. The current dashboard does not call a separate backend, which simplifies preview networking.

A future field installation would need an independent infrastructure specification: terminal hardware, network connectivity, power recovery, shared database capacity, backup media, physical integrations and monitoring. None of those requirements can be validated from the present UI alone.

The report's screenshots are evidence of software rendering in the tested Chromium environment. They do not certify kiosk suitability, low-end-device performance, touch ergonomics or behavior during power and network failures.

## 5.3 Programming Languages Used

### 5.3.1 TypeScript for Domain and Application Logic

TypeScript is used for the domain records, pure transition helpers, fixture generation, dashboard state and test code. The compiler configuration enables strict checking and no-emit type verification. The `@/*` path alias resolves to the source directory, reducing fragile relative imports and keeping UI-to-domain dependencies readable.

The domain's union types constrain status and kind vocabularies, while interfaces make relationships explicit. Function signatures document the inputs required for a transition and the paired collections returned afterward. For example, a check-in cannot be called without a bay ID, registration, vehicle string and start timestamp according to its source type. That compile-time contract still permits invalid string content, so runtime guards remain necessary.

### 5.3.2 TSX and Declarative Markup

The dashboard uses TSX to express the visual hierarchy and attach typed event handlers. A form submission prevents the default browser action, calls a transition or planning helper and updates React state. Rendering is conditional: a dialog appears only when its mode is active, a notice appears when text is present and an empty-map message appears when filtering produces no bays.

### 5.3.3 CSS and Configuration Data

CSS defines the visual tokens, component layouts, responsive breakpoints, focus styles and reduced-motion behavior. JSON is used in package manifests, TypeScript configuration and browser persistence. The saved JSON is a serialization format, not a database schema and not a runtime type validator.

The report-generation utilities use Python and a small Node-based capture script as documentation tooling. They are not part of the parking application's runtime stack. This distinction matters because producing a PDF should not imply that the application itself has a Python backend.

The language choices support a single main frontend toolchain with focused verification. Future server code could reuse domain concepts, but security and concurrency requirements would still need to be enforced at the new trusted boundary rather than inherited automatically from TypeScript declarations.

## 5.4 React State, Derived Values and Effects

React supplies `useState`, `useMemo`, `useEffect` and `useTransition` in the dashboard. State hooks hold the selected location and level, the operation collections, search/filter values, dialog inputs, notices and planning drafts. A state update causes the visible interface to be recalculated from the current snapshot [5, 9].

Derived values include the selected location, its current operation collections, the selected level's bays, floor statistics and total statistics. `filteredBays` is calculated with `useMemo`, combining the status condition with an ID/registration substring check. The memo does not make the whole dashboard constant-time: `levelBays` is constructed during render, and many counts still scan arrays. This is a reasonable small-data design, not a demonstrated optimization guarantee.

Effects manage the browser storage boundary. The first effect reads and parses a saved value, then schedules restored state through a transition. A second effect writes serialized state once the readiness flag is enabled. This keeps side effects out of the pure domain helpers but creates a responsibility to coordinate initialization and saving correctly. The recorded reload failure shows why effect presence alone is insufficient evidence of reliable restoration.

`useTransition` marks restoration-related state updates as transition work. It is a rendering-scheduling mechanism, not a data transaction, lock or durability guarantee. The flag named `storageReady` prevents unconditional writes before the read path completes, but the interaction between deferred restoration and later writes needs a regression-focused review.

A maintainability improvement would be to isolate persistence into a dedicated hook with an explicit loading state, schema validation and a canonical stored representation. Session and capacity UI could then become smaller components. Such a refactor should preserve the existing domain helper tests and add repeated-load tests before changing behavior.

React's role in this project is therefore state-driven presentation. Authority over physical inventory, authenticated identity and multi-user conflict resolution remain outside the current browser architecture.

## 5.5 Next.js Routing, Layout and Build

Next.js provides the application entry point, build pipeline and serving commands. The App Router files in `src/app` contain a root layout and a home page. The page imports and returns `ParkingDashboard`; the layout imports global CSS, sets metadata and wraps children in an English-language HTML document. The production build classified `/` as a prerendered route in the recorded run [8].

There are no parking API route handlers, server actions or database repositories in this source tree. The root page can be server-composed while the dashboard hydrates into an interactive client component. That combination should not be described as a full client-server parking operations backend.

**Table 5.3: Application entry files and relevant configuration**

| File | Responsibility | Important interpretation |
|---|---|---|
| `src/app/page.tsx` | Render the dashboard route | No parking business logic in the route |
| `src/app/layout.tsx` | Metadata, global CSS and root document | Description includes “Live”; not proof of telemetry |
| `src/app/globals.css` | Global and component styling | Responsive presentation, not physical floor geometry |
| `next.config.ts` | Next.js configuration | Development origin setting is not authentication |
| `next-env.d.ts` | Generated framework type declarations | Tooling support, not domain data |
| `tsconfig.json` | Compiler settings and source alias | Static checking only |

The scripts separate development, production compilation and production serving. `npm run dev` starts the development server. `npm run build` creates production output. `npm run start` serves that output. Production screenshots in this report were captured after a successful build, avoiding development overlays as a source of visual evidence.

The repository's agent guidance explicitly warns that installed Next.js documentation should be consulted before changing framework code. This documentation task did not need to alter route behavior or framework configuration. The report uses the installed documentation to explain the current boundary rather than assuming an older Next.js convention.

## 5.6 Styling, PostCSS and Iconography

Tailwind CSS is configured through `tailwind.config.ts` and included with base, component and utility directives at the beginning of the global stylesheet. PostCSS and Autoprefixer form part of the stylesheet build pipeline. The dashboard's main visual language, however, is expressed in named CSS classes such as `metric-grid`, `parking-panel`, `bay-occupied`, `management-modal` and `form-error` [16].

This arrangement combines a framework foundation with application-specific component styling. CSS custom properties define the colors, radius and shadow so that repeated panels and controls remain visually coherent. The desktop layout separates a fixed-width sidebar, a main content region and a right-hand information column. Smaller breakpoints reduce the sidebar, stack the content and change the bay grid from six columns to four.

Lucide React supplies SVG icons for locations, layers, vehicles, arrivals, departures, settings, search, accessibility and EV indicators [17]. Icons are embedded within buttons or paired with visible text. A visual icon is not an operational integration: a lightning symbol does not imply charging-station communication, and a bell does not imply a notification service.

**Table 5.4: Styling mechanisms and their practical role**

| Mechanism | Example | Role in the MVP |
|---|---|---|
| CSS variables | `--green`, `--canvas`, `--radius` | Shared visual tokens |
| State classes | `bay-available`, `bay-occupied`, `bay-reserved` | Visual occupancy categories |
| Grid and flex layout | Metrics, bay map, heading actions | Adaptive arrangement of panels and controls |
| Media queries | 1150 / 900 / 620 pixels | Breakpoint-specific presentation |
| Focus-visible outline | Buttons, links, inputs and selects | Keyboard focus indication |
| Reduced-motion query | Animation/transition overrides | Reduced visual motion preference |
| SVG components | `CarFront`, `Zap`, `Accessibility` | Consistent pictorial cues |

The observed mobile overflow illustrates a limitation of breakpoint-based styling: changing grid structure does not automatically guarantee that every child can shrink or wrap. Verification must inspect the rendered geometry and real action controls, not simply count media queries.

## 5.7 Unit, Browser and Static Verification Tools

Vitest is configured to run `src/**/*.test.ts` in a Node environment. The suite imports the pure domain helpers and managed-location data directly. This keeps tests fast and independent of browser layout. The existing file contains nine test cases covering occupancy, normalization through check-in, rejection paths, checkout, resizing, fixture consistency and new-location construction [6, 14].

Playwright is configured with the `e2e` test directory, a Chromium desktop project, a base URL and a web-server command for local development. It exercises three operator workflows. During documentation, an environment-specific runner used an available Chromium binary and the already running production server; the test scenario source itself was not changed. The recorded final run had two passing scenarios and one failed reload assertion [15].

ESLint uses the Next.js core-web-vitals and TypeScript configurations. TypeScript's `--noEmit` mode checks source without producing application JavaScript. The production build adds another compilation and route-generation gate. These tools answer different questions and are reported separately.

**Table 5.5: Verification commands and evidence categories**

| Command / script | Evidence category | Documentation-run result |
|---|---|---|
| `npm test` | Existing unit tests | Nine passed |
| `npm run lint` | Configured static lint | Passed |
| `npm run typecheck` | Source type consistency | Passed |
| `npm run build` | Production compilation | Passed |
| Playwright existing scenarios | Browser integration | Two passed; one failed after reload |
| `capture-report-evidence.mjs` | Supplemental report checks/screens | Fifteen passed; one mobile overflow check failed |

The supplemental script is documentation tooling. It is retained for reproducible screen capture and does not replace or inflate the original unit-test count. A successful script process exit means the evidence was collected; individual assertion statuses must be read from its JSON output. This distinction is especially important because the script intentionally records failures and continues to capture the remaining screens.

## 5.8 Development Workflow and Reproducibility

The repository uses npm scripts as the common interface for installation, verification and serving. A reproducible review starts from the checked-out revision and lockfile, installs dependencies, runs the static and unit checks, builds the application and only then captures browser behavior. The report source records the review date and source commit so that later changes are not confused with the documented baseline.

```
npm ci
npm test
npm run lint
npm run typecheck
npm run build
npm run start -- --hostname 0.0.0.0 --port 3000
```

Browser tests additionally need a browser installation and platform libraries. On a normally configured workstation, `npx playwright install chromium` provides the default browser binary. Restricted environments may require an explicitly supplied compatible binary. That environment accommodation is not a parking feature and should not be copied into production configuration without review.

The report-generation workflow reads editable metadata and Markdown chapter sources, then produces a PDF and Word document from the same content. Genuine application screenshots and deterministic diagrams are retained as assets. The generated PDF has a fixed page count verified by a PDF parser. Word pagination can vary with installed fonts and layout engine, so the PDF is the submission pagination reference.

Documentation output is kept separate from runtime application code. The report does not introduce a database dependency, alter parking rules or repair defects discovered during evaluation. This preserves the meaning of the test baseline and lets the project team decide how to prioritize the findings.

A final reproducibility caution concerns time-based fixtures. Some seed timestamps are generated at module evaluation, and the UI contains static date text. Counts and identities are suitable for deterministic checks, but relative ages and generated timestamp-based IDs may differ between runs. The documentation therefore records expected invariants rather than pretending every serialized byte is stable.

# CHAPTER 6
# SYSTEM IMPLEMENTATION

## 6.1 Implementation Overview and Source Inventory

The implementation is intentionally small enough to trace an operator action from the screen into a domain function. The source inventory below identifies the files that actually contribute to the parking application. It does not imply separate deployable services or modules beyond the Next.js application.

**Table 6.1: Application and test source inventory**

| Source path | Responsibility |
|---|---|
| `src/app/page.tsx` | Home route rendering `ParkingDashboard` |
| `src/app/layout.tsx` | Root document, metadata and stylesheet import |
| `src/app/globals.css` | Design tokens, component styles and breakpoints |
| `src/components/parking-dashboard.tsx` | State, derived view, handlers, dialogs and output |
| `src/types/parking.ts` | Five record interfaces and status/kind unions |
| `src/lib/parking.ts` | Occupancy, resizing, registration, arrival and departure helpers |
| `src/lib/parking-data.ts` | Seed inventory and managed-location creation |
| `src/lib/parking.test.ts` | Nine existing domain/fixture test cases |
| `e2e/parking-workflows.spec.ts` | Three existing operator browser scenarios |

The five interfaces are `ParkingLevel`, `ParkingBay`, `ParkingSession`, `ParkingActivity` and `ParkingLocation`. Two supporting union aliases define the status and kind vocabularies. The dictionary in Chapter 4 uses those exact names, and the pure domain functions consume these shared record contracts rather than separate UI-specific copies.

A typical interaction has four stages: a control updates view state, a form handler supplies current operational collections, a helper computes a valid result or throws, and the dashboard commits the result and feedback. Local saving is a subsequent effect. The remainder of the chapter follows that flow before presenting actual output screens.

The screenshots were captured from the built application, not generated as design mockups. The browser-evidence source records the source commit, Chromium version, viewports and individual supplemental assertion outcomes. Sample registrations and location names remain demonstration data.

## 6.2 Application Initialization and State Composition

The home route returns the dashboard without business logic. At initial client state construction, `garageLocations` uses the seed locations and `operationsByLocation` is built by mapping each location ID to its bays, sessions and activity. The initially selected location and level are the first seeded entries. Search is empty, the status filter is “all” and no session or planning dialog is open.

The storage-reading effect attempts to restore a saved JSON object. When its shallow conditions pass, it schedules updates to locations and operational collections, finds the saved selected location or falls back to the first saved location, and selects that location's first level. It does not restore the last floor, search query or filter. The readiness flag is enabled after the load attempt.

The dashboard then resolves a selected location and operations. It prefers the operational map's current record; if no entry exists, it falls back to the nested collections in the selected location. The selected level is looked up in that location, and the current-level bay list is filtered from the working bay array. Because the selected-level lookup assumes success, malformed restored records can cause problems that compile-time types do not prevent.

### 6.2.1 View State versus Operational State

View state includes dialog mode, selected bay/session, text inputs, filter, search and planning drafts. Operational state includes bays, sessions and activity. Keeping these categories separate means that opening a form or typing a plate does not itself allocate a bay. Only a successful submit handler changes the operational collection.

The management dialog uses a different state variable from the session dialog. Opening planning closes the session dialog, and changing location closes the session dialog and clears selected view conditions. Native controls keep draft strings until submission, when number conversion and helper validation occur.

This state composition is compact but complex enough to justify future extraction into dedicated hooks and smaller components. Any refactor should preserve the distinction between uncommitted drafts and authoritative local operation results.

## 6.3 Seed Inventory and Managed Location Construction

The default floor definitions are B1/Basement, L1/Ground floor and L2/Upper deck, each with 24 bays. The base fixture assigns selected IDs to occupied or reserved status, while unspecified bays are available. Accessible and EV sets determine physical kind independently. Occupied fixtures receive registrations and generated start times, then become active-session records.

Additional locations use a deterministic arithmetic status roll based on bay index and a seed value. Their registrations are constructed in the accepted TN pattern. This supplies independent sample inventories without a network request. It does not model actual occupancy at the named sites.

**Table 6.2: Initial inventory counts derived from the seed code**

| Location / floor | Total | Occupied | Reserved | Available |
|---|---|---|---|---|
| Riverside Centre — B1 | 24 | 10 | 2 | 12 |
| Riverside Centre — L1 | 24 | 14 | 2 | 8 |
| Riverside Centre — L2 | 24 | 10 | 2 | 12 |
| Riverside Centre — all floors | 72 | 34 | 6 | 32 |
| Marina Mall, Velachery | 72 | 39 | 11 | 22 |
| VR Mall, Anna Nagar | 72 | 37 | 12 | 23 |
| Phoenix Mall, OMR | 72 | 37 | 12 | 23 |

New locations follow a different path. `createManagedLocation` trims and validates a name, checks integer floor and bay bounds, generates floor definitions, and creates an all-available standard inventory. It returns empty sessions and activity. The dashboard supplies duplicate-name checking and inserts the resulting location into both the locations collection and operational map.

The helper's ID uses `Date.now()`, which is convenient for local creation but not a distributed uniqueness strategy. Generated fixture times also vary between module evaluations. These details matter for testing: counts, bounds and relational consistency are stable expectations, while exact time-based IDs and relative ages should not be treated as fixed constants.

## 6.4 Occupancy Calculation and Registration Normalization

`occupancyForLevel` filters the bay array by the requested level, counts occupied and reserved records, derives available by subtraction and rounds the occupied percentage. An unknown level returns a zero total and a zero percentage rather than dividing by zero. The function does not consult the level's declared capacity; it counts actual records [3].

```
return {
  total: levelBays.length,
  occupied,
  reserved,
  available: levelBays.length - occupied - reserved,
  percent: levelBays.length === 0 ? 0
    : Math.round((occupied / levelBays.length) * 100),
};
```

For Riverside B1, occupied percentage is round(10 / 24 × 100), or 42. Reserved bays remain excluded from parked-vehicle occupancy. The dashboard reduces these per-level counts to derive the selected location's total cards. Filtering the visible map does not alter the underlying statistics.

Registration normalization removes whitespace and hyphens, then uppercases. The accepted policy is exactly TN, two digits, two letters and four digits. It is a syntactic project policy, not a lookup against government registration records.

```
const plate = value.replace(/[\s-]/g, "").toUpperCase();
if (!/^TN\d{2}[A-Z]{2}\d{4}$/.test(plate)) {
  throw new Error(
    "Use TN + 2 digits + 2 letters + 4 digits, for example TN01AA1234.",
  );
}
return plate;
```

For example, `tn 01 aa 3042` becomes `TN01AA3042`, while `KA01AA1234` is rejected. Normalization occurs before duplicate checking, so formatting variations do not evade the active-registration rule. Search has a different implementation: it lowercases and trims but does not perform the same full normalization.

## 6.5 Vehicle Check-In Implementation

The `checkIn` helper receives bay/session arrays, target bay ID, registration, vehicle description and start time. It looks up the bay and normalizes the registration before checking availability. This ordering means an invalid plate may be reported before an unavailable-bay error when both inputs are wrong. Error precedence is an implementation detail worth preserving in tests.

```
const bay = bays.find((item) => item.id === bayId);
const normalizedPlate = normalizeTamilNaduPlate(plate);
if (!bay || bay.status !== "available") {
  throw new Error("Choose an available parking bay.");
}
if (sessions.some((session) => session.plate === normalizedPlate)) {
  throw new Error("This vehicle already has an active parking session.");
}
```

After validation, the function constructs a session with the bay and level references, normalized plate, trimmed description or “Vehicle”, and supplied start time. The new ID combines bay ID with `Date.parse(startedAt)`. The helper does not validate that timestamp; UI callers supply `new Date().toISOString()`, but other callers could provide malformed content.

The return value maps the target bay to occupied status while retaining its other properties, then appends the session to the active list. The input arrays are not mutated. The existing test explicitly checks that the original available bay remains unchanged after a successful result.

The dashboard uses the returned collections and prepends an arrival activity record. It then shows a confirmation and closes the dialog. This is a local state commit, not a server acknowledgment. Save failure can occur later in the storage effect without rolling back the visible operation.

The function checks duplicate registrations only in the supplied session array, which the dashboard scopes to one location. It does not enforce a global rule across independent garages. Nor does it verify accessible/EV eligibility, authenticate the operator or open a physical gate. Those are future policy and integration requirements.

## 6.6 Vehicle Check-Out Implementation

`checkOut` finds a session by ID and rejects an unknown ID. It maps the bay referenced by the session to available, clears `plate` and `since`, then filters the session out of the active collection. The bay's kind and identity are preserved.

```
const session = sessions.find((item) => item.id === sessionId);
if (!session) throw new Error("Active parking session not found.");

return {
  bays: bays.map((bay) => bay.id === session.bayId
    ? { ...bay, status: "available", plate: undefined,
        since: undefined }
    : bay),
  sessions: sessions.filter((item) => item.id !== sessionId),
};
```

The dashboard performs an additional selected-session check before calling the helper, then prepends a departure activity item and displays the normalized registration and released bay in a status notice. The session dialog closes after success. On a level with no sessions, a hint is displayed and completion is disabled.

### 6.6.1 Meaning of Departure in the Current Model

A departure ends active occupancy only. It does not retain a completed visit with an end time, calculate duration, charge a fee, generate a receipt or post a financial transaction. Activity is not a substitute for that ledger: it stores a human-readable time string and only the first five items are shown in the panel.

The helper also assumes that the session's bay exists and corresponds to a valid occupancy. If the supplied data is corrupt, a missing bay is not explicitly rejected by the mapping operation. This is another reason to validate restored data and strengthen relationship tests before relying on the prototype operationally.

For the normal valid path, the expected count effect is occupied minus one and available plus one, with total and reserved unchanged. The existing unit test and browser arrival/departure test exercise this relationship. They do not cover all corrupt-state or repeated-submission cases.

## 6.7 Capacity Resizing and Plan Commit

The resizing helper first rejects non-integer values or capacities outside one to ninety-nine. It filters the target level and rejects an unknown/empty level. It counts all non-available bays as protected and refuses a requested capacity below that count. This protects occupied and reserved records without needing to know their vehicle details.

For a reduction, available bays are sorted by descending numeric suffix, and the required number of IDs is removed from the overall array. Protected bays remain even if their suffix is high. For expansion, the helper collects existing suffixes, scans one through ninety-nine and creates unused IDs until it reaches the required difference.

```
const protectedCount = levelBays
  .filter((bay) => bay.status !== "available").length;
if (capacity < protectedCount) {
  throw new Error(
    "Capacity cannot be below the number of occupied or reserved bays.",
  );
}
const difference = capacity - levelBays.length;
if (difference === 0) return bays;
```

New records are standard available bays. The algorithm does not automatically restore an EV or accessible designation for a previously removed suffix. It also does not renumber surviving records. These policies make the behavior deterministic but leave special-kind inventory policy as future work.

The dashboard saves all floor drafts as one logical plan. It computes `nextBays` and `nextLevels` through the selected location's levels before calling setters. If any helper throws, the final commit is skipped and the management error is shown. On success, both location planning information and operational bays are updated, and a total-capacity notice appears.

The method is safe against partial local state updates within that handler's normal execution. It is not an atomic multi-user transaction: another tab can still hold an old plan, and saved JSON has no revision check. A production implementation must enforce the same protection rule against current server data.

## 6.8 Browser Persistence Implementation

The storage implementation uses two effects and the fixed key `parkline-garage-planning`. The first reads a value, parses JSON and checks that a locations array has length and an operations object exists. If accepted, it schedules restoration of locations, operations, selected location and that location's first floor. A catch path removes the saved key, while a finally path enables storage readiness.

The second effect runs after relevant state changes when readiness is true. It serializes locations, operations and selected location ID. A save failure is caught and logged as a browser warning. There is no user-visible “unsaved changes” state and no export or backup workflow.

```
window.localStorage.setItem(STORAGE_KEY, JSON.stringify({
  locations: garageLocations,
  operationsByLocation,
  locationId,
}));
```

The saved format duplicates some data because location objects contain nested bays, sessions and activity while the operational map also contains them. Session handlers update the operational map, so nested location collections may be older. The normal dashboard read prefers the operational map, but inconsistent or incomplete stored values could expose stale fallback data.

The restoration guard is not a schema validator. It does not confirm valid statuses, matching level references, array shapes in every location, unique IDs or session/bay consistency. Even the key-removal operation in the catch path assumes storage access remains usable. More robust recovery would isolate storage failures, validate a versioned schema and choose a single canonical representation.

The browser evidence is mixed rather than uniformly successful. Supplemental arrival and new-location reload checks passed in their exercised sequence, while the existing add/resize/reload scenario returned to `riverside` instead of the added location. This suggests a timing-sensitive restoration problem requiring investigation; it does not justify a claim that every reload fails or that persistence is reliable.

## 6.9 Location Switching, Search and Dialog Coordination

`changeLocation` resolves the requested location, selects its first floor, clears search and status filtering, closes the session dialog and clears the notice. Operational collections are not copied between locations. Instead, subsequent rendering selects the appropriate entry from `operationsByLocation`. Similar bay IDs can therefore exist in different garages without sharing status.

Search trims the query and converts it to lowercase. A bay matches when its ID or optional plate contains that query and its status also matches the selected filter. The calculation applies only to `levelBays`, so a registration on another floor is not found until that floor is selected. When no records match, the grid displays an explicit empty-state message.

Dialog opening depends on context. Arrival selects the clicked bay or the first available current-floor bay; the selector itself lists available bays from the entire location. Departure selects a matching bay's session or the first current-floor session. Close buttons and backdrop clicks dismiss the modal, but no Escape-key listener or focus trap is implemented in the reviewed component.

### 6.9.1 Planning Dialog Coordination

Opening capacity planning closes a session dialog, copies the current capacities into string drafts and resets management errors. Switching to “Add location” changes the management view while preserving number drafts. A successful creation selects the new location and resets the relevant map view conditions. The name field has a UI maximum length, while the reusable helper mainly validates trimming and non-blank content.

A reserved bay click does not open either session form; it displays a reserved notice. Notification, profile and activity-options controls remain presentation-level affordances without full workflows. The activity footer simply shows a notice that recent activity is being displayed.

These interaction details are important for evaluation because a dashboard can contain many clickable controls without implementing the larger service implied by their labels. The screenshot commentary that follows describes only the demonstrated behavior.

## 6.10 Output Screens — Garage Overview

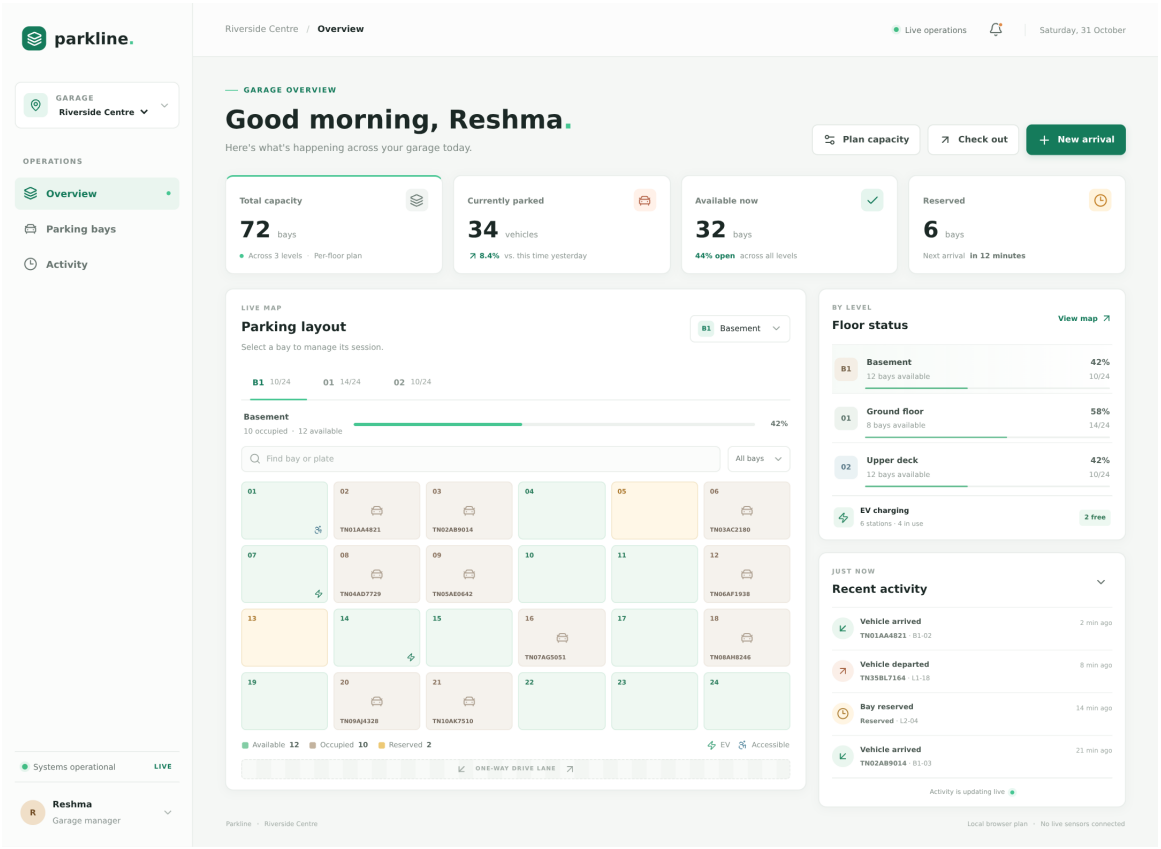



**Figure 6.1: Authentic Riverside overview with derived inventory and sample presentation labels**

The overview shows the initial Riverside inventory: 72 total, 34 occupied, 32 available and six reserved. The basement map contains 24 bays, with a 42% occupied indicator derived from ten occupied records. Floor cards summarize the same underlying bay collection, while recent activity displays fixture events. The sidebar selects the garage and links to sections of this page.

The metric totals are working outputs, but not every label is data-driven. “Live operations”, “8.4% vs. this time yesterday”, the next-arrival message, the manager profile and EV charging summary include static content. No sensor or historical analytics service produced them. The footer correctly identifies a local browser plan and says no live sensors are connected.

This screen is a genuine production-build capture at a 1440-by-1000 CSS-pixel viewport, with full-page capture extending to the content height. It is evidence of the reviewed UI, not a mockup or proof that any named physical garage has adopted Parkline.

### 6.10.1 Floor Selection and Status Filtering

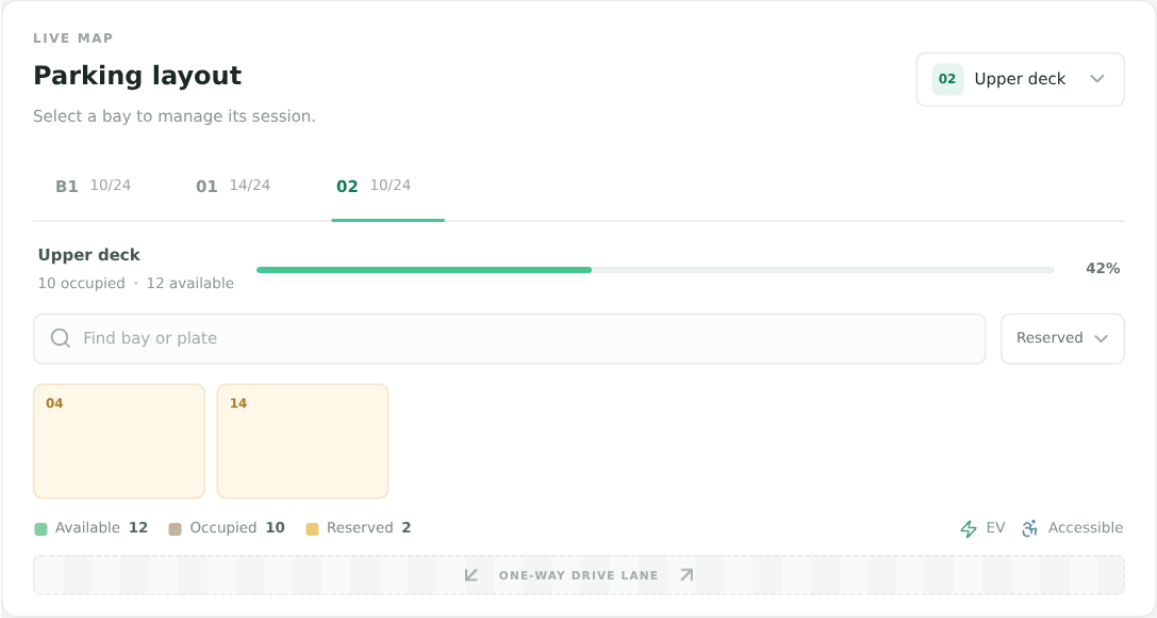



**Figure 6.2: Upper-deck map filtered to its two reserved sample bays**

The capture selects floor 02 and filters the map to reserved bays. Only L2-04 and L2-14 remain visible. The occupancy strip and legend still describe the full level rather than the filtered subset, which preserves the distinction between inventory statistics and search results.

Clicking L2-04 displays a reserved notice and does not open an arrival form. This behavior was exercised by supplemental check B03. Filtering and clicking do not change the reserved collection; there is no reservation creation or cancellation workflow behind the display.

A search can be combined with the filter because both predicates must pass. An unknown query produces “No bays match this search.” Search is local to the current floor and performs a case-insensitive substring match. It does not scan all garages or normalize formatted registrations the same way as arrival validation.

The map is a regular visual grid rather than a scaled drawing. Bay position in the grid is not a surveyed physical coordinate, and the lane decoration does not establish a navigable facility route. The screen is useful for operational demonstration while remaining limited as a genuine floor-planning tool.

### 6.10.2 Vehicle Arrival Form

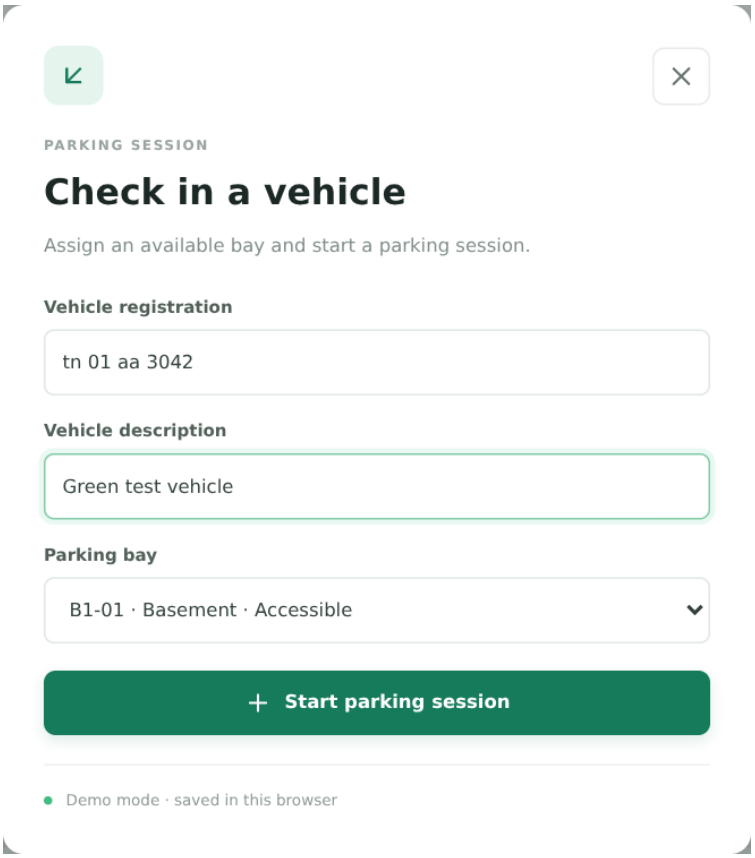



**Figure 6.3: Arrival dialog with a formatted TN registration and an available bay**

The arrival form collects a registration, optional description and available bay. The captured input `tn 01 aa 3042` demonstrates accepted formatting before normalization. The selected bay is B1-01, which is initially available and classified as accessible in the seed data. The current MVP does not collect or enforce accessibility eligibility.

On submission, the domain helper normalizes the plate to `TN01AA3042`, checks availability and active duplication, then returns a changed bay and new session. Supplemental check B05 observed totals of 72 capacity, 35 occupied, 31 available and six reserved after success. Check B06 then observed that arrival state after a same-browser reload in that sequence.

The form does not capture driver identity, contact information, ticket number, tariff, payment method or reservation reference. It does not open a gate. The “Demo mode” note identifies the local-browser scope, and errors leave the dialog open for correction rather than committing a successful transition.

The screenshot shows the actual form, including native controls and accessible visible labels. It does not establish complete modal keyboard behavior, which remains a documented improvement area.

### 6.10.3 Active Session and Departure Form

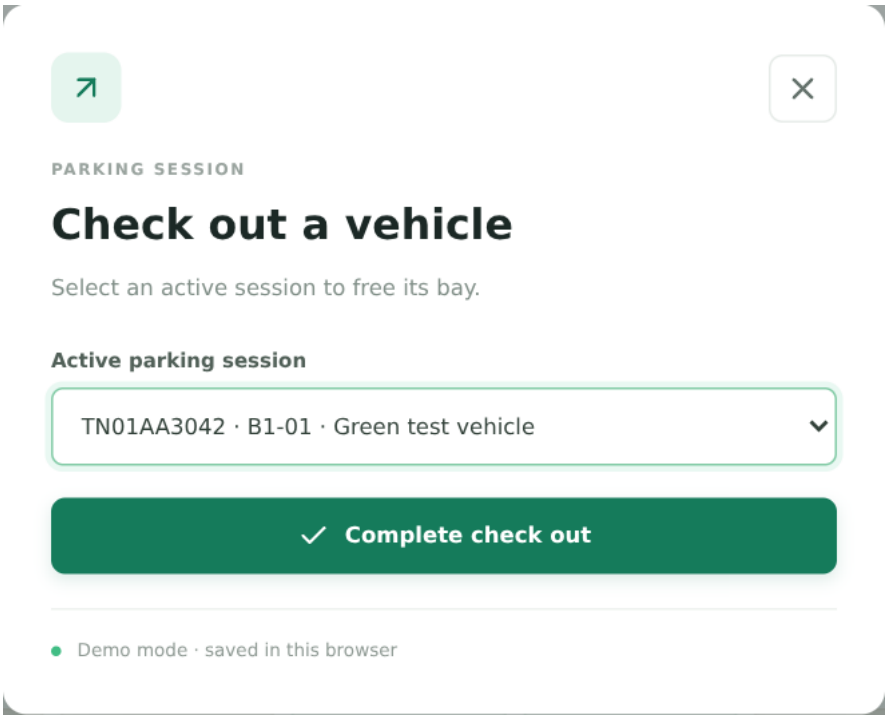



**Figure 6.4: Departure selector for the newly checked-in demonstration vehicle**

The departure form lists active sessions on the selected floor. Each option combines normalized registration, bay ID and vehicle description, helping the operator distinguish sessions. The screenshot selects `TN01AA3042 · B1-01 · Green test vehicle`, created during the same supplemental demonstration.

Completing checkout releases B1-01, removes the active session and restores Riverside totals to 72 total, 34 occupied, 32 available and six reserved. Supplemental check B08 verified both the bay's accessible name and the metric values. The bay's accessible kind remains unchanged because departure clears vehicle fields, not physical classification.

The scope limitation is visible in the selection logic rather than the screenshot alone: sessions from another floor are excluded until the operator changes floors. A level with no sessions displays a hint and disables completion. Unknown IDs are rejected by the helper.

This is an occupancy departure, not a financial checkout. There is no calculation of elapsed fees, no payment approval and no receipt. The prototype also removes the active record instead of moving it into a completed-session archive. Those limitations are central to interpreting the screen accurately.

### 6.10.4 Floor Capacity Planning

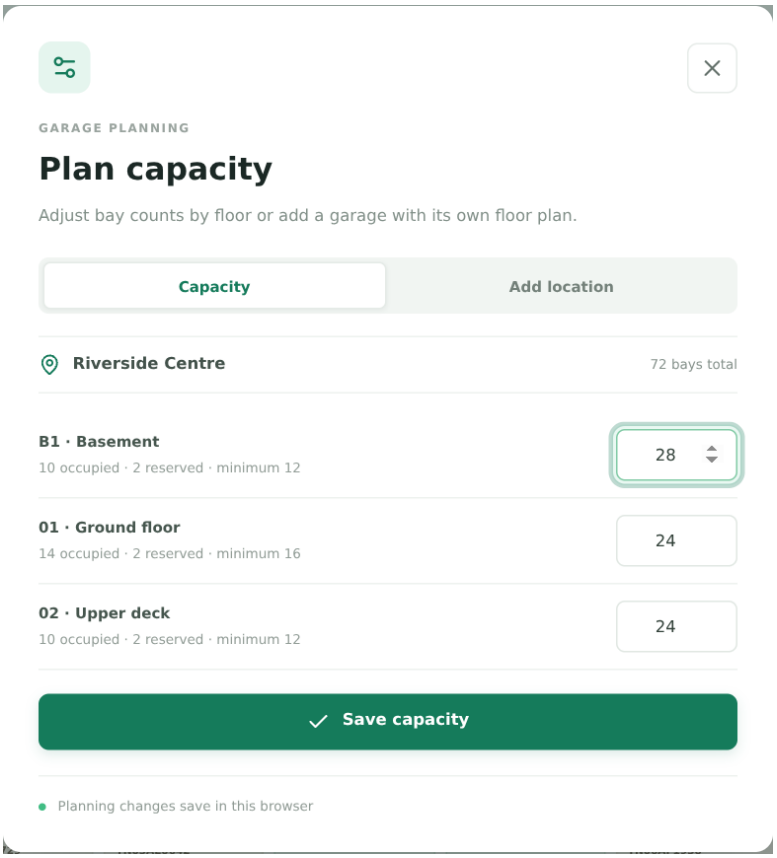



**Figure 6.5: Capacity dialog with occupied/reserved minimums and a draft basement expansion**

The planning dialog exposes one numeric field per existing level. Supporting text shows occupied, reserved and minimum protected counts. In the capture, the basement is drafted from 24 to 28 bays. Saving that plan produced 76 total, 34 occupied, 36 available and six reserved, demonstrating that expansion adds availability without disturbing protected records.

Supplemental check B10 also inspected the basement input's minimum of twelve, corresponding to ten occupied plus two reserved bays. A draft of eleven had the browser's range-underflow validity flag set. The domain helper independently enforces the same protection boundary, so correctness does not depend solely on the HTML minimum.

The form edits bay counts, not the number of floors or an architectural geometry. It cannot mark new bays EV or accessible, and it cannot approve a physical facility's legal capacity. Newly allocated suffixes are standard available records, while protected existing IDs remain stable.

The confirmation notice is evidence of a valid local plan update. It is not proof of durable saving or synchronization with another operator. Persistence is a later browser effect and remains subject to the open reload finding.

### 6.10.5 New Location Creation

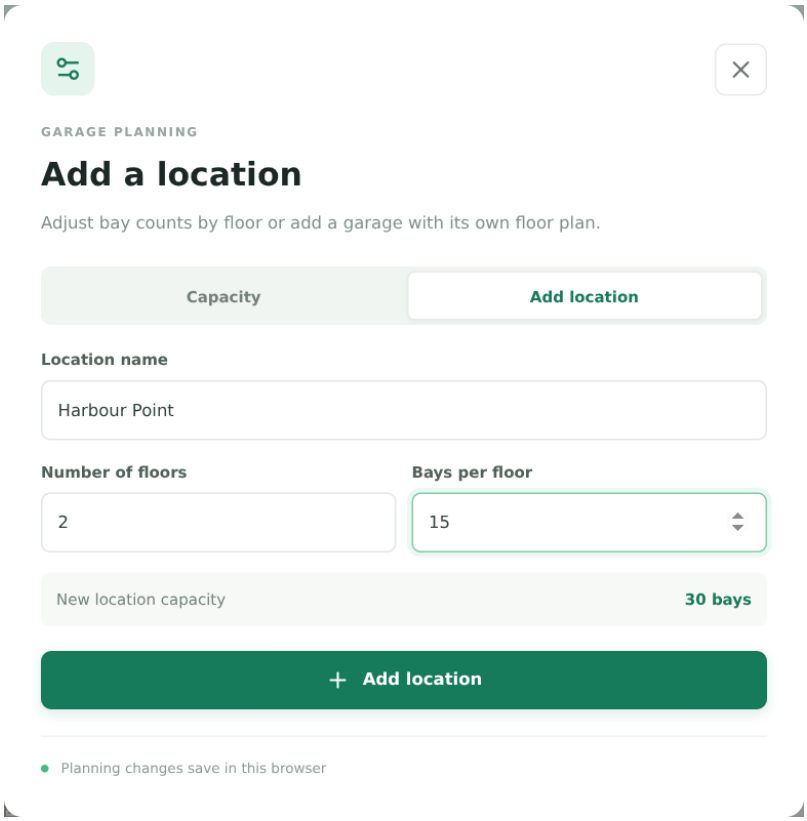



**Figure 6.6: Add-location form configured for Harbour Point with two floors and fifteen bays each**

The add-location tab collects a name, floor count and bays per floor. The preview multiplies the current numeric drafts, producing thirty bays in this example. Submission validates the counts and creates two levels with fifteen standard available bays each. The new location has no active sessions or activity.

The dashboard inserts the location into the selectable collection, creates an operations-map entry and selects its first level. Supplemental check B11 observed `[30, 0, 30, 0]` for total, occupied, available and reserved, then observed the selected name after reload in that specific sequence. A separate original reload scenario failed, so this successful capture must not be generalized into a reliability claim.

The UI rejects case-insensitive duplicate names, including surrounding whitespace after trimming. It limits the name field length, but the pure helper does not enforce list-wide uniqueness or a universal name-length rule. Floor and bay bounds are enforced by the helper independently of the form.

The new location is browser-local. It is not registered with a remote operator, verified as a physical property or made available to another device. The screen demonstrates controlled inventory construction, not enterprise site onboarding.

### 6.10.6 Registration Validation and Error Feedback

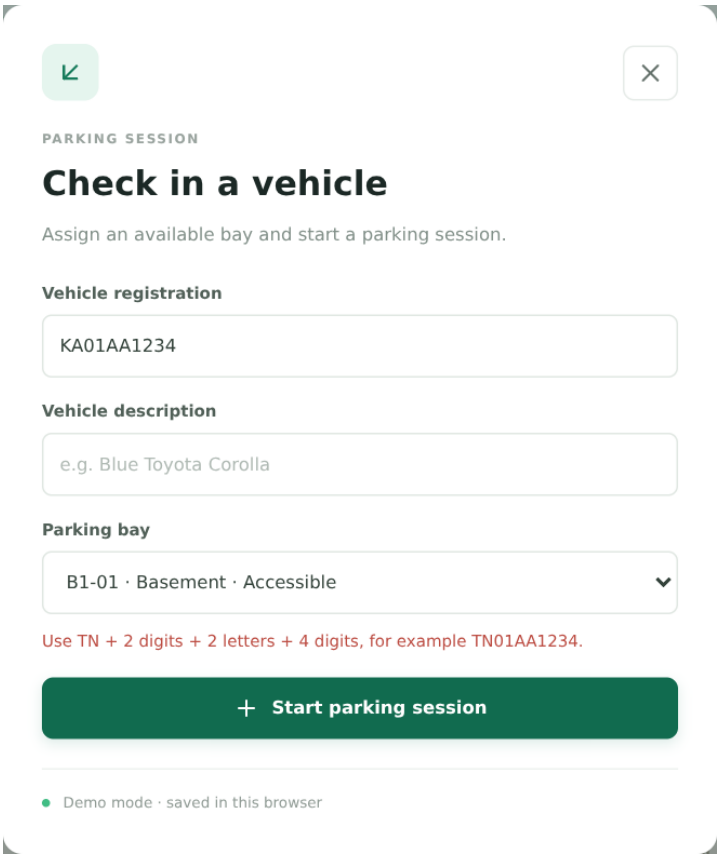



**Figure 6.7: Actual format-validation error for a registration outside the project's TN policy**

The screenshot submits `KA01AA1234`, which is outside the project's TN-only pattern. The form remains open and shows the required format: TN followed by two digits, two letters and four digits. This is an intentional project constraint, not a statement that the entered registration is invalid in its actual state of issue.

The supplemental script also exercised a duplicate active `TN01AA3042` registration. The helper rejected the second arrival even though the selected bay was available. The duplicate screenshot is retained among the source assets, while the report uses this format-error screen to illustrate clear recovery without changing operational state.

The message originates from the domain helper and is shown in an alert-marked paragraph. Native required fields and length/range constraints operate before or alongside domain validation. An empty description is allowed and becomes “Vehicle”; an invalid registration is not silently coerced into a different legal pattern.

Error feedback should be reviewed separately from modal accessibility. The current dialog does not implement a complete focus trap or Escape-key behavior. A future improvement should associate error text more explicitly with the affected field and verify the experience with assistive technology, while preserving the transition guards.

### 6.10.7 Mobile Output and an Observed Layout Defect

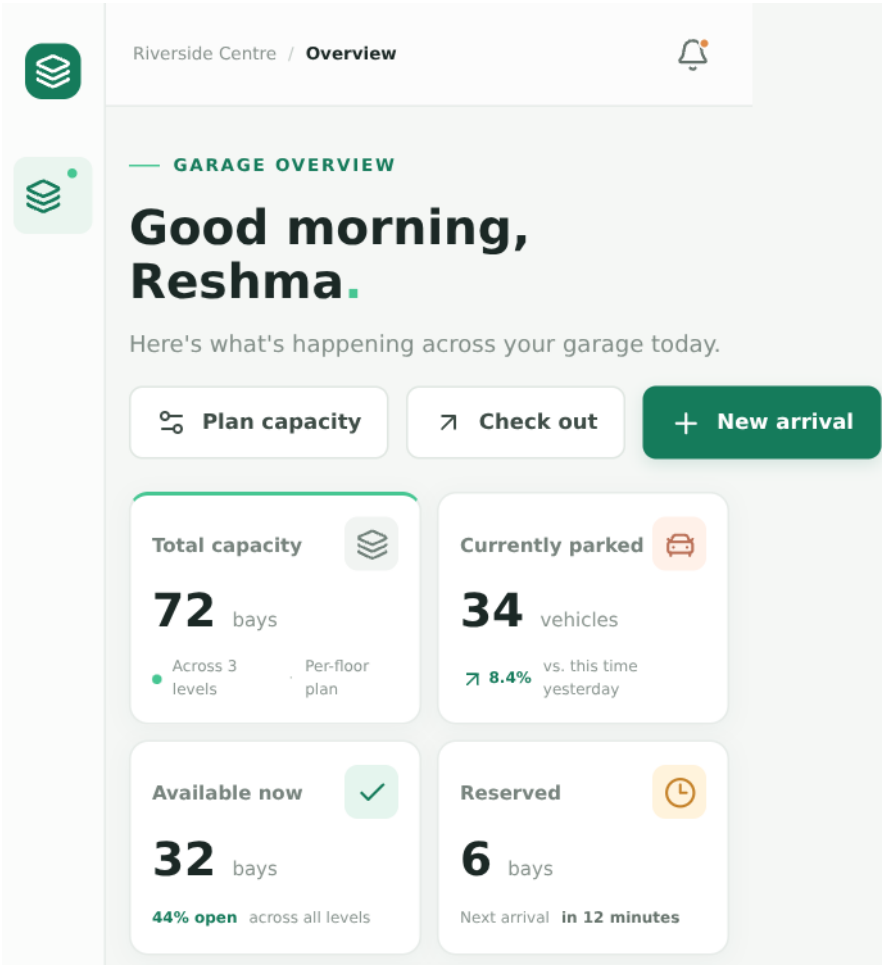



**Figure 6.8: Upper region of the authentic mobile capture, retaining the horizontal-overflow defect**

At a 390-by-844 CSS-pixel viewport, the measured document width was 457 pixels. The primary heading action extended to that right boundary. Supplemental check B15 therefore failed the assertion that the document had no horizontal overflow. The printed upper-region excerpt retains the full captured width, including the overflow, rather than hiding the defect. The uncropped full-page capture is retained with the source assets.

The stylesheet does adapt: the sidebar narrows, cards form two columns, major panels stack and the bay grid uses four columns. Nevertheless, adaptive rules do not guarantee that the action row can shrink sufficiently. The location switcher is also hidden at narrow widths, limiting mobile location changes.

This observation supports a targeted improvement: allow action wrapping or a smaller-width arrangement, preserve access to location selection and test several narrow widths. It does not establish that every mobile device fails or that the entire application is unusable. A broader touch and accessibility review was not performed.

## 6.11 End-to-End Demonstration Reconciliation

The implementation walkthrough concludes by reconciling visible output with domain effects. A correct demonstration should track a single operation across bay status, session membership, metric values and activity. Screens alone can look plausible even when those relationships disagree, so the capture script includes explicit assertions.

**Table 6.3: Supplemental scenario state progression**

| Scenario stage | Total | Occupied | Available | Reserved | Required interpretation |
|---|---|---|---|---|---|
| Initial Riverside | 72 | 34 | 32 | 6 | Seed baseline |
| Arrival at B1-01 | 72 | 35 | 31 | 6 | Capacity conserved; one new active session |
| Departure from B1-01 | 72 | 34 | 32 | 6 | Session removed; bay released |
| Basement expanded to 28 | 76 | 34 | 36 | 6 | Four standard available bays added |
| Harbour Point created | 30 | 0 | 30 | 0 | Independent empty two-floor inventory |
| Switch back to Riverside | 76 | 34 | 36 | 6 | Riverside's own planned state restored in view |

The scenario uses fictional vehicle input and does not perform a payment, reservation or hardware action. It demonstrates that the local operations remain scoped by location and that successful transitions conserve or alter the right quantities. Browser save/restore checks are reported separately because they involve effect timing beyond the pure transition result.

The documentation assets also include the post-arrival overview, the duplicate-validation dialog and the new-location result. These supporting captures can be used during a project demonstration even when not printed as separate full-page figures.

The chapter establishes a source-to-screen chain for the implemented MVP. The next chapter evaluates that chain, reports failures without changing application behavior and identifies which additional tests and controls are required before broader operational use.

# CHAPTER 7
# TESTING AND EVALUATION

## 7.1 Testing Strategy and Evidence Baseline

Testing combines source review, static verification, domain tests, browser workflows and screen/geometry checks. The aim is to establish what was observed at the reviewed revision, not to manufacture an all-green project narrative. The application behavior was not changed to repair findings during this documentation task.

The review baseline is commit `aa4b73016f271a7e0778c6c8bbd6c17346f52346`, with checks recorded on 31 October 2026. Dependencies were installed with `npm ci`. The application was built successfully and served locally in production mode for browser evidence. Chromium 153.0.8010.0 was supplied through an environment-specific launcher because a default browser was not preinstalled. The existing scenario source was unchanged.

**Table 7.1: Recorded verification summary**

| Verification | Result | Evidence boundary |
|---|---|---|
| Existing Vitest suite | 9 passed, 0 failed | Selected domain and fixture cases |
| ESLint | Passed | Configured static checks only |
| TypeScript no-emit check | Passed | Compile-time source consistency |
| Next.js production build | Passed | Compilation and route generation |
| Existing Playwright scenarios | 2 passed, 1 failed | Complete recorded run of three scenarios |
| Supplemental report browser checks | 15 passed, 1 failed | Sixteen separate documentation assertions |
| Uncaught page errors in supplemental sequence | None observed | Not a universal runtime-error guarantee |

The original browser failure occurred in the add-location, resize-capacity and reload scenario. After reload, the garage selector returned `riverside` rather than the expected generated location ID. The supplemental failure was the 390-pixel overflow assertion, with a measured document width of 457 pixels.

A unit-test pass does not override either browser failure. Likewise, a successful local reload in another sequence does not prove restoration reliability. The sections below retain these distinctions and identify untested boundaries, observed issues and proposed acceptance checks.

## 7.2 Existing Unit and Fixture Test Cases

The existing `src/lib/parking.test.ts` file contains seven parking-operation tests and two location tests. It uses small explicit fixtures for transition checks and loops through seeded locations for consistency assertions. All nine cases passed in the recorded unit run [6].

**Table 7.2: Existing Vitest cases and observed results**

| ID | Existing scenario | Main expected condition | Result |
|---|---|---|---|
| UT-01 | Count occupancy without treating reservations as vehicles | 3 total, 1 occupied, 1 reserved, 1 available; unknown-level percent 0 | Pass |
| UT-02 | Check in and normalize registration | TN01AA9876; occupied result; original bay unchanged | Pass |
| UT-03 | Reject duplicate and unavailable bay | Duplicate and availability exceptions | Pass |
| UT-04 | Reject registrations outside TN format | Format error for invalid input | Pass |
| UT-05 | Check out and release the bay | Available status, cleared plate, no active session; missing ID rejected | Pass |
| UT-06 | Expand/reduce capacity safely | Protected occupied/reserved IDs survive | Pass |
| UT-07 | Reject capacity below protection or above limit | Protected minimum and 100-bay errors | Pass |
| UT-08 | Validate configured locations and sample inventory | Names, session/occupied consistency, TN plate shape, independent arrays | Pass |
| UT-09 | Create requested floor/bay counts | Trimmed name, 2 floors, 24 available bays and capacity 12 | Pass |

UT-03 and UT-05 include more than one assertion path, but each remains one test case in the reported count. The report does not inflate the suite by counting every expectation as a separate test.

The unit suite gives useful confidence in the normal local transitions and selected rejection paths. It does not test every integer boundary, malformed timestamp, unknown resize level, special-kind removal policy, corrupted saved value or cross-tab conflict. It also cannot detect a dialog selector problem or a persistence effect race because it does not mount the dashboard in a browser.

The recommended next step is to add focused cases for the open boundaries while retaining the current small fixtures. Property-style assertions, such as capacity conservation after any valid arrival/departure pair, would complement example-based tests without changing the scope of the existing results.

## 7.3 Existing Browser Workflow Results

The existing Playwright file defines three scenarios. The documentation runner pointed them at the built local application and used isolated browser contexts. A complete recorded run executed all three, with two passes and one failure. An earlier launcher setup timeout was treated as environment troubleshooting rather than evidence of a parking-rule defect; the results below refer to the subsequent complete run.

**Table 7.3: Existing Playwright scenarios and final recorded outcomes**

| ID | Scenario and steps | Expected result | Recorded outcome |
|---|---|---|---|
| E2E-01 | Select Marina, VR and Phoenix locations | Selector reflects each requested ID | Pass |
| E2E-02 | Add Harbour Point with 2×15 bays; resize Ground floor to 18; reload | Added location remains selected and Ground floor remains 18 | Fail after reload: selector value `riverside` |
| E2E-03 | Check TN01AA3042 into B1-01 and back out | Confirmation, occupied bay, then released bay | Pass |

E2E-02 reached the creation and capacity-update stages before failing its post-reload selected-location assertion. The recorded expected pattern was a generated `location-` ID. The received value was the seed location ID. This is stronger evidence than a generic statement that “persistence has an issue”, because it identifies the exact user journey and assertion boundary.

The current evidence does not prove a single root cause. Source review suggests that restoration scheduling and the readiness-gated write effect need coordination, and the duplicated saved representations complicate recovery. These are investigation hypotheses, not confirmed fixes. No application code was modified as part of the report.

The supplemental capture sequence successfully reloaded a new location in a different flow, showing that restoration can work in some circumstances. The appropriate conclusion is a reliability concern requiring repeated regression tests, not that persistence always fails or always succeeds.

Future acceptance should repeat fresh-load and saved-load scenarios under varied timing, verify both the selected location and its full operational inventory, and inspect the stored value before and after hydration. Merely increasing a test timeout would not address an incorrect saved or restored state.

## 7.4 Supplemental Documentation Browser Checks

The capture utility exercises additional interactions and records each assertion independently. It continues after an assertion failure so that remaining evidence and screenshots can still be collected. Its zero process exit on completed collection must not be mistaken for all assertions passing.

**Table 7.4: Supplemental browser assertions B01 to B16**

| ID | Assertion | Result |
|---|---|---|
| B01 | Riverside cards equal 72 / 34 / 32 / 6; 24 basement bays | Pass |
| B02 | Upper-deck reserved filter returns two bays | Pass |
| B03 | Reserved-bay click gives a notice, not a session dialog | Pass |
| B04 | Unknown search displays the empty-map message | Pass |
| B05 | Formatted arrival normalizes and changes counts correctly | Pass |
| B06 | Arrived vehicle remains visible after reload in this sequence | Pass |
| B07 | Duplicate active registration is rejected | Pass |
| B08 | Checkout releases B1-01 and restores seed counts | Pass |
| B09 | Capacity expansion adds availability only | Pass |
| B10 | Basement input minimum is 12 and rejects 11 natively | Pass |
| B11 | New Harbour Point has 30 empty bays and reloads in this sequence | Pass |
| B12 | Switching back restores Riverside's own expanded inventory | Pass |
| B13 | Non-TN format displays the domain error | Pass |
| B14 | Trimmed case-insensitive duplicate location name is rejected | Pass |
| B15 | No document-level overflow at a 390-pixel viewport | Fail |
| B16 | No uncaught page errors during the exercised sequence | Pass |

The results establish specific browser observations, not a cross-browser or assistive-technology audit. B15 measured `innerWidth=390` and `documentElement.scrollWidth=457`. The overflowing element identified by the diagnostic was a primary action button in the heading row. The associated full-page mobile screenshot is printed in Chapter 6.

The evidence file also records Chromium version, desktop and mobile viewports, source commit and assertion details. It is retained separately from the existing Playwright result so that original tests and documentation-added checks remain distinguishable.

## 7.5 Boundary, Negative and Recovery Test Design

The following cases are recommended additions. Unless explicitly linked to the executed suite, they were not run as separate acceptance tests in this documentation task. Static inspection can predict a code path, but it is not a substitute for an executed result.

**Table 7.5: Proposed additional boundary and recovery cases — not executed as a new suite**

| Proposed ID | Input / situation | Expected criterion |
|---|---|---|
| PT-01 | Capacity 0, negative, fractional or NaN | Reject without mutating input arrays |
| PT-02 | Capacity exactly protected count | Remove available records only; retain all protected IDs |
| PT-03 | Expand to 99 with gaps in suffixes | Fill unused numbers; produce no duplicate IDs |
| PT-04 | Resize an unknown level | Reject “Parking level not found” |
| PT-05 | Create 1×1 and 10×99 locations | Correct bounds, counts and all-available standard records |
| PT-06 | Create 0/11 floors or 0/100 bays | Reject invalid creation bounds |
| PT-07 | Check in with a malformed timestamp | Define and test a future timestamp-validation policy |
| PT-08 | Checkout session references a missing bay | Reject inconsistent state rather than silently remove session |
| PT-09 | Invalid JSON or incompatible saved schema | Safe recovery without uncaught error or overwrite |
| PT-10 | Repeated saved-plan loads under varied timing | Preserve selected site and canonical operations |
| PT-11 | Two tabs modify the same bay | Document current conflict; later reject stale revisions |
| PT-12 | Storage quota or access denial | Preserve usable UI and expose an unsaved-state warning |

Some rejection branches already exist for PT-01, PT-04 and PT-06, but the current suite does not exercise all values. PT-07, PT-08 and PT-12 include proposed behavior that would need implementation decisions before a passing expectation can be adopted.

A useful testing principle is to preserve the original collection after every rejected operation. Another is to compare identity sets, not just counts, when shrinking capacity. Losing one occupied bay while creating another could preserve a total yet violate operational safety. Boundary tests should therefore inspect protected IDs, session relationships and mutation behavior together.

## 7.6 Static Checks, Build Verification and Evidence Handling

The documentation run executed linting, type checking and a production build after dependency installation. All completed successfully. The build generated the home route and framework not-found route. These results confirm that the reviewed application can be compiled with the resolved toolchain in the tested environment.

```
npm test             9 tests passed
npm run lint         completed successfully
npm run typecheck    completed successfully
npm run build        production compilation completed successfully
Playwright           2 passed, 1 failed
Report browser checks 15 passed, 1 failed
```

The static checks have defined limits. ESLint evaluates configured rules; it does not validate the truth of “LIVE” labels or prove a persistence effect is race-free. TypeScript checks source types, but the saved JSON is asserted into a shape without a full runtime validator. A successful production build does not guarantee correct browser state after reload or eliminate responsive-layout defects.

### 7.6.1 Evidence Preservation

The report source includes machine-readable unit results, existing browser results, supplemental browser results and dependency versions. Screenshots are tied to the same reviewed baseline. The generated report's validation summary records page count, content inventory and layout checks. These artifacts allow a reviewer to inspect how the report's claims were formed.

Generated caches, dependencies and browser binaries are not treated as project deliverables. The report assets contain only the material needed to reproduce or review the document. The source commit identifies the application baseline; documentation files added later do not imply a changed parking implementation.

No test coverage percentage, security score, endpoint latency or field-availability figure is reported because those measurements were not collected. The absence of such numbers is deliberate. A precise statement of executed checks is stronger academic evidence than a fabricated completeness metric.

## 7.7 Security and Privacy Review

The current browser-local design has no authentication, authorization or trusted audit identity. The manager profile is static presentation. Anyone with access to the running page and its browser context can operate the UI or inspect local data. This is a documented MVP boundary, not a vulnerability assessment claiming that all production controls should already exist.

**Table 7.6: Source-level security/privacy assessment and future controls**

| Concern | Current observation | Recommended production control |
|---|---|---|
| Operator identity | No sign-in or role checks | Authenticate users and authorize each operation |
| Registration data | Stored in plain JSON in the browser | Minimize collection, restrict access and define retention |
| Data integrity | Client can edit collections and storage | Server-side validation and transactional constraints |
| Concurrent allocation | No shared authority | Atomic bay/session updates and conflict responses |
| Auditability | Activity lacks actor and structured event time | Append-only, attributed audit events |
| Storage corruption | Shallow restoration checks | Versioned runtime schema and recovery path |
| Credentials/payments | Not collected or integrated | Use compliant providers; never add secrets to local storage |
| Connected devices | No device protocol or identity | Device authentication, stale-data handling and manual override |

The TN regular expression is input validation, not identity verification. It cannot prove that a registration exists, belongs to the arriving vehicle or has authorization to use an accessible space. Likewise, preventing a duplicate in one session array is not a security guarantee against malicious edits or concurrent clients.

No penetration test, dependency vulnerability audit or OWASP certification was performed. The review is based on source structure and exercised behavior. The existing architecture should use fictional data during demonstrations, especially on shared computers. Clearing site data removes local records but is not a centrally governed deletion or audit process.

Production security work should begin with a threat model and data policy, then implement server authority and access controls. Adding a login screen alone would not repair client-side trust or concurrency limitations.

## 7.8 Accessibility and Responsive Evaluation

The source review found labels, native controls, semantic buttons, visible focus styles, bay accessible names and status/error roles. These features provide a useful foundation. The browser capture utility used accessible names for most interactions, which also confirms that key controls can be located semantically in the tested DOM.

The evaluation did not include screen-reader testing, a complete keyboard-only session, contrast measurements or a formal WCAG audit. Therefore, the report does not label the application fully accessible. ARIA roles alone cannot supply missing keyboard behavior or guarantee that focus remains within a modal dialog.

**Table 7.7: Accessibility/responsive evidence and open review items**

| Item | Evidence type | Outcome / next action |
|---|---|---|
| Form labels and bay names | Source and browser selectors | Present in exercised controls |
| Success and error feedback | Source and browser assertions | Status/alert text observed |
| Dialog focus trap and return | Source review | Not implemented; add tested behavior |
| Escape-key dismissal | Source review | Not implemented |
| Tab keyboard pattern | Source review | Add arrow-key and roving-focus behavior |
| Reduced motion | Stylesheet review | Media query exists; no user study |
| 390-pixel overflow | Executed geometry assertion | Failed: document width 457 pixels |
| Narrow-screen garage selection | Stylesheet review | Selector hidden below sidebar breakpoint |
| Contrast and assistive technology | Not executed | Requires separate evaluation |

The overflow defect is localized evidence, not a general statement about all viewports. The desktop layout rendered coherently in the tested capture, while narrow-width heading actions exceeded the viewport. A corrective test should assert both document width and visibility/reachability of primary actions at several widths.

Accessibility acceptance should also verify that opening a dialog moves focus predictably, Tab remains within it, Escape closes it, and focus returns to the invoking control. These behaviors should be tested with native keyboard input rather than inferred from `aria-modal=true`.

## 7.9 Performance and Scalability Reasoning

No response-time benchmark, network load test or large-data experiment was performed. The analysis below is algorithmic reasoning from the implementation, not a measured performance result. No parking API exists to benchmark.

Let B be the selected location's bay count, S its active-session count and L its floor count. Occupancy calculation scans bays to obtain a level and then counts statuses, so its work is linear in B. The dashboard calls that helper repeatedly for totals, tabs and floor cards, producing work that grows roughly with L×B during a render, in addition to map filtering and session selectors.

**Table 7.8: Dominant operation costs inferred from source**

| Operation | Approximate work | Interpretation |
|---|---|---|
| One level occupancy | O(B) | Filters selected-level records and counts states |
| Arrival | O(B + S) | Bay lookup, duplicate scan and immutable updates |
| Departure | O(B + S) | Session lookup, bay mapping and session filtering |
| Capacity shrink | O(B + A log A) | A available target-level bays sorted by suffix |
| Capacity expansion | O(B + 99) | Used-number set and bounded suffix scan |
| Search/filter | O(B_level) | Current-floor substring and status checks |
| Save snapshot | O(serialized state size) | Entire saved object is serialized on relevant changes |

New-location creation is bounded at 990 bays. That limit makes array-based logic plausible for a prototype, but it does not establish acceptable rendering on every device. Local saving is synchronous browser work and stores duplicated structures; larger activity collections can increase serialization cost because the array is not pruned even though only five items are shown.

Future optimization should follow measurement. Possible directions include one-pass aggregate statistics, stable derived collections, bounded activity retention and a canonical persistence representation. Once a server is introduced, pagination, indexing, transaction contention and network behavior become separate evaluation topics.

The current conclusion is modest: the algorithms are understandable and bounded for the intended demonstration, while scalability and responsiveness at maximum inventory remain unmeasured.

## 7.10 Defect Register, Acceptance Review and Evaluation Summary

The defect register separates observed failures from source-level risks. A proposed fix is not described as completed. The report preserves the current application's behavior so that the project team can reproduce the findings and decide on follow-up work.

**Table 7.9: Open findings and suggested acceptance evidence**

| ID | Classification | Finding | Suggested closure evidence |
|---|---|---|---|
| D-01 | Observed browser failure | Added/expanded location not retained as selected after reload | Repeated existing scenario passes with verified stored content |
| D-02 | Observed layout failure | 390-pixel viewport has 457-pixel document width | Width assertions and reachable actions across narrow widths |
| D-03 | Source-level quality gap | Dialog focus lifecycle incomplete | Keyboard and assistive-technology tests |
| D-04 | Source-level integrity gap | Saved JSON lacks complete runtime validation | Versioned schema and corruption/recovery tests |
| D-05 | Architectural limitation | Browser snapshots cannot coordinate operators | Transactional server and stale-write conflict tests |
| D-06 | Presentation limitation | Static values and live labels can imply telemetry | Clear sample labels or removal of unsupported claims |

No formal user acceptance study or college approval was conducted. The acceptance criteria in this report are proposed review criteria derived from the implemented workflows. A supervisor or project evaluator can use the appendices to reproduce the demonstration and inspect the evidence.

The evaluation supports three conclusions. First, core local domain behavior is supported by a passing nine-case suite and successful arrival/departure browser evidence. Second, static verification and compilation are healthy in the tested environment. Third, persistence and mobile quality are not fully accepted because concrete browser checks failed, while production identity, history and concurrency remain outside scope.

This mixed outcome is useful rather than contradictory. It identifies a working educational foundation and a prioritized improvement list. The final chapter uses those findings to conclude the project without overstating readiness or hiding open engineering work.

# CHAPTER 8
# CONCLUSION AND FUTURE ENHANCEMENTS

## 8.1 Conclusion

Parkline demonstrates a coherent browser-based model for multi-level parking operations and capacity planning. The project combines a location-scoped dashboard, floor-level bay inspection, registration normalization, controlled arrivals and departures, safe bay-count resizing and configurable new locations. Its value is the connection between understandable operator interactions and explicit, testable state-transition rules.

The implementation correctly separates several concepts that are easy to confuse. A bay's physical kind is independent of its occupancy status. A reservation is unavailable inventory but not a parked vehicle. A location total is different from the selected floor's map. Operational changes conserve capacity, while planning changes alter inventory without discarding protected records. These distinctions are visible in the data model, domain helpers and output commentary.

The project also demonstrates the importance of evidence-based evaluation. Nine existing unit tests passed, as did linting, type checking and the production build. The recorded browser results were not uniformly green: two original scenarios passed and one reload-persistence scenario failed; fifteen supplemental assertions passed and one mobile overflow assertion failed. Those outcomes support the conclusion that the MVP is functional but still needs quality hardening.

The application must not be presented as a production parking system. Browser local storage is not a shared database, the profile is not authenticated identity, reserved records do not form a reservation workflow and interface “LIVE” labels do not establish sensor connectivity. Payments, durable visit history and multi-operator transactions are absent.

Within those boundaries, the project is a useful academic software-engineering artifact. It provides a small codebase that can be explained from types through algorithms to screens, along with concrete tests and open findings. The next stage should improve reliability and clarity before expanding the domain or introducing physical integrations.

## 8.2 Achievements against Project Objectives

The achievements are assessed against the objectives stated in Chapter 1. A source-backed feature and a fully accepted quality outcome are not the same thing, so the evaluation retains qualifications rather than marking every objective complete by default.

**Table 8.1: Objective-by-objective outcome assessment**

| Objective | Outcome at the reviewed revision | Qualification |
|---|---|---|
| O1 Occupancy visibility | Implemented and demonstrated | Counts describe local records, not a live facility |
| O2 Bay inspection | Implemented with floors, search and filters | Search is current-floor only |
| O3 Registration validation | Implemented and tested | TN syntax policy only; no authority verification |
| O4 Safe arrivals | Local guards and tested successful/rejection paths | No concurrent-client guarantee |
| O5 Departure release | Implemented and demonstrated | No completed-session archive or billing |
| O6 Protected resizing | Implemented and tested | Protection is by status, not minimum special-kind inventory |
| O7 New locations | Implemented and demonstrated | No edit/delete or arbitrary floor geometry |
| O8 Location isolation | Demonstrated in working state | IDs remain scoped by collection |
| O9 Local retention | Save/restore exists | Reload reliability finding remains open |
| O10 Honest verification | Tests and failures recorded | No field deployment or formal acceptance study |

The strongest achievement is the reusable domain core. It permits tests to inspect operations without depending on screen layout and gives a future backend a clear behavioral specification. The visual dashboard then demonstrates how those rules support actual operator tasks.

A second achievement is transparent documentation. The report adapts the college reference format while replacing its unrelated architecture with Parkline's actual stack and data model. Genuine captures, explicit scope exclusions and a defect register make the document useful for review rather than merely lengthy.

A third achievement is a staged development foundation. The project does not need to adopt every production feature at once. Reliability, runtime schema checking and accessible interaction can be improved within the existing MVP before the system is expanded into shared operations.

## 8.3 Limitations of the Current Work

The principal limitation is authority. All working parking state resides in the browser, where it can be cleared, edited or overwritten. The application cannot safely coordinate two gate operators, arbitrate simultaneous arrivals or guarantee durable recovery. A local no-duplicate guard is valuable but cannot serve as a global allocation constraint.

Persistence has a specific observed quality issue. The original add/resize/reload browser scenario did not retain the added location as selected, even though supplemental reloads succeeded in another sequence. The exact root cause remains unconfirmed. The write/read effect ordering, transition scheduling and duplicated stored representations require investigation and repeatable regression evidence.

The model also omits historical lifecycle detail. Checkout removes an active session; it does not preserve departure time or a completed visit. Activity uses human-readable strings rather than structured event times and operator attribution. Consequently, the current records cannot support billing reconciliation, audited dispute resolution or meaningful historical analytics.

The operator interface has unfinished quality areas. Narrow-width actions overflowed in the measured mobile viewport, the garage selector is hidden in the collapsed sidebar and modal focus behavior is incomplete. A formal accessibility audit, cross-browser review and maximum-inventory performance study were not performed.

Scope limitations are equally important. There is no login, role-based authorization, reservation lifecycle, tariff engine, payment integration, sensor connection, gate protocol or number-plate recognition. EV and accessible indicators do not enforce eligibility or manage charging. Named sites and registrations are fixtures, not verified real-world operational data.

Finally, the academic evaluation is code- and execution-based. It includes no stakeholder interviews, field trial, customer survey, measured traffic reduction or financial return analysis. Those methods could be added in a future study, but the present report does not substitute invented numbers for missing evidence.

These limitations do not negate the MVP's educational value. They define where its conclusions stop and provide a concrete roadmap for the next engineering iteration.

## 8.4 Future Enhancements

Enhancements should be ordered by dependency and risk rather than visual prominence. Reliability and information clarity come first, followed by shared authority and only then external integrations. The roadmap below is proposed work; none of the listed production capabilities is claimed as already implemented.

**Table 8.2: Staged enhancement roadmap and acceptance gates**

| Stage | Proposed work | Acceptance gate |
|---|---|---|
| 1 — MVP hardening | Fix restoration/write coordination; canonical saved schema; runtime validation | Repeated reload and corrupt-storage tests pass |
| 1 — Interface quality | Wrap narrow action controls; preserve mobile location selection; complete modal focus | Geometry, keyboard and accessibility review |
| 1 — Honest presentation | Mark or remove static trends, next-arrival and live labels | Every operational claim has a data source |
| 2 — Shared data authority | Server API and durable database; atomic bay/session operations | Concurrent allocation and stale-write tests |
| 2 — Identity and privacy | Authentication, authorization, attribution and retention rules | Threat-model and access-control review |
| 2 — History and recovery | Completed sessions, structured events, backup and restore | Recovery drill and auditable visit lifecycle |
| 3 — Reservation rules | Create/cancel/expire/consume transitions and timing policy | Conflict and expiration tests |
| 3 — Commercial functions | Tariffs, grace periods and compliant payment provider | Idempotent settlement and reconciliation tests |
| 4 — Physical integration | Device identity, stale sensor data, overrides and monitoring | Hardware failure and safe fallback exercises |

A future backend should treat the browser as an untrusted client. It must repeat validation against current state and commit related records atomically. Composite identity or globally unique keys should prevent location-scoped bay IDs from colliding in a shared schema.

Sensor work should begin only after manual-state authority, stale-data handling and override ownership are agreed. Otherwise, a sensor reading and an operator action could compete without a defined resolution policy. Payment work similarly requires an explicit visit lifecycle and idempotent transaction design rather than a button added to the current checkout form.

## 8.5 Academic Contribution and Closing Summary

The project contributes a practical case study in resource-state modeling. It demonstrates how a familiar physical domain can be represented with typed records, explicit guards, immutable updates and derived outputs. Its small size makes the relationship between requirements, code and behavior inspectable, which is valuable in an undergraduate project evaluation.

A second contribution is the treatment of planning as a protected transformation rather than unrestricted deletion. The capacity helper preserves occupied and reserved identities, chooses a deterministic removal order and reuses unused suffixes during expansion. This supplies a clear example of business rules that must remain stable even when the interface changes.

A third contribution is the evaluation method. The report preserves passing checks and failing checks together, distinguishes original tests from supplemental capture assertions and labels unexecuted recommendations. This avoids a common documentation weakness in which every desirable property is written as an achieved result. The evidence boundary is part of the engineering outcome.

### 8.5.1 Recommended Demonstration Narrative

A project presentation can begin with the initial Riverside counts, explain status versus kind, check in a fictional TN registration, show the changed bay and totals, check the vehicle out, expand a floor and create a new location. The presenter should then show the test summary and openly discuss the persistence and mobile findings. This narrative demonstrates both working software and the ability to evaluate it critically.

The final submission should replace candidate and signatory placeholders, confirm institutional wording with the guide and use the PDF as the pagination reference. The certificate and declaration require actual candidate review and institutional approval; a generated document does not supply either.

In conclusion, Parkline is a functional browser-local parking dashboard and a sound foundation for further software-engineering work. Its current achievement is a transparent, testable MVP—not an already deployed secure or sensor-connected parking platform. The proposed roadmap provides a disciplined way to extend that foundation while preserving its core invariants.

# REFERENCES

## Repository and Directly Inspected Sources

[1] Parkline project. **README.md — Parkline: Multi-Level Parking Operations.** Repository `ra9198/multi-level-parking`, reviewed revision `aa4b73016f271a7e0778c6c8bbd6c17346f52346`, 31 October 2026. Describes MVP scope, local persistence, commands and production next steps.

[2] Parkline project. **src/types/parking.ts.** Same reviewed revision. Authoritative declarations for BayStatus, BayKind, ParkingLevel, ParkingBay, ParkingSession, ParkingActivity and ParkingLocation.

[3] Parkline project. **src/lib/parking.ts.** Same reviewed revision. Authoritative implementation of occupancyForLevel, resizeLevelBays, normalizeTamilNaduPlate, checkIn and checkOut.

[4] Parkline project. **src/lib/parking-data.ts.** Same reviewed revision. Initial levels, bay statuses, registrations, sample activity, additional fixture locations and createManagedLocation.

[5] Parkline project. **src/components/parking-dashboard.tsx; src/app/page.tsx; src/app/layout.tsx; src/app/globals.css.** Same reviewed revision. UI behavior, state ownership, local persistence, routes, metadata and presentation.

[6] Parkline project. **src/lib/parking.test.ts; e2e/parking-workflows.spec.ts.** Same reviewed revision. Existing unit/fixture tests and three operator browser scenarios.

[7] Parkline project. **package.json; package-lock.json; tsconfig.json; next.config.ts; vitest.config.ts; playwright.config.ts; eslint.config.mjs.** Same reviewed revision. Dependency declarations, resolved toolchain, compiler and verification configuration.

[8] Next.js project. **Installed Next.js 16.4.3 documentation**, especially App Router server/client component and deployment guidance under `node_modules/next/dist/docs/01-app/`. Official documentation entry point: https://nextjs.org/docs. Installed package materials were inspected directly for this report.

[9] React project. **Managing State; Synchronizing with Effects.** Official learning documentation. https://react.dev/learn/managing-state and https://react.dev/learn/synchronizing-with-effects. Reader resources for state-driven rendering and effects; not a claim of a live documentation retrieval during this task.

[10] Microsoft. **The TypeScript Handbook.** https://www.typescriptlang.org/docs/handbook/intro.html. Reader resource for interfaces, union types and compile-time checking. Runtime storage-validation limitations were established from the project code, not inferred as solved by static types.

## Standards, Official Tool Guidance and Books

[11] WHATWG. **HTML Living Standard — Web Storage.** https://html.spec.whatwg.org/multipage/webstorage.html. Normative reader resource for browser key/value storage and its scope.

[12] World Wide Web Consortium. **Web Content Accessibility Guidelines (WCAG) 2.2.** W3C Recommendation. https://www.w3.org/TR/WCAG22/. Evaluation framework only; this report does not certify conformance.

[13] OWASP Foundation. **HTML5 Security Cheat Sheet — Local Storage.** https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html. Reader resource for browser-storage trust and sensitive-data considerations.

[14] Vitest project. **Vitest Guide and Configuration Reference.** https://vitest.dev/guide/ and https://vitest.dev/config/. Tool documentation; executed unit outcomes are preserved in the accompanying result file.

[15] Microsoft. **Playwright Test Documentation.** https://playwright.dev/docs/intro. Tool guidance for browser scenarios, assertions and isolation. The environment-specific launcher used for this review is described in the evidence notes.

[16] Tailwind Labs. **Tailwind CSS v3 Documentation.** https://v3.tailwindcss.com/docs/installation. Version-family reader resource corresponding to the project's v3 dependency.

[17] Lucide project. **Lucide React Guide.** https://lucide.dev/guide/packages/lucide-react. Reader resource for the SVG icon component library.

[18] Sommerville, Ian. **Software Engineering.** 10th edition. Pearson, 2016. General reference for requirements, architecture, verification and project feasibility.

[19] Fowler, Martin. **Refactoring: Improving the Design of Existing Code.** 2nd edition. Addison-Wesley, 2018. General reference for maintainable decomposition and behavior-preserving change.

[20] OpenJS Foundation / Node.js project. **Node.js Documentation.** https://nodejs.org/docs/latest-v22.x/api/. Runtime reader resource. The observed review environment was Node.js 22.22.3; the repository's stated minimum remains 20.9.

[21] User-supplied college reference. **IMS_BCA_Project_Report.pdf**, `Example Docs/`, 115 pages. Used only as a formatting and report-organization reference. Its healthcare architecture, author identity, signatory names and implementation claims are not evidence about Parkline.

External links above are bibliographic reader resources. No external paper experiment, vendor benchmark or field survey is claimed. Source-level implementation claims in this report are grounded in items [1]–[8] and the accompanying executed evidence.

# APPENDIX A
# SAMPLE SOURCE CODE

## A.1 Complete Domain Type Declarations

These complete declarations from `src/types/parking.ts` describe the reviewed record shapes and vocabularies. They are not a database schema or runtime validator.

```
export type BayStatus = "available" | "occupied" | "reserved";
export type BayKind = "standard" | "ev" | "accessible";

export interface ParkingLevel {
  id: string;
  label: string;
  name: string;
  capacity: number;
}

export interface ParkingBay {
  id: string;
  levelId: string;
  status: BayStatus;
  kind: BayKind;
  plate?: string;
  since?: string;
}

export interface ParkingSession {
  id: string;
  bayId: string;
  levelId: string;
  plate: string;
  vehicle: string;
  startedAt: string;
}

export interface ParkingActivity {
  id: string;
  type: "arrival" | "departure" | "reservation";
  plate: string;
  bayId: string;
  time: string;
}

export interface ParkingLocation {
  id: string;
  name: string;
  levels: ParkingLevel[];
  bays: ParkingBay[];
  sessions: ParkingSession[];
  activity: ParkingActivity[];
}
```

An occupied bay's optional fields and session relationship must still be validated at runtime. Location scope is carried by the containing collection, not by a `locationId` field in every bay.

## A.2 Occupancy and Registration Helpers

These complete helper functions are reproduced from `src/lib/parking.ts`. Occupancy derives availability from actual bay records, while normalization supplies one accepted representation before duplicate checking. Neither helper performs network access.

```
export function occupancyForLevel(bays: ParkingBay[], levelId: string) {
  const levelBays = bays.filter((bay) => bay.levelId === levelId);
  const occupied = levelBays.filter((bay) => bay.status === "occupied").length;
  const reserved = levelBays.filter((bay) => bay.status === "reserved").length;

  return {
    total: levelBays.length,
    occupied,
    reserved,
    available: levelBays.length - occupied - reserved,
    percent: levelBays.length === 0 ? 0 : Math.round((occupied / levelBays.length) *
    100),
  };
}

export function normalizeTamilNaduPlate(value: string): string {
  const plate = value.replace(/[\s-]/g, "").toUpperCase();
  if (!/^TN\d{2}[A-Z]{2}\d{4}$/.test(plate)) {
    throw new Error("Use TN + 2 digits + 2 letters + 4 digits, for example
    TN01AA1234.");
  }
  return plate;
}
```

The zero-level branch avoids a non-finite percentage. The regular expression accepts a project-specific Tamil Nadu format only. It does not verify legal registration existence or ownership, and it does not accept every real-world vehicle-registration variant.

The distinction between status and kind is preserved because counting reads status only. A reserved accessible bay is reserved inventory; an occupied EV bay is one parked vehicle. Derived percentages should not be substituted with static trend labels elsewhere in the UI.

## A.3 Complete Local Check-In Transition

The following function is reproduced from `src/lib/parking.ts`. Its returned arrays form the next local state. The dashboard is responsible for committing those arrays, generating activity and saving locally.

```
export function checkIn(
  bays: ParkingBay[],
  sessions: ParkingSession[],
  bayId: string,
  plate: string,
  vehicle: string,
  startedAt: string,
): { bays: ParkingBay[]; sessions: ParkingSession[] } {
  const bay = bays.find((item) => item.id === bayId);
  const normalizedPlate = normalizeTamilNaduPlate(plate);
  if (!bay || bay.status !== "available") throw new Error("Choose an available parking
    bay.");
  if (sessions.some((session) => session.plate === normalizedPlate)) {
    throw new Error("This vehicle already has an active parking session.");
  }

  const session: ParkingSession = {
    id: `session-${bayId}-${Date.parse(startedAt)}`,
    bayId,
    levelId: bay.levelId,
    plate: normalizedPlate,
    vehicle: vehicle.trim() || "Vehicle",
    startedAt,
  };

  return {
    bays: bays.map((item) => item.id === bayId
      ? { ...item, status: "occupied", plate: normalizedPlate, since: startedAt }
      : item),
    sessions: [...sessions, session],
  };
}
```

The caller supplies `startedAt`; this helper does not validate its date syntax. Duplicate detection is restricted to the supplied active-session collection. UI callers scope that collection to the selected location, so this is not a global multi-garage or concurrent-operator guarantee.

## A.4 Complete Protected Capacity-Resizing Helper

This complete implementation from `src/lib/parking.ts` is the authoritative local resizing rule. It protects status rather than bay kind, keeps surviving IDs stable and fills unused numeric suffixes on expansion.

```
export function resizeLevelBays(bays: ParkingBay[], levelId: string, capacity: number):
    ParkingBay[] {
  if (!Number.isInteger(capacity) || capacity < 1 || capacity > 99) {
    throw new Error("Capacity must be between 1 and 99 bays per level.");
  }

  const levelBays = bays.filter((bay) => bay.levelId === levelId);
  if (levelBays.length === 0) throw new Error("Parking level not found.");

  const protectedCount = levelBays.filter((bay) => bay.status !== "available").length;
  if (capacity < protectedCount) {
    throw new Error("Capacity cannot be below the number of occupied or reserved
    bays.");
  }

  const difference = capacity - levelBays.length;
  if (difference === 0) return bays;

  if (difference < 0) {
    const removable = [...levelBays]
      .filter((bay) => bay.status === "available")
      .sort((left, right) => Number(right.id.split("-").at(-1)) -
    Number(left.id.split("-").at(-1)))
      .slice(0, Math.abs(difference));
    const removedIds = new Set(removable.map((bay) => bay.id));
    return bays.filter((bay) => !removedIds.has(bay.id));
  }

  const usedNumbers = new Set(levelBays.map((bay) => Number(bay.id.split("-").at(-1))));
  const additions: ParkingBay[] = [];
  for (let number = 1; additions.length < difference && number <= 99; number += 1) {
    if (usedNumbers.has(number)) continue;
    additions.push({
      id: `${levelId}-${String(number).padStart(2, "0")}`,
      levelId,
      status: "available",
      kind: "standard",
    });
  }
  if (additions.length !== difference) throw new Error("Unable to allocate bay numbers
    for this capacity.");
  return [...bays, ...additions];
}
```

A production service must enforce the same rule against current authoritative data and an approved facility-capacity policy. This helper alone cannot synchronize multiple browsers.

## A.5 Checkout and an Existing Protection Test

The complete checkout helper below is reproduced from `src/lib/parking.ts`. The test excerpt is an existing case from `src/lib/parking.test.ts`, not a newly invented passing result.

```
export function checkOut(
  bays: ParkingBay[],
  sessions: ParkingSession[],
  sessionId: string,
): { bays: ParkingBay[]; sessions: ParkingSession[] } {
  const session = sessions.find((item) => item.id === sessionId);
  if (!session) throw new Error("Active parking session not found.");

  return {
    bays: bays.map((bay) => bay.id === session.bayId
      ? { ...bay, status: "available", plate: undefined, since: undefined }
      : bay),
    sessions: sessions.filter((item) => item.id !== sessionId),
  };
}
```

```
it("resizes capacity without removing occupied or reserved bays", () => {
    const expanded = resizeLevelBays(bays, "B1", 5);
    expect(expanded.filter((bay) => bay.levelId === "B1")).toHaveLength(5);
    expect(expanded.find((bay) => bay.id === "B1-02")?.status).toBe("occupied");
    const reduced = resizeLevelBays(expanded, "B1", 3);
    expect(reduced.filter((bay) => bay.levelId === "B1")).toHaveLength(3);
    expect(reduced.map((bay) => bay.id)).toContain("B1-02");
    expect(reduced.map((bay) => bay.id)).toContain("B1-03");
  });
```

The test inspects protected IDs as well as capacity. It does not test every boundary or corrupted session/bay relationship. The original suite's total remains nine cases, and its results are reported separately from supplemental browser checks.

# APPENDIX B
# OPERATIONAL GUIDE AND PROPOSED EXTENSION

## B.1 Installation and Demonstration Setup

This guide reproduces the current browser-local demonstration. Use fictional vehicle information. Do not treat the application as the sole operational record of a real garage, because it lacks shared authority, authenticated access and durable recovery.

1. Install Node.js 20.9 or newer and npm. The report run used Node.js 22.22.3.

2. Open the repository and install the locked dependencies with `npm ci`.

3. Run the unit, lint and type checks. Build the application before production serving.

4. Start the application and open its served address in a modern browser.

5. Confirm the local-browser/no-sensor footer and the initial Riverside metrics.

```
npm ci
npm test
npm run lint
npm run typecheck
npm run build
npm run start -- --hostname 0.0.0.0 --port 3000
```

For ordinary development, `npm run dev` is also available. If serving through a remote preview, use the preview's address in the browser; a user's device cannot reach the sandbox by calling its own localhost. Browser tests require a configured Playwright browser and operating-system libraries in addition to npm dependencies.

**Table B.1: Current route and interaction inventory — no parking API exists**

| Address / anchor | Purpose | Security interpretation |
|---|---|---|
| `/` | Dashboard and all dialogs | No login or role gate |
| `/#overview` | Main overview section | Same page, not another service |
| `/#parking-layout` | Bay map section | Same client state |
| `/#activity` | Recent activity section | Not a historical audit endpoint |

The browser may restore an earlier local plan, so initial counts can differ if this origin already has saved data. For a clean fictional-data demonstration, use a fresh browser profile or clear only this application's site data after confirming that no wanted local records will be lost. There is no implemented export, backup or undo workflow.

## B.2 Operator Guide — Inspect, Arrive and Depart

### B.2.1 Inspect the Correct Scope

Select the garage first, then the floor. Read the capacity cards for the selected location and the occupancy strip for the selected level. Use the legend to distinguish available, occupied and reserved. EV and accessible icons indicate kind, not separate availability states. Search applies to the current floor only; change floors if a registration is not found.

### B.2.2 Start a Demonstration Session

1. Click an available bay or choose “New arrival”.
2. Enter a fictional TN registration, for example `TN01AA3042`.
3. Optionally enter a short vehicle description.
4. Check the selected bay; the dropdown can include other levels of this location.
5. Submit and confirm the registration/bay notice, occupied bay and changed totals.
6. If an error appears, correct the input. Do not assume a session began merely because the dialog was opened.

### B.2.3 Complete a Departure

Select the floor containing the active session. Click “Check out” or its occupied bay, choose the matching registration/bay/description and complete checkout. Confirm the bay is available again. No payment or receipt follows because neither is implemented.

**Table B.2: Common operator messages and appropriate response**

| Message or symptom | Meaning | Response |
|---|---|---|
| TN format error | Input does not match the project policy | Correct spacing/content; other-state formats remain outside scope |
| Already active session | Normalized plate already exists at this location | Locate the existing session rather than allocate again |
| Choose an available bay | Target is missing or not available | Select a currently available option |
| No active sessions on this level | Departure list is floor-scoped | Switch to the relevant floor |
| No bays match this search | Current predicates return no records | Clear the query/filter or select another floor |
| Bay is reserved | Sample reserved inventory is protected | Do not attempt ordinary arrival; no reservation workflow exists |

A success notice confirms a local transition, not durable server storage. Reload behavior has an open finding, so preserve the distinction during demonstrations and evaluation.

## B.3 Operator Guide — Capacity, Locations and Local Data

Open “Plan capacity” to edit existing floor bay counts. Each row shows occupied and reserved totals and the protected minimum. Enter a whole number within one to ninety-nine and do not reduce below that minimum. Saving computes a complete valid plan before committing it. Expansion creates standard available bays; reduction removes highest-numbered available bays and retains protected IDs.

To add a site, choose the “Add location” tab, enter a non-blank unique name, choose one to ten floors and one to ninety-nine bays per floor, review the preview and submit. New locations begin empty and standard. The current UI does not edit a location name, delete a location or change the number of floors after creation.

### B.3.1 Saved Data and Safe Reset

Planning and session changes are serialized under `parkline-garage-planning` for the current origin. The data is not synchronized to another computer. A different preview hostname is a different origin and may therefore show a different plan. Clearing site data discards the browser copy; no server backup will restore it.

If a clean demonstration is required, first confirm that only fictional disposable data is present. Close other tabs for this origin, then use browser site-data controls to clear this application and reload. Do not clear unrelated sites or assume there is an undo. For investigation, inspect the saved JSON before resetting so that a persistence defect is not erased without evidence.

**Table B.3: Demonstration and submission checklist**

| Review item | Required action |
|---|---|
| Candidate identity | Replace student name and register-number placeholders |
| Institutional wording | Confirm degree, department and guide details with the institution |
| Certificate/signatures | Obtain actual approval; no generated endorsement is valid |
| Data privacy | Use fictional registrations and avoid shared-device personal records |
| Test presentation | Retain both passing results and open findings |
| Pagination | Use the generated PDF as the fixed-page reference |
| Editable copy | Review Word layout after fonts or content are changed |

The documentation tooling can regenerate the report from source and metadata. Rebuilding the document does not fix application defects or automatically update institutional approval. Both require separate review.

## B.4 Proposed Shared-Service Design — Not Implemented

This future-design sketch has no implemented API routes, tables, migrations or access-control handlers. Operational rules and privacy requirements must be agreed before implementation.

**Table B.4: Proposed authoritative records and invariants — future design only**

| Proposed record | Possible identity / fields | Invariant to enforce |
|---|---|---|
| Location | Stable location key, name, revision | Authorized site scope and unique policy-defined name |
| Level | Location key plus level key, approved capacity | Capacity agrees with approved active inventory |
| Bay | Location-scoped unique key, level, kind, status, revision | One current authoritative state per physical bay |
| Visit | Stable key, bay, plate, arrival, departure, lifecycle | At most one active visit per bay and scoped registration |
| Operational event | Event key, actor, location, event time, action | Attributed append-only history with defined retention |

### B.4.1 Proposed Transaction Boundary

A future arrival request should authenticate an operator, authorize the location, normalize and validate input, acquire or conditionally update the current bay, check active-registration uniqueness and commit the visit plus bay state atomically. A stale client should receive a conflict and refresh authoritative state rather than overwrite another operator's work. Departure should preserve completed history and be safe under repeated submission.

Capacity reduction should verify current protected occupancy inside the same transaction as inventory modification. Browser-calculated counts are informative, not authoritative. If a future API accepts an expected revision or idempotency key, the server must define its exact semantics and test retries and conflicts.

### B.4.2 Proposed Integration Gates

Authentication, database recovery and privacy controls should precede payments or sensors. Reservation work needs explicit timing and state-transition policy. Sensor work needs device identity, stale-data detection and manual override ownership. Payment work needs tariff rules, idempotency and reconciliation. These are separate acceptance gates, not capabilities created by this appendix.

This is an enhancement sketch only; the documented implementation remains the browser-local frontend/domain prototype.